Chương 3

Từ và vectơ từ

Từ là đơn vị cơ bản của ngôn ngữ [Jackendoff, 1992]. Hầu hết các hệ thống ngôn ngữ tự nhiên dùng để diễn đạt và giao tiếp đều liên quan đến việc tạo ra, cấu thành và kết hợp các từ. Trước khi tìm hiểu cách các từ tạo nên những đơn vị ngôn ngữ lớn hơn, ta nên hiểu trước *từ* là gì. Điều này dẫn tới hai câu hỏi nền tảng:

- Hình thức bề mặt của một từ là gì?
- Nghĩa của một từ là gì?

Tuy nhiên, trả lời các câu hỏi này không dễ, vì không có quy tắc đơn giản nào mô tả đầy đủ cách các từ được cấu tạo, hay cách nghĩa của chúng được xác định và suy ra. Trong khi nhiều lý thuyết ngôn ngữ học khác nhau đã đề cập đến những câu hỏi này, các nhà nghiên cứu NLP lại quan tâm nhiều hơn đến hai vấn đề thực tiễn:

- **Tách token** (*tokenization*): Cho một chuỗi ký tự, làm thế nào để phân đoạn nó thành một chuỗi các từ (còn gọi là các *token*) sao cho các đơn vị này có thể dùng được trong các tác vụ NLP phía sau?
- **Học biểu diễn từ** (*word representation learning*): Cho một kho ngữ liệu (*corpus*), làm thế nào để biểu diễn mỗi từ dưới dạng có thể tính toán được, và các mô hình NLP thực hiện tính toán trên biểu diễn đó như thế nào?

Một mục tiêu của chương này là trình bày cách các câu được phân đoạn bằng phương pháp ngôn ngữ học hoặc phương pháp thống kê. Cụ thể, chúng tôi mô tả một số cách tách một chuỗi ký tự thành các từ hoặc

từ con (*subword*), dùng các quy tắc kinh nghiệm hoặc các mô hình thống kê học từ dữ liệu. Mục tiêu còn lại là cho thấy cách biểu diễn các từ bằng các vectơ số thực. Chúng tôi trình bày các phương pháp hiện đại để học và đánh giá các vectơ từ (*word vector*) này. Trọng tâm của phần này không phải là việc suy diễn công thức và mô hình, mà là khái niệm cốt lõi về biểu diễn từ bằng vectơ — nền tảng của rất nhiều hệ thống NLP. Trong các chương sau, ta sẽ thấy ý tưởng này được tổng quát hóa để mô hình hóa những đơn vị ngôn ngữ lớn hơn, chẳng hạn như câu.

3.1 Tách token

Trong khoa học máy tính và các lĩnh vực liên quan, thuật ngữ *token* có thể được dùng theo nhiều cách khác nhau. Ở đây ta đơn giản coi một token là một *từ* theo nghĩa ngôn ngữ học, dù nó cũng có thể là thứ khác (xem Mục 3.1.4). Trong NLP, **tách token** (*tokenization*) hay **phân đoạn** (*segmentation*) là một tác vụ liên quan đến phân tích hình thái (*morphological analysis*) [Aronoff and Fudeman, 2011]. Trong khi các bộ phân tích hình thái thường được dùng để nghiên cứu cấu trúc bên trong của từ, việc tách token quan tâm đến cách các câu được chia nhỏ thành các từ. Có vẻ như ta cần biết từ được cấu tạo ra sao nếu muốn biết câu được tạo thành từ các từ như thế nào. Sự phức tạp này còn tăng thêm bởi tính đa dạng của các ngôn ngữ, khiến việc xây dựng một hệ thống chung để mô tả hình thái học trở nên khó khăn. Ví dụ, các ngôn ngữ đơn lập (*analytic language*) (như tiếng Trung) có rất ít biến tố (*inflection*) và dựa vào trật tự từ để truyền đạt ý nghĩa. Ngược lại, các ngôn ngữ tổng hợp (*synthetic language*) (như tiếng Pháp) có thể có hệ thống biến tố phong phú, và nghĩa của một từ chịu ảnh hưởng lớn bởi hình thái.

Chú giải của người dịch: Nghĩa của các câu ví dụ trong Hình 3.1

Câu tiếng Trung có nghĩa: "Từ trước đến nay, dịch máy hoàn hảo là một trong những ước mơ của nhân loại." Câu tiếng Nhật đại ý: "Vùng Tây Nhật Bản và biển trời quang đãng, nắng nóng đến đổ mồ hôi." Câu tiếng Anh: "Cô ấy nói: 'Học sâu không phải là lời giải cho mọi vấn đề của thế giới'." Chú ý trong tiếng Anh, *'s* (biểu thị sở hữu) được tách khỏi *world* thành một token riêng.

Mặt khác, việc chia câu thành các đơn vị ngôn ngữ nhỏ hơn là quan

Tiếng Trung	Đầu vào:	一直以来，完美的机器翻译是人类的梦想之一。
	Đầu ra:	一直/以来/，/完美/的/机器翻译/是/人类/的/梦想/之一/。
Tiếng Nhật	Đầu vào:	西日本や海はく晴れて、汗ばむ暑さとなる。
	Đầu ra:	西日本/や/海/は/く/晴れて/、/汗ばむ/暑さ/と/なる/。
Tiếng Anh	Đầu vào:	She said, “Deep learning is not the solution to all world’s problems”.
	Đầu ra:	She/said/,/“/Deep/learning/is/not/the/solution/to/all/world/’s/problems/”/.

Hình 3.1: Tách token cho các ngôn ngữ khác nhau (dấu gạch chéo = ranh giới từ). Với tiếng Trung và tiếng Nhật, nơi không có dấu phân cách giữa các từ, việc tách token thường được gọi là *phân đoạn từ* (*word segmentation*).

trọng trong nhiều tác vụ NLP, kể cả khi nhiều ngôn ngữ trên thế giới có rất ít hiện tượng hình thái. Chẳng hạn, tiếng Trung là một ngôn ngữ đơn giản về hình thái và không có ranh giới từ tường minh. Dù việc lấy các chữ (ký tự) làm đơn vị để hiểu nghĩa văn bản tiếng Trung cũng hợp lý, nhưng khi xử lý văn bản, xét các đơn vị lớn hơn lại là lựa chọn đáng mong muốn và hợp lý hơn. Lưu ý rằng, ngay cả với những ngôn ngữ có dấu phân cách giữa các từ như tiếng Anh, ta vẫn phải tách token các câu để chúng được chuẩn hóa khi làm đầu vào và/hoặc đầu ra của một hệ thống NLP.

Chú giải của người dịch: Còn tiếng Việt thì sao?

Tiếng Việt cũng là một ngôn ngữ đơn lập như tiếng Trung: từ không biến đổi hình thức (không có kiểu *learn/learns/learning*), ngữ pháp dựa chủ yếu vào trật tự từ và hư từ (*đã, đang, sẽ, của*…). Điểm đặc biệt là chữ viết tiếng Việt dùng dấu cách để phân tách *âm tiết*, không phải *từ*. Ví dụ câu “Học sinh học sinh học” có 5 âm tiết, nhưng nếu tách từ đúng ta được “Học_sinh / học / sinh_học” (3 từ: *học sinh*, *học*, *sinh học*). Vì vậy, chỉ tách theo dấu cách là chưa đủ; tách từ tiếng Việt cũng là một bài toán phân đoạn có nhập nhằng, gần với tiếng Trung hơn là tiếng Anh. Các công cụ tách từ tiếng Việt thường nối các âm tiết của một từ bằng dấu gạch dưới như trên.

Trong mục này, chúng tôi bỏ qua việc bàn chính xác *từ* là gì trong hình thái học và cú pháp học, mà đơn giản coi tách token là tác vụ thêm ranh giới từ hoặc ranh giới token vào một chuỗi cho trước (xem Hình 3.1). Chúng tôi sẽ cho thấy một câu có thể được chia thành các từ hoặc token theo cách kinh nghiệm hoặc theo cách thống kê. Lưu ý rằng quá trình này được thiết kế để tạo ra những đơn vị giúp việc xử lý ngôn ngữ trong các

hệ thống NLP dễ dàng hơn, chứ không nhất thiết phải có ý nghĩa chặt chẽ về mặt ngôn ngữ học.

3.1.1 Tách token bằng quy tắc và kinh nghiệm

Một cách tách token phổ biến và đơn giản là nhận diện mọi từ trong câu bằng cách áp dụng một tập quy tắc định sẵn. Nhìn chung, các quy tắc này xuất phát từ ngôn ngữ học và phản ánh hiểu biết có sẵn của chúng ta về hình thức của một từ. Ví dụ, xét câu tiếng Anh trong Hình 3.1. Ta có thể định nghĩa các quy tắc sau để tách token câu này:

- Từ không chứa dấu cách. Do đó, ta có thể chia câu thành các "ứng viên từ" dựa vào dấu cách.
- Mọi ứng viên từ chỉ gồm các chữ cái tiếng Anh (tức a–z và A–Z) là một từ.
- Mọi dấu câu (ví dụ dấu ngoặc kép, dấu phẩy, dấu chấm) phải được tách riêng thành một từ.
- *'s* được coi là một token riêng, biểu thị danh từ sở hữu.

Đây có lẽ là tập quy tắc tối thiểu ta có thể dùng để tách token tiếng Anh. Tất nhiên, có thể thêm nhiều quy tắc hơn để bao quát thêm các hiện tượng ngôn ngữ, ví dụ từ có dấu gạch nối, từ chứa chữ cái không thuộc tiếng Anh, v.v. Tuy nhiên, không có tiêu chuẩn nào để định nghĩa một tập quy tắc như vậy. Trong thực tế, đặc biệt là trong các ứng dụng NLP, ta muốn có một tập quy tắc tối thiểu xử lý được phần lớn các vấn đề, và việc tách token thường được cài đặt bằng một số biểu thức chính quy (*regular expression*). Ở đây chúng tôi không bàn chi tiết về các quy tắc và biểu thức chính quy này, mà giới thiệu bạn đọc một số giáo trình để đọc thêm [Lawson, 2003; Friedl, 2006; Jurafsky and Martin, 2008][1].

[1] Các script tách token có thể tìm thấy trong nhiều dự án mã nguồn mở, chẳng hạn Moses [Koehn và cộng sự, 2007] (`https://github.com/moses-smt/mosesdecoder/blob/master/scripts/tokenizer/tokenizer.perl`) và các bộ tách token trong SacreBLEU (`https://github.com/mjpost/sacrebleu/tree/master/sacrebleu/tokenizers`).

Chú giải của người dịch: Biểu thức chính quy

Biểu thức chính quy là một cách viết "mẫu" để máy tính tìm các đoạn văn bản khớp với mẫu đó. Ví dụ, mẫu `[a-zA-Z]+` nghĩa là "một hoặc nhiều chữ cái Latinh liên tiếp", còn mẫu `[,.!?]` khớp với một dấu câu bất kỳ trong bốn dấu đã liệt kê. Áp dụng lần lượt vài mẫu như vậy lên câu là ta có ngay một bộ tách token đơn giản.

Ngoài ra, người ta thường *chuẩn hóa* văn bản trước khi tách token để đầu vào của bộ tách token ở dạng chuẩn tắc. Ví dụ, với tiếng Anh và các ngôn ngữ dùng chữ cái khác, chuẩn hóa (*normalization*) hay chính tắc hóa (*canonicalization*) là quá trình chuyển các từ về chữ thường, chuẩn hóa cách biểu diễn ký tự (ví dụ các ký tự Unicode), v.v. Hơn nữa, ta có thể ánh xạ các dạng khác nhau của một từ về cùng một dạng để việc tách token khái quát hơn. Một cách đơn giản là gộp mọi dạng biến tố của một từ về dạng cơ bản của nó. Trong ngôn ngữ học, dạng cơ bản của một từ được gọi là **từ nguyên dạng** (*lemma*), và quá trình ánh xạ các từ về từ nguyên dạng được gọi là **đưa về nguyên dạng** (*lemmatization*). Dưới đây là một số ví dụ:

$$\begin{aligned} \textit{learn} &\rightarrow \textit{learn} \\ \textit{learning} &\rightarrow \textit{learn} \\ \textit{learns} &\rightarrow \textit{learn} \\ \textit{best} &\rightarrow \textit{good} \end{aligned}$$

Có những từ ứng với hai hay nhiều từ nguyên dạng khác nhau (thường có từ loại (*part-of-speech*) khác nhau). Khi đó, ta phải chọn từ nguyên dạng đúng tùy theo ngữ cảnh. Nói cách khác, việc đưa về nguyên dạng phụ thuộc vào ngữ cảnh.

Có liên hệ chặt chẽ với việc đưa về nguyên dạng là **lấy gốc từ** (*stemming*), tức biểu diễn một từ bằng *gốc từ* (*stem*) của nó. Giống từ nguyên dạng, gốc từ cũng là một dạng cơ bản nào đó của từ. Tuy nhiên, khác với từ nguyên dạng, gốc từ không nhất thiết phải là một từ hợp lệ, dù có nhiều từ có từ nguyên dạng và gốc từ trùng nhau. Một điểm khác nữa so với đưa về nguyên dạng là việc lấy gốc từ được thực hiện trên từng từ riêng lẻ, không cần ngữ cảnh để khử nhập nhằng. Vì vậy, lấy gốc từ không phụ thuộc ngữ cảnh. Có một số thuật toán hiệu quả để lấy gốc từ. Một thuật toán phổ biến là *cắt hậu tố* (*suffix stripping*) [Porter, 1980]. Nó

đơn giản loại bỏ các hậu tố *ing*, *ed*, *ion*, v.v., như dưới đây:

$$\begin{aligned} \textit{remove} &\rightarrow \textit{remov} \\ \textit{removing} &\rightarrow \textit{remov} \\ \textit{removal} &\rightarrow \textit{remov} \\ \textit{best} &\rightarrow \textit{best} \end{aligned}$$

Để có thêm ví dụ, Hình 3.2 trình bày kết quả chuẩn hóa, đưa về nguyên dạng và lấy gốc từ cho một câu tiếng Anh.

Câu gốc	She said, “Deep learning is not the solution to all world’s problems”.
Chuẩn hóa	she said, “deep learning is not the solution to all world’s problems”.
Tách token	she/said/,/“/deep/learning/is/not/the/solution/to/all/world/’s/problems/”/.
Đưa về nguyên dạng	she/say/,/“/deep/learning/be/not/the/solution/to/all/world/’s/problem/”/.
Lấy gốc từ	she/said/,/“/deep/learn/is/not/the/solut/to/all/world/’s/problem/”/.

Hình 3.2: Chuẩn hóa, đưa về nguyên dạng và lấy gốc từ cho một câu tiếng Anh. Khi chuẩn hóa, toàn bộ câu được chuyển thành chữ thường. Khi đưa về nguyên dạng, mỗi từ được viết lại thành từ nguyên dạng của nó. Khi lấy gốc từ, hậu tố của một số từ bị loại bỏ.

Chú giải của người dịch: Nguyên dạng khác gốc từ thế nào?

Trong Hình 3.2, *said* (quá khứ của *say* — “nói”) được đưa về nguyên dạng *say*, *is* về *be* (“là”), vì bộ đưa về nguyên dạng “biết” từ điển. Còn bộ lấy gốc từ chỉ máy móc cắt đuôi: *solution* thành *solut* — không phải từ có thật — nhưng lại để nguyên *said*. Tiếng Việt hầu như không cần hai bước này vì từ không biến đổi hình thức: “học” luôn là “học”, dù ở thì nào hay ngôi nào.

Cần lưu ý rằng các phương pháp trên thường được cài đặt bằng biểu thức chính quy, tra từ điển và một số kinh nghiệm bổ sung. Dù phần tổng quan ngắn gọn này có thể khiến ta nghĩ rằng tách token là việc đơn giản, nhưng cần nhiều công sức hơn nữa để nó dùng được trong thực tế. Đặc biệt, khi xử lý các ngôn ngữ có hệ chữ viết không dùng bảng chữ cái, hoặc các ngôn ngữ không có khoảng trắng tường minh giữa các từ, việc tách token trở thành một bài toán phức tạp, và khi đó dùng các quy tắc đơn giản không phải là chiến lược tốt. Trong các tiểu mục tiếp theo, chúng tôi sẽ phát biểu lại việc tách token như một bài toán học máy (*machine learning*), trong đó cách tách token hay phân đoạn câu được học từ dữ liệu. Các phương pháp này không phụ thuộc vào ngôn ngữ và có thể áp dụng cho nhiều bài toán tách token hoặc các bài toán dạng phân đoạn.

3.1.2 Tách token như bài toán mô hình hóa ngôn ngữ

Giờ hãy chuyển sang mô hình hóa thống kê cho bài toán tách token. Để tiện trình bày, tiểu mục này tập trung vào các hệ chữ viết không có ranh giới từ tường minh, nhưng các phương pháp này có thể tổng quát hóa cho các ngôn ngữ có dấu phân cách. Gọi $\mathbf{x} = x_1 \cdots x_l$ là một chuỗi ký tự, và $\mathbf{y} = y_1 \cdots y_m$ là một chuỗi các từ hoặc token. Ta nói $\mathbf{y}$ là một kết quả tách token của $\mathbf{x}$ nếu $\mathbf{y}$ xác định một cách phân đoạn trên $\mathbf{x}$. Xét câu tiếng Trung sau:

$$\mathbf{x} = \text{机器翻译是人类的梦想之一。}$$

Ta có thể định nghĩa một cách phân đoạn cho câu này, chẳng hạn[2],

$$\begin{aligned}\mathbf{y} &= \text{机器翻译/是/人类/的/梦想/之一/。} \\ &= \left[\text{“机器翻译” “是” “人类” “的” “梦想” “之一” “。”}\right] \end{aligned} \tag{3.1}$$

(nghĩa: “Dịch máy / là / nhân loại / của / ước mơ / một trong / .”, tức “Dịch máy là một trong những ước mơ của nhân loại.”). Theo cách này, tách token có thể được phát biểu như bài toán ánh xạ $\mathbf{x}$ thành $\mathbf{y}$. Cho một chuỗi đầu vào, đầu ra là cách phân đoạn có khả năng xảy ra cao nhất:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \Pr(\mathbf{y}|\mathbf{x}) \\ &= \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}|\mathbf{x}).\end{aligned} \tag{3.2}$$

Phương trình (3.2) mô tả một mô hình dự đoán mà ta đã nhắc đến nhiều lần trong sách này. Tuy nhiên, bài toán ta đang xét dễ hơn, vì mọi cách phân đoạn hợp lệ $\mathbf{y}$ đều sinh ra đúng chuỗi $\mathbf{x}$ một cách tất định. Do đó, việc tìm cách phân đoạn có khả năng nhất được đơn giản hóa thành việc cực đại hóa xác suất tiên nghiệm (*prior probability*) $\Pr(\mathbf{y})$:

$$\begin{aligned}\hat{\mathbf{y}} &= \arg\max_{\mathbf{y}} \log \Pr(\mathbf{y}) \\ &= \arg\max_{\mathbf{y}} \log \Pr(y_1, \cdots, y_m).\end{aligned} \tag{3.3}$$

[2]Theo ký hiệu đã dùng trước đây, chúng tôi dùng cả $\mathbf{y} = y_1 \cdots y_m$ và $\mathbf{y} = \begin{bmatrix} y_1 & \cdots & y_m \end{bmatrix}$ để ký hiệu một chuỗi biến.

Chú giải của người dịch: Vì sao $\Pr(\mathbf{y}|\mathbf{x})$ chỉ còn $\Pr(\mathbf{y})$?

Ký hiệu $\arg\max_{\mathbf{y}} f(\mathbf{y})$ nghĩa là "giá trị $\mathbf{y}$ làm $f(\mathbf{y})$ lớn nhất" (khác với max là *giá trị lớn nhất* của f). Theo định nghĩa xác suất có điều kiện, $\Pr(\mathbf{y}|\mathbf{x}) = \Pr(\mathbf{x},\mathbf{y})/\Pr(\mathbf{x})$. Mẫu số $\Pr(\mathbf{x})$ không phụ thuộc $\mathbf{y}$ nên không ảnh hưởng đến việc chọn $\mathbf{y}$ tốt nhất. Còn khi $\mathbf{y}$ là một cách tách của $\mathbf{x}$ thì ghép các từ của $\mathbf{y}$ lại chắc chắn được $\mathbf{x}$, nên $\Pr(\mathbf{x},\mathbf{y}) = \Pr(\mathbf{y})$. Hàm log đồng biến nên lấy log không làm đổi vị trí cực đại; ta dùng log để biến tích các xác suất nhỏ thành tổng, dễ tính hơn.

Dễ kiểm tra rằng Phương trình (3.3) thực ra mô tả một bài toán mô hình hóa ngôn ngữ (*language modeling*). Có vài cách khác nhau để ước lượng xác suất đồng thời $\Pr(y_1, \cdots, y_m)$. Một cách đơn giản là viết lại $\log \Pr(y_1, \cdots, y_m)$ thành tổng của các logarit xác suất có điều kiện:

$$\log \Pr(y_1, \cdots, y_m) = \log \Pr(y_1) + \log \Pr(y_2|y_1) + \cdots + \log \Pr(y_m|y_1, \cdots, y_{m-1}). \tag{3.4}$$

Mỗi xác suất có điều kiện $\Pr(y_i|y_1, \cdots, y_{i-1})$ có thể được xấp xỉ bằng

$$\Pr(y_i|y_1, \cdots, y_{i-1}) \approx \Pr(y_i|y_{i-n+1}, \cdots, y_{i-1}), \tag{3.5}$$

tức là việc sinh ra y_i chỉ phụ thuộc vào $n-1$ từ ngữ cảnh đứng trước. Để tính $\Pr(y_i|y_{i-n+1}, \cdots, y_{i-1})$, ta có thể dùng phương pháp tần suất tương đối hoặc mạng nơ-ron (xem Chương 2).

Chú giải của người dịch: Ví dụ nhỏ với $n = 2$

Với $n = 2$ (mô hình *bigram*), mỗi từ chỉ phụ thuộc vào đúng một từ đứng trước. Khi đó, với câu ba từ, $\log \Pr(y_1, y_2, y_3) \approx \log \Pr(y_1) + \log \Pr(y_2|y_1) + \log \Pr(y_3|y_2)$. Giả sử $\Pr(y_1) = 0.1$, $\Pr(y_2|y_1) = 0.5$, $\Pr(y_3|y_2) = 0.2$ thì xác suất của câu là $0.1 \times 0.5 \times 0.2 = 0.01$. Ta tính điểm như vậy cho từng cách tách có thể có và chọn cách có điểm cao nhất.

Bây giờ ta có thể coi tách token là một bài toán học có giám sát (*supervised learning*). Quy trình được phác thảo như sau:

- Chuẩn bị một số câu đã được phân đoạn đúng.
- Học một mô hình ngôn ngữ (*language model*) $\Pr(\mathbf{y})$ trên các câu có nhãn này.

- Với một câu mới, tìm cách tách token "tốt nhất" $\hat{\mathbf{y}}$ làm cực đại $\Pr(\hat{\mathbf{y}})$, như trong Phương trình (3.3).

Dù quy trình này tuân theo các bước chuẩn của học có giám sát, vẫn có vài vấn đề thực tế ta phải giải quyết. Thứ nhất, mô hình ngôn ngữ cần một bộ từ vựng (*vocabulary*) định sẵn để tính xác suất cho mỗi token y_i, nhưng từ mới thì luôn xuất hiện. Để xử lý chúng, một cách là phân đoạn chuỗi con chưa biết thành các ký tự, tức là ta coi các ký tự là từ nếu chuỗi con sinh ra các ký tự đó không có trong bộ từ vựng. Một cách khác là xét mọi chuỗi con không được bộ từ vựng bao phủ, và thay chúng bằng thẻ <unk>. Mẹo <unk> được dùng rộng rãi trong các mô hình ngôn ngữ tiên tiến nhất (*state-of-the-art*) và thường có ích.

Thứ hai, mô hình ngôn ngữ mô tả ở trên có thiên hướng ưu tiên các chuỗi ngắn, vì $\Pr(y_1, \cdots, y_m)$ sẽ lớn nếu m nhỏ. Một cách chung để giảm thiên lệch này là đưa vào mô hình một *phần thưởng độ dài* (*length reward*, hay *length bonus*), chẳng hạn

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}} \; \log \Pr(\mathbf{y}) + \lambda \cdot m, \tag{3.6}$$

hoặc

$$\hat{\mathbf{y}} \;=\; \arg\max_{\mathbf{y}} \; \frac{\log \Pr(\mathbf{y})}{m^{\lambda}}, \tag{3.7}$$

trong đó $\lambda \cdot m$ và m^{λ} thưởng cho các chuỗi dài, và $\lambda > 0$ là một siêu tham số (*hyperparameter*) điều khiển mức độ ta dựa vào phần thưởng khi đánh giá độ tốt của $\mathbf{y}$. Điều thú vị là trong thực tế, người ta nhận thấy thiên lệch độ dài không phải vấn đề lớn với tách token, vì độ dài của các kết quả tách token "tốt" ít dao động. Chẳng hạn, dùng một mô hình ngôn ngữ unigram (tức $n = 1$) không có phần thưởng độ dài nào vẫn cho kết quả tốt trong nhiều ứng dụng thực tế. Ta sẽ thấy vài ví dụ ở Mục 3.1.4.

Chú giải của người dịch: Vì sao mô hình ưa chuỗi ngắn?

Mỗi xác suất đều nhỏ hơn 1, nên mỗi lần nhân thêm một thừa số, tích lại nhỏ đi (tương ứng, mỗi số hạng log đều âm nên tổng càng nhiều số hạng càng âm). Ví dụ nếu mỗi từ có xác suất khoảng 0.1 thì cách tách thành 2 từ được điểm $0.1^2 = 0.01$, còn cách tách thành 4 từ chỉ được $0.1^4 = 0.0001$. Như vậy cách tách ít từ (từ dài) bị ưu ái "một cách không công bằng". Số hạng $\lambda \cdot m$ cộng thêm điểm cho mỗi từ để bù lại.

Thứ ba, việc thực hiện arg max là khó, vì số cách tách token ứng viên tăng theo hàm mũ. Tuy nhiên, việc dùng mô hình ngôn ngữ ở đây cho phép dùng các thuật toán tìm kiếm hiệu quả. Chẳng hạn, xét việc áp dụng mô hình ngôn ngữ unigram vào tách token. Với chuỗi đầu vào $\mathbf{x}$, tại mỗi vị trí j của $\mathbf{x}$, ta lưu một *trạng thái* mô tả xác suất của cách tách token tốt nhất trên $x_1 \cdots x_j$ (ký hiệu $p(j)$) cùng với từ cuối cùng của cách tách này. Tại vị trí $j+1$, ta tạo một trạng thái mới và tính xác suất của cách tách token tốt nhất trên $x_1 \cdots x_{j+1}$ bằng

$$\begin{aligned} p(j+1) &= \max_{1 \le i \le j} p(i) \cdot \Pr(x_{i+1} \cdots x_{j+1}) \\ &= \max_{1 \le i \le j} p(i) \cdot \Pr(w_{[i+1,j+1]}), \end{aligned} \tag{3.8}$$

trong đó $\Pr(w_{[i+1,j]})$ là xác suất của từ trải trên đoạn $x_{i+1} \cdots x_j$. Về mặt thuật toán, Phương trình (3.8) mô tả một phương pháp quy hoạch động (*dynamic programming*) có độ phức tạp thời gian $O(l^2)$ với đầu vào độ dài l. Để có đầu ra cuối cùng, ta có thể lần ngược từ trạng thái cuối và xuất ra chuỗi từ dọc theo đường đi của cách tách token tối ưu.

Chú giải của người dịch: Quy hoạch động qua ví dụ

Xét chuỗi 3 ký tự abc. Các cách tách có thể: abc; a/bc; ab/c; $a/b/c$ — tổng cộng $2^{l-1} = 4$ cách (mỗi khe giữa hai ký tự có thể cắt hoặc không), tăng theo hàm mũ khi l lớn. Quy hoạch động tránh liệt kê hết: ta tính lần lượt $p(1), p(2), p(3)$, trong đó $p(j)$ là điểm của cách tách tốt nhất cho j ký tự đầu. Muốn tính $p(3)$, ta chỉ cần thử "từ cuối cùng" là c, bc hay abc, rồi ghép với $p(2)$, $p(1)$ hay $p(0) = 1$ (chuỗi rỗng) đã tính sẵn. Mỗi vị trí thử tối đa l khả năng, nên tổng cộng cỡ l^2 phép tính — đó là ý nghĩa của ký hiệu $O(l^2)$.

Lưu ý rằng các phương pháp ở đây mang tính tổng quát và có thể áp dụng để tách token cho các ngôn ngữ khác. Chẳng hạn, khi áp dụng cho tiếng Anh, ta chỉ cần điều chỉnh nhẹ định dạng đầu vào: đầu vào không phải là chuỗi ký tự mà là chuỗi các mảnh nhỏ nhất có thể, được tách ra bởi dấu câu và dấu cách. Ví dụ, với câu *Is this Tom's laptop?* ("Đây có phải máy tính xách tay của Tom không?"), ta có

$$\mathbf{x} = \Big[\text{Is} \quad \text{this} \quad \text{Tom} \quad \text{'} \quad \text{s} \quad \text{laptop} \quad ?\Big]. \tag{3.9}$$

Sau đó, quá trình tách token có thể tiến hành như trong Phương trình (3.2–3.7).

3.1.3 Tách token như bài toán gán nhãn chuỗi

Một cách phổ biến để phân đoạn các ký tự thành từ là gán cho mỗi ký tự trong chuỗi một nhãn lấy từ một tập định sẵn, quá trình này được gọi là gán nhãn chuỗi (*sequence labeling*). Dù ở đây ta tập trung vào bài toán phân đoạn từ, khuôn khổ này cũng áp dụng được cho nhiều tác vụ NLP khác, chẳng hạn gán nhãn từ loại (*part-of-speech tagging*) và nhận dạng thực thể có tên (*named entity recognition*). Vì ý tưởng gán nhãn chuỗi đã được bàn ở Chương 1, ở đây chúng tôi trình bày cách điều chỉnh nó cho tác vụ tách token.

Các tập nhãn dùng trong tách token thường đơn giản. Một ví dụ là tập "IB". Nhãn "I" chỉ một mảnh ngôn ngữ nằm *bên trong* một từ (*inside*), còn nhãn "B" chỉ *đầu* một từ (*beginning*). Tập nhãn có thể được làm phong phú hơn bằng cách thêm nhãn "E" (tức *cuối* một từ, *ending*) và/hoặc tách nhãn "B" thành các nhãn con (ví dụ B_1 và B_2 chỉ mảnh ngôn ngữ thứ nhất và thứ hai của một từ) [Zhao và cộng sự, 2006]. Cho một chuỗi đầu vào $\mathbf{x}$ và một kết quả tách token $\mathbf{y}$, việc chuyển $\mathbf{y}$ thành chuỗi nhãn khá đơn giản. Xét lại ví dụ đã dùng ở tiểu mục trước. Ta có thể gán nhãn chuỗi theo các định dạng khác nhau:

x:	机	器	翻	译	是	人	类	的	梦	想	之	一	。
y:	机器		翻译		是	人类		的	梦想		之一		。
{I,B}	B	I	I	I	B	B	I	B	B	I	B	I	B
{I,B,E}	B	I	I	E	B	B	E	B	B	E	B	E	B
{I,B_1,B_2,E}	B_1	B_2	I	E	B_1	B_1	B_2	B	B_1	B_2	B_1	B_2	B_1

Chú giải của người dịch: Gán nhãn B/I cho tiếng Việt

Với tiếng Việt, ta gán nhãn cho từng *âm tiết* thay vì từng chữ Hán. Chẳng hạn câu "Học sinh học sinh học" được tách thành "học_sinh / học / sinh_học" tương ứng chuỗi nhãn {I,B}: *Học*/B, *sinh*/I, *học*/B, *sinh*/B, *học*/I. Biết chuỗi nhãn là khôi phục được ngay cách tách: mỗi nhãn B mở đầu một từ mới. Vì vậy bài toán tách từ trở thành bài toán "đoán nhãn cho từng vị trí".

Vì chuỗi nhãn có thể được xem như một dạng khác của kết quả tách token, ta có thể phát biểu lại bài toán là tìm chuỗi nhãn tốt nhất cho một đầu vào:

$$\hat{\mathbf{c}} = \arg\max_{\mathbf{c}} \log \Pr(\mathbf{c}|\mathbf{x}), \tag{3.10}$$

trong đó $\mathbf{c} = c_1 \cdots c_l$ là một chuỗi nhãn độ dài l, và $\hat{\mathbf{c}}$ là chuỗi nhãn có xác suất cao nhất. Nhiều phương pháp đã được đề xuất để mô hình hóa $\Pr(\mathbf{c}|\mathbf{x})$. Một cách kinh điển là viết lại $\Pr(\mathbf{c}|\mathbf{x})$ bằng quy tắc Bayes:

$$\begin{aligned}\hat{\mathbf{c}} &= \arg\max_{\mathbf{c}} \log \frac{\Pr(\mathbf{x}|\mathbf{c})\Pr(\mathbf{c})}{\Pr(\mathbf{x})} \\ &= \arg\max_{\mathbf{c}} \log \Pr(\mathbf{x}|\mathbf{c}) + \log \Pr(\mathbf{c}). \end{aligned} \tag{3.11}$$

Vì $\Pr(\mathbf{x})$ là hằng số với mọi chuỗi ứng viên $\mathbf{c}$, ta có thể bỏ qua nó trong quá trình tìm kiếm. $\Pr(\mathbf{x}|\mathbf{c})$ là xác suất sinh ra đầu vào $\mathbf{x}$ (tức các *quan sát*) khi biết chuỗi nhãn $\mathbf{c}$ (tức các *biến ẩn*), còn $\Pr(\mathbf{c})$ là một mô hình ngôn ngữ định nghĩa trên chuỗi nhãn. Nhìn chung, cần đơn giản hóa để mô hình có thể tính toán được. Chẳng hạn, ta có thể đặt **giả thiết Markov** (*Markov assumption*) rằng việc chọn c_i chỉ phụ thuộc vào việc chọn c_{i-1}. Điều này dẫn tới **mô hình Markov ẩn** (*hidden Markov model, HMM*), được dùng rộng rãi trong mô hình hóa sinh cho các bài toán NLP.

Chú giải của người dịch: Quy tắc Bayes và giả thiết Markov

Quy tắc Bayes: $\Pr(\mathbf{c}|\mathbf{x}) = \dfrac{\Pr(\mathbf{x}|\mathbf{c})\Pr(\mathbf{c})}{\Pr(\mathbf{x})}$, cho phép "lật ngược" chiều điều kiện. Thay vì trực tiếp hỏi "với câu này thì nhãn nào hợp lý?", ta hỏi hai câu dễ ước lượng hơn: "chuỗi nhãn này tự nó có hợp lý không?" ($\Pr(\mathbf{c})$) và "nếu nhãn là thế thì sinh ra đúng câu này dễ đến đâu?" ($\Pr(\mathbf{x}|\mathbf{c})$). *Giả thiết Markov* giống như nói: để đoán nhãn của chữ hiện tại, chỉ cần nhìn nhãn của chữ ngay trước, không cần nhớ toàn bộ quá khứ. Ví dụ, sau nhãn E (cuối từ) thì nhãn tiếp theo phải là B, không thể là I hay E.

Một phương pháp khác là mô hình hóa phân biệt (*discriminative modeling*). Một ý tưởng phổ biến là coi gán nhãn chuỗi như một loạt các bài toán phân loại độc lập. Chẳng hạn, ta có thể xây dựng một bộ phân loại (*classifier*) cục bộ, dự đoán c_i dựa trên một tập đặc trưng (*feature*) xung quanh vị trí i. Trong các mô hình phức tạp hơn, như **trường ngẫu nhiên có điều kiện** (*conditional random fields, CRFs*), ngữ cảnh của toàn bộ chuỗi có thể được dùng để dự đoán. Bàn chi tiết về các mô hình gán nhãn chuỗi này nằm ngoài phạm vi của mục này. Bạn đọc có thể tham khảo [Kupiec, 1992; McCallum và cộng sự, 2000; Lafferty và cộng sự, 2001] để có thảo luận đầy đủ về cách các mô hình này được xây dựng và áp dụng. Ngoài ra, về so sánh giữa mô hình hóa sinh và mô hình hóa

phân biệt, mời bạn đọc xem Chương 1.

3.1.4 Học từ con

Từ được xem rộng rãi là đơn vị cơ bản của ngôn ngữ. Tuy nhiên, chúng không phải là những thành phần ngôn ngữ nhỏ nhất. Thay vào đó, chúng có thể được phân tách thành các đơn vị có nghĩa nhỏ hơn, chẳng hạn các hình vị (*morpheme*). Cấu trúc phân cấp này — trong đó hình vị tạo thành từ, còn từ tạo thành cụm từ và câu — làm nổi bật vai trò trung tâm của từ trong cú pháp. Dù vậy, trong NLP, việc coi câu một cách cứng nhắc là chuỗi các từ thường không phải lựa chọn tối ưu. Vấn đề chính là *dữ liệu thưa* gây ra bởi các từ hiếm, cản trở việc huấn luyện mô hình hiệu quả. Chẳng hạn, từ *uncopyrightable* ("không thể đăng ký bản quyền") hiếm khi xuất hiện trong kho ngữ liệu. Một hệ thống NLP có thể coi nó là một token ngoài bộ từ vựng (*out-of-vocabulary*), dù nghĩa của nó có thể suy ra được bằng cách tách nó thành *un* (không), *copy* (sao chép), *right* (quyền) và *able* (có thể). Một vấn đề khác là các tiêu chuẩn tách token dựa trên ngôn ngữ học hạn chế khả năng của mô hình trong việc tự động học các đơn vị phân đoạn tối ưu theo nghĩa học máy. Do đó, thường hiệu quả hơn nếu ta xác định các đơn vị "từ con" — tuy không hẳn mang tính ngôn ngữ học, nhưng phù hợp hơn với các hệ thống học máy.

1. Mã hóa cặp byte

Mã hóa cặp byte (*Byte Pair Encoding, BPE*) là một trong những phương pháp thành công nhất để học các đơn vị **từ con** từ một tập các chuỗi từ [Sennrich và cộng sự, 2016b]. Dù BPE bắt nguồn từ nén dữ liệu [Gage, 1994], trong NLP nó thường được dùng như một giải pháp cho bài toán *bộ từ vựng mở* (*open vocabulary*). Ý tưởng cơ bản của BPE là lặp đi lặp lại việc thay cặp byte xuất hiện nhiều nhất trong dữ liệu bằng một byte mới. Kết quả là các byte phổ biến thường tham gia vào việc gộp các chuỗi byte con, còn các byte hiếm thường đứng riêng lẻ và được coi là các đơn vị riêng. Đầu ra của BPE là một bộ từ vựng byte có thể dùng để mã hóa dữ liệu mới.

Trong NLP, một byte có thể tương ứng gần đúng với một ký tự. Mỗi mục trong bộ từ vựng là một chuỗi ký tự, được gọi là một *ký hiệu* (*symbol*) hay từ con. BPE bắt đầu bằng việc tách văn bản cho trước thành chuỗi ký

tự, chẳng hạn ta có thể chèn một dấu cách sau mỗi chữ cái tiếng Anh hoặc dấu câu. Tuy nhiên, cách này thường tạo ra một chuỗi quá dài. Dù bản thân BPE không giới hạn độ dài đầu vào, cách phổ biến hơn là không cho phép các ký hiệu vượt qua ranh giới từ, để tăng hiệu quả. Vì vậy, ta có thể biểu diễn văn bản thành một danh sách các từ được phân cách bằng dấu cách, mỗi từ đi kèm tần suất của nó. Ví dụ, xét danh sách từ:

f l o w # : 2
b l o w # : 2
f l a t # : 1
f l a g # : 4

trong đó # là ký hiệu đặc biệt chỉ điểm kết thúc một từ[3]. Từ danh sách từ này, ta có thể thu được bộ từ vựng ban đầu:

f : 7	b : 2
l : 9	a : 5
o : 4	t : 1
w : 4	g : 4
# : 9	

Sau đó, ta đếm số lần xuất hiện của mỗi *bigram ký hiệu* (cặp hai ký hiệu đứng liền nhau):

f l : 7	a g : 4
l a : 5	g # : 4
l o : 4	b l : 2
o w : 4	a t : 1
w # : 4	t # : 1

Ta gộp bigram ký hiệu xuất hiện nhiều nhất là “f l” thành một ký hiệu mới “fl” và thay mọi lần xuất hiện của “f l” trong danh sách từ bằng “fl”:

[3]Thay vì coi # là một ký hiệu riêng, có thể gắn nó vào ký tự cuối cùng của mỗi từ

f l o w# : 2
b l o w# : 2
f l a t# : 1
f l a g# : 4

trong đó “w#”, “t#” và “g#” biểu diễn các ký tự xuất hiện ở cuối một từ.

fl o w # : 2
b l o w # : 2
fl a t # : 1
fl a g # : 4

Tương ứng, ký hiệu "fl" được thêm vào bộ từ vựng:

f : 7	b : 2
l : 9	a : 5
o : 4	t : 1
w : 4	g : 4
# : 9	fl : 7

Chú giải của người dịch: Các con số đếm được từ đâu?

Mỗi từ trong danh sách được "nhân" với tần suất của nó. Ví dụ ký hiệu "l" xuất hiện một lần trong mỗi từ, nên tổng là $2 + 2 + 1 + 4 = 9$; "f" có trong *flow*, *flat*, *flag* nên đếm được $2 + 1 + 4 = 7$. Cặp "f l" đứng liền nhau trong đúng ba từ đó nên cũng có tần suất 7 — cao nhất bảng, vì vậy được gộp đầu tiên. Có thể hình dung BPE như việc "đặt tên tắt" cho những cụm chữ hay đi cùng nhau nhất.

Quá trình này sau đó được lặp lại. Lần này, ta gộp bigram ký hiệu "fl a" và tạo ký hiệu mới "fla". Như vậy, ta có danh sách từ mới:

fl o w # : 2
b l o w # : 2
fla t # : 1
fla g # : 4

và bộ từ vựng mới:

f : 7	b : 2	fla : 5
l : 9	a : 5	
o : 4	t : 1	
w : 4	g : 4	
# : 9	fl : 7	

Quá trình gộp này có thể được lặp lại với một số lần định trước. Càng gộp nhiều lần, bộ từ vựng càng lớn. Các mục của bộ từ vựng cuối cùng

được sắp xếp lại theo tần suất ký hiệu. Ví dụ, nếu ta đặt số phép gộp là 6, ta sẽ có bộ từ vựng như sau:

l : 9	fla : 5	ow# : 4
# : 9	o : 4	flag : 4
f : 7	w : 4	flag# : 4
fl : 7	g : 4	b : 2
a : 5	ow : 4	t : 1

Bộ từ vựng này ứng với danh sách từ:

fl ow# : 2
b l ow# : 2
fla t # : 1
flag# : 4

Khi đã có một bộ từ vựng như trên, ta có thể dùng nó để tách token cho các từ mới. Việc tách thành từ con tuân theo cùng thủ tục gộp các bigram ký hiệu như khi xây dựng bộ từ vựng. Cho một bộ từ vựng BPE, trước hết ta phân đoạn văn bản đầu vào thành các ký hiệu ký tự. Sau đó, ta xét từng bigram ký hiệu trong chuỗi và gộp bigram có tần suất cao nhất trong bộ từ vựng. Ta lặp lại thao tác này cho đến khi không còn phép gộp nào nữa. Ví dụ, xét văn bản sau:

tow a flag

("kéo một lá cờ"). Trước hết nó được chuyển thành chuỗi ký tự:

t o w # a # f l a g #

Dùng bộ từ vựng BPE đã thu được, ta có thể thực hiện các phép gộp BPE trên chuỗi này như sau

t o w # a # f l a g #
$\xrightarrow{\text{f l} \Rightarrow \text{fl}}$ t o w # a # fl a g #
$\xrightarrow{\text{fl a} \Rightarrow \text{fla}}$ t o w # a # fla g #
$\xrightarrow{\text{o w} \Rightarrow \text{ow}}$ t ow # a # fla g #
$\xrightarrow{\text{ow \#} \Rightarrow \text{ow\#}}$ t ow# a # fla g #

$$\cdots \quad \cdots$$
$$\longrightarrow \text{t ow\# a \# flag\#}$$

Chuỗi từ con này có thể làm đầu vào và/hoặc đầu ra cho một tác vụ NLP phía sau, chẳng hạn dịch máy (*machine translation*). Đôi khi ta muốn ánh xạ các từ con ngược lại thành từ. Việc này đơn giản: ta giữ dấu cách sau mỗi lần xuất hiện ký hiệu #, rồi xóa mọi dấu cách khác và mọi ký hiệu #. Cũng cần lưu ý rằng phương pháp BPE mô tả ở đây đòi hỏi đầu vào đã được tách từ, tức là ta cần một bộ *tách token sơ bộ* (*pre-tokenizer*) để chia sơ bộ chuỗi đầu vào thành một số đơn vị. Việc này có thể làm bằng các phương pháp trình bày ở Mục 3.1.1–3.1.3.

Chú giải của người dịch: Vì sao BPE giải quyết được từ hiếm?

Từ *tow* chưa hề có trong danh sách từ dùng để học, nhưng BPE vẫn biểu diễn được nó thành *t* + *ow#*. Trong trường hợp xấu nhất, một từ lạ vẫn luôn tách được thành các ký tự đơn lẻ, vốn chắc chắn có trong bộ từ vựng. Nhờ vậy, mô hình gần như không bao giờ gặp token "không biết" (<unk>). Còn các từ phổ biến (như *flag*) thì được giữ nguyên thành một token, giúp chuỗi ngắn gọn. Các mô hình ngôn ngữ lớn hiện nay (họ GPT, LLaMA…) đều dùng một biến thể của BPE hoặc các phương pháp từ con dưới đây.

2. WordPiece

WordPiece rất giống BPE: trước hết nó chia văn bản đầu vào thành các ký hiệu nhỏ nhất, sau đó dần dần gộp các ký hiệu liên tiếp để tạo thành các ký hiệu lớn hơn [Schuster and Nakajima, 2012]. Khác biệt giữa hai phương pháp chỉ nằm ở cách chọn bigram ký hiệu để gộp. Ở mỗi bước, BPE gộp bigram ký hiệu có tần suất cao nhất. Gọi (x_i, x_{i+1}) là một bigram trong chuỗi $\mathbf{x}$. Quy tắc gộp của BPE có thể được mô tả là

$$(x_{\hat{i}}, x_{\hat{i}+1}) \;=\; \underset{i \in [1, |\mathbf{x}|-1]}{\arg\max}\; \text{count}(x_i, x_{i+1}), \tag{3.12}$$

trong đó hàm $\text{count}(x_i, x_{i+1})$ trả về tần suất của (x_i, x_{i+1}) trong kho ngữ liệu, và $(x_{\hat{i}}, x_{\hat{i}+1})$ là bigram có tần suất cao nhất.

Ngược lại, phương pháp WordPiece dùng tiêu chí hợp lý cực đại (*maximum likelihood*) để chọn bigram. Cụ thể, nó gộp bigram làm độ hợp lý (*likelihood*) của dữ liệu huấn luyện tăng nhiều nhất. Điều này có

thể được hình thức hóa là:

$$(x_{\hat{i}}, x_{\hat{i}+1}) = \underset{i \in [1,|\mathbf{x}|-1]}{\arg\max} \log \frac{\Pr(x_i, x_{i+1})}{\Pr(x_i)\Pr(x_{i+1})}$$
$$= \underset{i \in [1,|\mathbf{x}|-1]}{\arg\max} \big[\log \Pr(x_i, x_{i+1}) - \log\big(\Pr(x_i)\Pr(x_{i+1})\big)\big]. \tag{3.13}$$

Số hạng $\log \Pr(x_i, x_{i+1}) - \log\big(\Pr(x_i)\Pr(x_{i+1})\big)$ biểu thị lượng tăng của log-độ hợp lý của kho ngữ liệu khi thay các ký hiệu liên tiếp (x_i, x_{i+1}) bằng ký hiệu đã gộp $x_i x_{i+1}$[4]. Do đó, áp dụng quy tắc gộp này sinh ra một dãy các thao tác làm tăng dần độ hợp lý tổng thể của dữ liệu. Kết quả của quá trình này là một *bảng mã* (tức một bộ từ vựng) mà nhờ nó ta có thể xác định chuỗi mã có khả năng nhất cho văn bản cho trước.

Chú giải của người dịch: BPE và WordPiece khác nhau ra sao?

BPE chỉ hỏi: "cặp nào đi cùng nhau *nhiều lần nhất*?". WordPiece hỏi: "cặp nào đi cùng nhau nhiều hơn *mức ngẫu nhiên* nhất?". Nếu a và b xuất hiện độc lập, xác suất gặp chúng đứng cạnh nhau xấp xỉ $\Pr(a)\Pr(b)$ (như tung hai đồng xu độc lập). Tỉ số $\frac{\Pr(a,b)}{\Pr(a)\Pr(b)}$ lớn hơn 1 nghĩa là a, b "hút nhau" thật sự. Ví dụ, hai chữ rất phổ biến như "e" và "t" có thể đứng cạnh nhau nhiều lần chỉ vì bản thân chúng phổ biến; WordPiece sẽ ưu tiên những cặp hiếm hơn nhưng gần như luôn đi cùng nhau, như "q" và "u" trong tiếng Anh.

3. SentencePiece

Cả BPE và WordPiece đều đòi hỏi văn bản đầu vào được tách sơ bộ thành từ. Sự phụ thuộc này làm quy trình tách token phức tạp thêm. Như một lựa chọn thay thế, SentencePiece cung cấp một khuôn khổ tổng quát hơn, xử lý trực tiếp văn bản thô bằng cách coi mọi ký tự, kể cả dấu cách, là các đơn vị nguyên tử cơ bản [Kudo and Richardson, 2018]. Dù SentencePiece có thể kết hợp với nhiều thuật toán từ con khác nhau, nó thường gắn với các mô hình ngôn ngữ unigram (hay gọi tắt là *phương pháp unigram*), vốn áp dụng một chiến lược xây dựng bộ từ vựng khác hẳn: thay vì xây bộ từ

[4] Trong thống kê, $\frac{\Pr(a,b)}{\Pr(a)\Pr(b)}$ được gọi là **thông tin tương hỗ theo điểm** (*pointwise mutual information*) của hai biến a và b. Xem thêm chi tiết ở Mục 3.3.1. Khái niệm này còn được gọi là **độ lợi thông tin** (*information gain*) và có thể được diễn giải bằng độ phân kỳ Kullback-Leibler hoặc các độ đo khác trong lý thuyết thông tin (xem Chương 1).

vựng từ dưới lên, nó bắt đầu với một bộ từ vựng lớn rồi dần dần cắt tỉa để cực đại hóa độ hợp lý tổng thể của dữ liệu [Kudo, 2018][5].

Phương pháp unigram phát biểu việc phân đoạn từ con như một bài toán mô hình hóa ngôn ngữ unigram, giống dạng tổng quát của Phương trình (3.3–3.4). Gọi $\mathbf{x}$ là một chuỗi ký tự và $\mathbf{y}$ là một chuỗi các ký hiệu hay từ con mà khi nối lại tạo thành $\mathbf{x}$. Xác suất của $\mathbf{y}$ là

$$\Pr(\mathbf{y}) = \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i). \tag{3.14}$$

Khi đó, ta có thể viết độ hợp lý của $\mathbf{x}$ theo xác suất đồng thời của $\mathbf{x}$ và $\mathbf{y}$:

$$\Pr(\mathbf{x}) = \sum_{\mathbf{y} \in Y(\mathbf{x})} \Pr(\mathbf{x}, \mathbf{y}), \tag{3.15}$$

trong đó tổng được lấy trên mọi kết quả tách token có thể có $Y(\mathbf{x})$. Vì $\mathbf{x}$ được suy ra một cách tất định từ $\mathbf{y}$, xác suất đồng thời được đơn giản thành $\Pr(\mathbf{x}, \mathbf{y}) = \Pr(\mathbf{y})$. Do đó, ta có thể viết lại Phương trình (3.15) thành:

$$\begin{aligned} \Pr(\mathbf{x}) &= \sum_{\mathbf{y} \in Y(\mathbf{x})} \Pr(\mathbf{y}) \\ &= \sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i). \end{aligned} \tag{3.16}$$

Chú giải của người dịch: Đọc Phương trình (3.16) bằng ví dụ

Ký hiệu $\prod$ là *tích* (giống $\sum$ là tổng): $\prod_{i=1}^{3} a_i = a_1 a_2 a_3$. Giả sử chuỗi $\mathbf{x}$ = "ab" và bộ từ vựng có {a, b, ab} với $\Pr(\text{a}) = 0.3$, $\Pr(\text{b}) = 0.2$, $\Pr(\text{ab}) = 0.1$. Có hai cách tách: "a/b" với xác suất $0.3 \times 0.2 = 0.06$, và "ab" với xác suất 0.1. Theo (3.16), $\Pr(\mathbf{x}) = 0.06 + 0.1 = 0.16$: ta *cộng* xác suất của mọi cách tách, vì đó là các khả năng loại trừ nhau dẫn đến cùng một chuỗi.

Từ phương trình này, log-độ hợp lý của một tập chuỗi X là

$$\Pr(X) = \log \prod_{\mathbf{x} \in X} \sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i)$$

[5] Thuật ngữ *kích thước bộ từ vựng* (*vocabulary size*) thường chỉ số mục trong bộ từ vựng. Tuy nhiên, trong một số ngữ cảnh tính toán, nó có thể chỉ tổng số byte cần để lưu bộ từ vựng.

$$= \sum_{\mathbf{x} \in X} \log \left(\sum_{\mathbf{y} \in Y(\mathbf{x})} \prod_{i=1}^{|\mathbf{y}|} \Pr(y_i) \right). \tag{3.17}$$

Nếu ta coi $-\Pr(X)$ là một hàm mất mát (*loss function*), thì nhiệm vụ ở đây có thể phát biểu là tìm ước lượng tốt nhất cho mỗi xác suất unigram $\Pr(y)$ sao cho $\Pr(X)$ lớn nhất có thể. Thoạt nhìn, bài toán tối ưu này có vẻ phức tạp. May mắn là có một số công cụ mạnh để giải nó. Một phương pháp phổ biến là dùng **thuật toán Kỳ vọng–Cực đại hóa** (*Expectation-Maximization*, **EM**) [Dempster và cộng sự, 1977], thường được dùng khi ta muốn tìm một mô hình thống kê làm cực đại độ hợp lý của dữ liệu. Lưu ý rằng lời giải dựa trên EM cho Phương trình (3.17) tương tự lời giải cho các bài toán NLP khác, chẳng hạn dịch máy thống kê, và đã được bàn kỹ trong những ngữ cảnh đó. Vì vậy, chúng tôi giới thiệu bạn đọc tài liệu [Brown và cộng sự, 1993] để biết chi tiết về các phương pháp này. Trong chương này, ta dùng EM như một công cụ có sẵn để ước lượng $\Pr(y)$ dựa

trên Phương trình (3.17).[6]

Chú giải của người dịch: Trực giác về thuật toán EM

Nếu biết sẵn mỗi câu được tách thế nào, ta chỉ việc *đếm* số lần mỗi từ con xuất hiện rồi chia cho tổng để có xác suất (như tính tần suất trong thống kê lớp 10). Khó khăn là ta không biết cách tách đúng. EM giải quyết như "con gà và quả trứng": đoán tạm các xác suất → dùng chúng để tính mỗi cách tách có khả năng bao nhiêu (bước E) → đếm "có trọng số": một cách tách có xác suất 0.7 thì mỗi từ con trong nó được cộng 0.7 lần, nên số đếm có thể lẻ, gọi là *số đếm phân số* → chuẩn hóa thành xác suất mới (bước M) → lặp lại. Mỗi vòng lặp không làm độ hợp lý giảm đi.

Về bản chất, SentencePiece là một phương pháp "cắt tỉa", loại bỏ các mục có xác suất thấp khỏi bộ từ vựng. Nó bắt đầu với một bộ từ vựng ban đầu lớn V. Chẳng hạn, ta có thể tạo bộ từ vựng ban đầu bằng cách liệt kê mọi chuỗi thỏa một ràng buộc về độ dài. Thông thường, các chuỗi vượt

[6]Trong EM, ta có thể coi X là một *quan sát*, và $\Pr(X|\theta)$ là một mô hình thống kê mô tả khả năng quan sát đó xảy ra. Ở đây θ là các tham số mô hình mà ta cần xác định. EM dựa trên mục tiêu ước lượng hợp lý cực đại, tức là

$$\hat{\theta} \;=\; \arg\max_{\theta} \Pr(X|\theta). \tag{3.18}$$

Với mô hình ở đây, ta có thể coi $\{\Pr(y)\}$ là các tham số mô hình. Chúng tôi bỏ qua chi tiết suy diễn ước lượng EM của $\Pr(y)$ mà trình bày trực tiếp kết quả. Thuật toán EM gồm hai bước.

- **Bước Kỳ vọng** (hay *bước E*): Cho ước lượng hiện tại của $\Pr(y)$ (gọi là $\Pr_t(y)$), ta tính xác suất hậu nghiệm $\Pr_t(\mathbf{y})$ cho mỗi $\mathbf{y}$ theo Phương trình (3.14). Sau đó, ta tính **số đếm phân số** (*fractional count*) của mỗi từ con y trong bộ từ vựng V như sau

$$\text{fcount}(y) \;=\; \sum_{\mathbf{x}\in X} \sum_{\mathbf{y}\in Y(\mathbf{x})} \left(\Pr_t(\mathbf{y}) \sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i) \right), \tag{3.19}$$

  trong đó $\delta(y, y_i)$ trả về 1 nếu $y = y_i$, và 0 nếu ngược lại. $\sum_{i=1}^{|\mathbf{y}|} \delta(y, y_i)$ đếm số lần y xuất hiện trong chuỗi từ con $\mathbf{y}$.

- **Bước Cực đại hóa** (hay *bước M*): Từ các số đếm phân số thu được ở bước E, ta ước lượng lại các xác suất unigram bằng phương trình:

$$\Pr_{t+1}(y) \;=\; \frac{\text{fcount}(y)}{\sum_{y'\in V} \text{fcount}(y')}. \tag{3.20}$$

Hai bước này được lặp lại qua nhiều vòng cho đến khi các tham số hội tụ về những giá trị nhất định.

qua ranh giới từ bị loại để giảm kích thước bộ từ vựng. Sau đó, ta chạy các bước sau:

- Ước lượng xác suất cho mỗi mục y của V bằng cách tối ưu Phương trình (3.17).
- Tính mất mát cho mỗi mục y của V theo chiến lược *bỏ-một* (*remove-one*), tức là mất mát bằng lượng giảm của độ hợp lý (xem Phương trình (3.17)) khi y bị loại khỏi bộ từ vựng.
- Loại bỏ một tỉ lệ nhất định các mục của V có mất mát lớn. Chẳng hạn, ta giữ lại 80% số mục và bỏ phần còn lại.

Kết quả của quá trình này là một bộ từ vựng mới cùng với xác suất gán cho mỗi từ con. Ta có thể lặp lại quá trình này một số lần cho đến khi kích thước bộ từ vựng giảm đến mức mong muốn.

chuỗi từ con	xác suất unigram ([từ con]:xác suất)
t/ow_/a/_flag	[t]:0.030 [ow_]:0.002 [a]:0.041 [_flag]:0.001
t/ow/_/a_/f/lag	[t]:0.030 [ow]:0.005 [_]:0.113 [a_]:0.093 [f]:0.041 [lag]:0.002
t/ow/_a_/fla/g	[t]:0.030 [ow]:0.005 [_a_]:0.084 [fla]:0.003 [g]:0.027
tow/_/a_/f/lag	[tow]:0.001 [_]:0.113 [a_]:0.093 [f]:0.041 [lag]:0.002
t/ow_/a_/flag	[t]:0.030 [ow_]:0.002 [a_]:0.093 [flag]:0.001

Hình 3.3: Các kết quả tách token khác nhau cho *tow a flag*. Mỗi từ con được gán một xác suất ước lượng bằng một mô hình ngôn ngữ unigram. Mỗi khoảng trắng được thay bằng "_" để trình bày rõ ràng.

SentencePiece khác BPE và WordPiece ở chỗ nó xét mọi chuỗi từ con có thể có cho một chuỗi cho trước (xem tổng $\sum_{\mathbf{y}\in Y(\mathbf{x})}$ trong Phương trình (3.15)). Từ góc nhìn học máy, điều này có thể xem là một cách chính quy hóa (*regularization*), tức là ta có thể giảm nguy cơ ước lượng quá cao các tham số ứng với chuỗi từ con tốt nhất duy nhất, vốn có thể chứa lỗi. Một cách khác là chỉ xét một số chuỗi từ con trong $Y(\mathbf{x})$ để tăng hiệu quả. Chẳng hạn, ta có thể lấy mẫu k chuỗi từ con theo $\Pr(\mathbf{y})$ để tạo thành tập ứng viên $Y(\mathbf{x})$.

Lưu ý rằng phương pháp SentencePiece không phụ thuộc vào chuỗi đầu vào đã được tách từ. Dù các phương pháp BPE và WordPiece cũng có thể xử lý văn bản thô nếu được điều chỉnh, phương pháp SentencePiece coi một cách tường minh dấu cách và các dấu phân cách khác là một phần của từ con. Xem Hình 3.3 để thấy một vài kết quả tách token cho *tow a*

flag.

Chú giải của người dịch: Đọc Hình 3.3

Mỗi dòng là một cách tách; xác suất của cả cách tách bằng tích các xác suất trong dòng (Phương trình (3.14)). Ví dụ dòng cuối: $0.030 \times 0.002 \times 0.093 \times 0.001 \approx 5.6 \times 10^{-9}$. Chú ý dấu cách ("_") được coi như một ký tự bình thường, nên "a_" (chữ *a* kèm dấu cách theo sau) và "a" là hai từ con khác nhau. Nhờ vậy, từ chuỗi từ con có thể khôi phục chính xác văn bản gốc, kể cả vị trí các dấu cách — rất tiện cho các ngôn ngữ như tiếng Việt, nơi dấu cách nằm giữa các âm tiết.

Với một bộ từ vựng đã học và các xác suất unigram tương ứng, ta có thể dùng chúng để xử lý một văn bản mới. Đây thực chất là một bài toán tìm kiếm: ta tìm chuỗi từ con có khả năng nhất theo xác suất unigram. Vì mô hình hóa ngôn ngữ là một chủ đề đã được nghiên cứu kỹ trong NLP, nhiều thuật toán tìm kiếm có thể áp dụng trực tiếp cho trường hợp ở đây. Chẳng hạn, các phương pháp trình bày ở Mục 3.1.2 có thể áp dụng ngay.

3.2 Biểu diễn vectơ cho từ

Từ tự thân mang nghĩa[7]. Theo nghĩa rộng nhất, nghĩa của một từ đến từ cách nó được diễn giải. Đó là thứ nằm phía sau hình thức bề mặt của từ, nhưng người nói ngôn ngữ đó có thể hiểu được. Ví dụ, xét mấy dòng sau trong một bài thơ [Knight, 2018]:

There was a little sparrow
Who sat on a wheelbarrow,
And tweeted to all her friends around.
A cat with open jaws
And very pointed claws,
Spied her as he raced along the ground.

(Tạm dịch nghĩa: "Có một chú chim sẻ nhỏ / đậu trên chiếc xe cút kít, /

[7]Dù đến giờ ta đã bàn về một số thành phần ngôn ngữ dùng trong NLP như từ con, ở đây ta vẫn dùng từ làm đơn vị cơ bản trong thảo luận. Các phương pháp trình bày trong phần còn lại của chương này nhìn chung có thể áp dụng cho các đơn vị ngôn ngữ khác dùng trong hệ thống NLP, bao gồm ký tự, từ con, v.v.

ríu rít gọi bạn bè quanh đó. / Một con mèo há rộng hàm / với những móng vuốt thật nhọn, / phóng dọc mặt đất và rình thấy nó.")

Những từ này không chỉ là các chuỗi ký tự tùy ý; chúng gợi ra những khái niệm cụ thể. Chẳng hạn, "little" nghĩa là nhỏ về kích thước, "sparrow" nghĩa là một loài chim (chim sẻ), còn "friends" nghĩa là những người bạn quý mến và tin tưởng. Tuy nhiên, từ góc nhìn NLP, nghĩa của từ (hay nghĩa từ (*word sense*)) không thể chỉ dừng lại là một khái niệm trừu tượng trong nhận thức; nó phải được hình thức hóa thành một dạng máy có thể đọc và tính toán được.

3.2.1 Biểu diễn one-hot

Một cách tự nhiên để biểu diễn nghĩa của từ là dùng ngôn ngữ để mô tả chúng. Chẳng hạn, ta có thể tra các từ vừa nhắc đến ở trên trong từ điển để tìm mã số (id) và định nghĩa của chúng[8]:

[8]Tất cả các từ này và nghĩa của chúng được lấy từ `https://dictionary.cambridge.org/`.

cat	511	*A small animal with fur, four legs, a tail, and claws, usually kept as a pet or for catching mice* (con mèo: động vật nhỏ có lông, bốn chân, đuôi và móng vuốt, thường nuôi làm thú cưng hoặc để bắt chuột)
her	5220	*Used, usually as the object of a verb or preposition, to refer to a woman, girl, or female animal that has just been mentioned or is just about to be mentioned* (cô ấy/nó: thường làm tân ngữ của động từ hoặc giới từ, chỉ một người phụ nữ, cô gái hay con vật cái vừa hoặc sắp được nhắc đến)
jaws	6186	*The mouth, including the teeth* (hàm: miệng, kể cả răng)
ground	6402	*The surface of the earth* (mặt đất)
sparrow	8331	*A common, small, gray-brown bird* (chim sẻ: loài chim nhỏ, phổ biến, màu nâu xám)
wheelbarrow	9954	*A large, open container for moving things in with a wheel at the front and two handles at the back, used especially in the garden* (xe cút kít: thùng lớn, hở, để chở đồ, có một bánh xe phía trước và hai tay cầm phía sau, dùng nhiều trong vườn)

Để biểu diễn một từ, ý tưởng đơn giản nhất là thay nó bằng id trong từ điển. Theo cách này, biểu diễn của mỗi từ là một con số duy nhất. Một dạng tương đương là **biểu diễn one-hot** (*one-hot representation*). Đó là một vectơ có số chiều bằng kích thước bộ từ vựng. Trong vectơ này, chỉ phần tử ứng với chỉ số của từ được đặt bằng 1, còn mọi phần tử khác đều bằng 0. Ví dụ, từ *sparrow* có thể được biểu diễn bằng vectơ one-hot $\mathbf{1}_{\text{sparrow}}$ dựa trên id của nó (8331):

$$\begin{bmatrix} 0 & 0 & \dots & 0 & \underset{\uparrow}{1} & 0 & \dots & 0 & 0 \end{bmatrix}$$
$$\text{id} = 8331$$

Chú giải của người dịch: Vectơ one-hot

Giả sử bộ từ vựng chỉ có 4 từ: {*cat*, *dog*, *bird*, *tree*}. Khi đó $cat = [1\ 0\ 0\ 0]$, $dog = [0\ 1\ 0\ 0]$, $bird = [0\ 0\ 1\ 0]$, $tree = [0\ 0\ 0\ 1]$. Tên gọi "one-hot" ("một

điểm nóng") chỉ việc đúng một vị trí "bật sáng". Với bộ từ vựng thật cỡ vài chục nghìn từ, mỗi vectơ dài vài chục nghìn chiều nhưng gần như toàn số 0.

3.2.2 Biểu diễn phân tán

Tuy nhiên, có vẻ như biểu diễn one-hot chỉ cung cấp "danh tính" của từ chứ không cung cấp "mô tả" xem từ đó là gì. Một vấn đề hiển nhiên là mọi từ đều *trực giao* với mọi từ khác. Do tính trực giao này, không thể định lượng bằng toán học mối quan hệ ngữ nghĩa giữa các từ, kể cả những từ có liên hệ rất chặt trong ngôn ngữ tự nhiên. Ở đây, điều ta mong muốn là một mô hình trong đó các từ được mô tả bằng các thuộc tính định lượng được và sự gần gũi về ngữ nghĩa được nắm bắt một cách tự nhiên. Một cách làm điều đó là làm giàu biểu diễn bằng phần mô tả của từ. Xét lại từ *sparrow* làm ví dụ. Trong từ điển, nghĩa của nó là *a common, small, gray-brown bird*. Dùng các phương pháp tách token và chuẩn hóa nêu ở Mục 3.1.1, đoạn văn bản này có thể được chuyển thành một chuỗi từ

$$\left[\ a \quad common \quad , \quad small \quad , \quad gray \quad - \quad brown \quad bird\ \right]$$

Sau đó, ta vectơ hóa chuỗi này bằng mô hình túi từ (*bag-of-words*) (xem Chương 1), thu được một vectơ số mới

$$\begin{array}{ccccccccccccccccccccc} [& 0 & 0 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 1 & \dots & 0 & 0 &] \\ & & & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & \uparrow & & & & \\ & & & & , & & - & & a & & bird & & brown & & common & & gray & & small & & & & \end{array}$$

trong đó giá trị của một phần tử bằng 1 nếu từ tương ứng có mặt, và bằng 0 nếu ngược lại. Cách này cho phép các từ chia sẻ nội dung với nhau. Ta nói hai từ là tương tự nhau nếu vectơ từ của chúng có phần trùng lặp. Xét một từ mới *cuckoo* (chim cu cu). Ta có thể tìm nghĩa của nó trong từ điển, ví dụ *a grey bird with a two-note call that sounds similar to its name* (một loài chim xám có tiếng kêu hai nốt nghe giống tên của nó). Dễ thấy *sparrow* và *cuckoo* là hai từ có điểm tương đồng, vì cả hai đều đánh dấu chiều "bird" bằng 1 và độ tương tự vectơ giữa hai vectơ từ này lớn hơn 0[9].

[9]Độ tương tự của hai vectơ có thể được đo bằng cosin của góc giữa chúng.

Chú giải của người dịch: Trực giao và độ tương tự cosin

Ở lớp 10, hai vectơ vuông góc khi tích vô hướng của chúng bằng 0, và $\cos(\vec{u}, \vec{v}) = \dfrac{\vec{u} \cdot \vec{v}}{|\vec{u}|\,|\vec{v}|}$. Với vectơ nhiều chiều, công thức y hệt: tích vô hướng là tổng các tích từng cặp tọa độ. Hai vectơ one-hot khác nhau luôn có tích vô hướng bằng 0 (không có vị trí nào cùng bằng 1), nên cosin bằng 0: *cat* "xa" *dog* đúng bằng *cat* "xa" *tree*. Còn với vectơ túi từ ở trên, *sparrow* và *cuckoo* cùng có 1 ở chiều "bird" (và chiều "a"), nên tích vô hướng dương và cosin lớn hơn 0.

Coi từ là vectơ số cung cấp một công cụ tổng quát để biểu diễn từ theo nhiều cách khác nhau. Ta thậm chí không cần giải thích vectơ từ dưới góc độ ngữ nghĩa. Chẳng hạn, ta có thể thêm một chiều mới vào vectơ để đánh dấu từ có thuộc một phạm trù cú pháp nào đó hay không. Theo nghĩa rộng, ta có thể định nghĩa một hàm tùy ý trên mỗi phần tử của vectơ và coi đầu ra của hàm là một đặc trưng (*feature*) mô tả từ. Ví dụ, một cải tiến đơn giản cho biểu diễn trên là dùng hàm đếm số lần xuất hiện của từ thay cho hàm nhị phân đánh dấu sự có mặt của từ. Có thể tìm thêm các hàm đặc trưng khác ở Mục 3.3.

Lưu ý rằng không nhất thiết phải ràng buộc các hàm đặc trưng ở những dạng có ý nghĩa ngôn ngữ học, dù các thiết kế hàm đặc trưng xuất phát từ ngôn ngữ học thường được các nhà nghiên cứu NLP quan tâm. Một dạng tổng quát hơn để biểu diễn từ đơn giản là một vectơ số thực nhiều chiều. Nó thường được gọi là **biểu diễn phân tán** (*distributed representation*) của từ, hay **nhúng từ** (*word embedding*). Chẳng hạn, từ *sparrow* có thể được biểu diễn bằng vectơ

$$\begin{bmatrix} 1.9 & -7 & 3 & -1.2 & \dots & 2.01 & -2.05 \end{bmatrix}$$

Từ góc nhìn học máy, vectơ này có thể mô tả một số thuộc tính tiềm ẩn của từ. Những thuộc tính này có thể không giải thích được theo cách hiểu của con người nhưng có thể học được từ dữ liệu. Một trong những thách thức khi học biểu diễn như vậy là khó đo được độ tốt của một vectơ. Nhìn chung, hỏi biểu diễn phân tán của một từ đơn lẻ là tốt hay không là vô nghĩa. Thay vào đó, ta muốn biết biểu diễn của một nhóm từ có "cư xử" hợp lý hay không. Chẳng hạn, người ta thường tin rằng các từ tương tự nhau nên có biểu diễn tương tự nhau. Vì vậy, quan hệ giữa các từ thường được hình dung như một loại "khoảng cách" giữa các biểu diễn từ trong

một không gian vectơ. Điều này dẫn tới nhiều phương pháp trực quan hóa và đánh giá biểu diễn từ. Ở Mục 3.6, chúng tôi sẽ bàn chi tiết hơn về những vấn đề này.

Chú giải của người dịch: Vì sao gọi là "phân tán"?

Trong vectơ one-hot, toàn bộ thông tin về một từ "dồn" vào đúng một vị trí. Trong biểu diễn phân tán, thông tin được *rải* ra trên tất cả các chiều: mỗi chiều có thể phản ánh một phần ý nghĩa (một cách mơ hồ, chẳng hạn "mức độ liên quan đến động vật"), và mỗi từ là một tổ hợp riêng của các giá trị đó. Vectơ thường ngắn (vài trăm chiều) và dày đặc (hầu hết khác 0), trái ngược với vectơ one-hot dài và thưa.

Thông thường, biểu diễn từ không hoạt động độc lập trong các hệ thống NLP mà được dùng như những trạng thái trung gian của một mô hình. Một cách tiếp cận chuẩn trong NLP để học biểu diễn phân tán của từ là coi nó là *sản phẩm phụ* của việc huấn luyện một hệ thống "lớn". Tức là mô hình biểu diễn hoạt động như một thành phần của hệ thống, và được tối ưu cùng các thành phần khác khi hệ thống được huấn luyện theo một cách nào đó. Điều này gợi ra một mô hình học biểu diễn (*representation learning*) đầy hứa hẹn: mô hình biểu diễn được học như một mô hình con trong một hệ thống dễ huấn luyện, rồi có thể dùng như một mô-đun "cắm vào" cho một hệ thống hoàn toàn khác. Chẳng hạn, trong mô hình hóa ngôn ngữ bằng mạng nơ-ron, ta có thể buộc mô hình ánh xạ mỗi vectơ từ one-hot đầu vào thành một biểu diễn phân tán số thực, ít chiều. Các biểu diễn phân tán này được đưa vào một mạng nơ-ron (*neural network*) dự đoán một phân phối xác suất (*probability distribution*) trên bộ từ vựng. Hàm ánh xạ hay *hàm nhúng* được huấn luyện sao cho cực tiểu hóa mất mát của mô hình ngôn ngữ trên một tập dữ liệu nào đó (xem Mục 3.4). Khi áp dụng hàm nhúng đã học, ta bỏ mọi phần khác của mô hình ngôn ngữ và dùng hàm này để sinh biểu diễn phân tán cho mỗi từ trong các tác vụ phía sau. Một chiến lược khác là thiết kế mô hình chuyên biệt cho bài toán học biểu diễn từ. Các hệ thống loại này thường không được thiết kế để giải các bài toán NLP tiêu chuẩn, mà nhấn mạnh vào các vấn đề cụ thể trong học biểu diễn từ, chẳng hạn mô hình hóa tường minh quan hệ giữa các từ (xem Mục 3.5).

Chú giải của người dịch: Hàm nhúng thực chất là "tra bảng"

Nhân vectơ one-hot của từ thứ k với một bảng số (ma trận) có mỗi hàng là một vectơ nhúng, kết quả chính là hàng thứ k của bảng. Vì vậy, hàm nhúng có thể hình dung như một cuốn "từ điển số": tra id của từ, lấy ra vectơ của nó. Các con số trong bảng ban đầu được chọn ngẫu nhiên, rồi được điều chỉnh dần trong quá trình huấn luyện mô hình ngôn ngữ.

3.2.3 Tính kết hợp và tính phụ thuộc ngữ cảnh

Dù thảo luận của ta giới hạn ở việc học biểu diễn từ trong NLP, nghiên cứu nghĩa của từ là một phân ngành truyền thống của ngôn ngữ học. Chẳng hạn, trong ngữ nghĩa học từ vựng (*lexical semantics*), các nhà nghiên cứu tìm hiểu nghĩa của từ được định nghĩa thế nào và chúng kết hợp tạo nên nghĩa ở cấp câu ra sao. Về mặt lịch sử, biểu diễn từ trên máy tính không bám chặt vào ngữ nghĩa học ngôn ngữ, mà tập trung vào việc chuyển các đơn vị ngôn ngữ thành dạng tính toán được. Tuy nhiên, vì ngữ nghĩa là nền tảng của việc hiểu ngôn ngữ, việc đưa các nguyên lý ngữ nghĩa vào thiết kế mô hình biểu diễn vẫn rất quan trọng. Xét các tính chất sau:

- **Tính kết hợp** (*compositionality*). Tính kết hợp là một khái niệm phổ biến trong ngữ nghĩa học, logic học và các lĩnh vực liên quan. Nó thường đi kèm với **nguyên lý kết hợp** (*principle of compositionality*):

  > *Nghĩa của một biểu thức phức hợp được xác định bởi cấu trúc của nó và nghĩa của các thành phần cấu tạo nên nó.*
  >
  > – Szabó [2020]

  Nguyên lý kết hợp là nền tảng và có mặt ở khắp nơi trong ngôn ngữ tự nhiên. Chẳng hạn, nghĩa của cụm *white cat* (mèo trắng) dễ dàng suy ra từ nghĩa riêng của *white* (trắng) và *cat* (mèo). Tương tự, câu ghép có được nghĩa tổng thể từ sự kết hợp của các mệnh đề độc lập và liên từ. Lưu ý rằng nguyên lý kết hợp không chỉ là một quy tắc đơn giản ta dùng để mô tả cách các đơn vị ngôn ngữ lớn được tạo nên từ các đơn vị nhỏ, dù các nhà nghiên cứu đã cố gắng định nghĩa nó một cách hình thức [Montague, 1974]. Vẫn đang có những

tranh luận về việc nguyên lý này nên được diễn giải và mô hình hóa thỏa đáng bởi các lý thuyết ngữ nghĩa như thế nào. Dẫu vậy, nếu ta tập trung vào các ứng dụng NLP và gác lại các tranh luận lý thuyết ngôn ngữ học, tính kết hợp vẫn là một tính chất rất có giá trị cho việc thiết kế và đánh giá hệ thống. Khi một bài toán có tính kết hợp, điều đó hàm ý rằng các tác vụ phức tạp có thể được phân rã một cách có hệ thống thành các thành phần con dễ xử lý hơn. Với việc học biểu diễn từ, ta mong các vectơ thu được thể hiện tính kết hợp để phản ánh bản chất tổ hợp của ngôn ngữ. Ở Mục 3.6, ta sẽ thấy một số ví dụ về điều này. Chẳng hạn, các biểu diễn học được bởi mạng nơ-ron thường thể hiện những tính chất đại số có ý nghĩa dưới các phép toán tuyến tính, dù bản thân các mô hình là phi tuyến. Tuy nhiên, tính kết hợp không phải là quy tắc tuyệt đối. Nó thường bị phá vỡ trong các trường hợp như *cụm từ cố định* (*collocation*) và *thành ngữ* (*idiom*), nơi ngôn ngữ tự nhiên thể hiện tính phi kết hợp. Điều này giải thích vì sao các bài toán NLP lại khó đến vậy.

- **Tính phụ thuộc ngữ cảnh** (*contextuality*). Tính phụ thuộc ngữ cảnh thể hiện một dạng phi kết hợp. Nó nói rằng một từ có thể có nhiều nghĩa, và cách diễn giải cụ thể của nó được quyết định bởi ngữ cảnh (*context*) xung quanh. Ví dụ, xét các câu sau[10]

  They sat round the dinner table, arguing about politics.
  (Họ ngồi quanh bàn ăn tối, tranh luận về chính trị.)
  Come to the table everybody - supper's ready.
  (Mọi người lại bàn nào, bữa tối xong rồi.)
  He came in with four shopping bags and dumped them on the table.
  (Anh ấy bước vào với bốn túi đồ và quăng chúng lên bàn.)
  The table can help you evaluate the potential risks of investing in the Fund.
  (Bảng này có thể giúp bạn đánh giá rủi ro tiềm ẩn khi đầu tư vào Quỹ.)
  Building societies dominate the best-value tables for mortgages.

[10] Tất cả các câu này được lấy từ `https://dictionary.cambridge.org/dictionary/english/table`

(Các hiệp hội nhà ở chiếm lĩnh các bảng xếp hạng khoản vay thế chấp có lợi nhất.)
This table represents export sales.
(Bảng này thể hiện doanh số xuất khẩu.)

Trong các câu ví dụ này, *table* là một từ đa nghĩa (*polysemous word*) với hai nghĩa:

Sense 1: a flat surface used for putting things on. (Nghĩa 1: một mặt phẳng dùng để đặt đồ vật lên — cái bàn.)
Sense 2: an arrangement of items in rows, or columns, or blocks. (Nghĩa 2: sự sắp xếp các mục theo hàng, cột hoặc khối — cái bảng.)

Nói cách khác, *table* là một từ nhập nhằng. Sự nhập nhằng này thường được giải quyết bằng cách xem xét các từ lân cận. Chẳng hạn, khi *table* đứng sau *dinner*, dễ nhận ra nó mang nghĩa 1. Sự nhập nhằng cũng tồn tại khi một từ bắt nguồn từ vài dạng hay *từ vị* (*lexeme*) khác nhau (hiện tượng được gọi là **đồng âm** (*homonymy*)). Ví dụ, *bear* có thể là động từ (chịu đựng) hoặc danh từ (con gấu). Khử nhập nhằng một từ đối với một tập nghĩa từ cho trước đã được nghiên cứu hàng chục năm trong NLP và thường được gọi là **khử nhập nhằng nghĩa từ** (*word sense disambiguation, WSD*) [Kelly and Stone, 1975]. Tuy nhiên, bài toán biểu diễn từ bàn ở đây khó hơn, vì ta thường không có sẵn một tập nghĩa từ định trước. Thay vào đó, ta muốn có một *mô hình biểu diễn theo ngữ cảnh* có thể sinh ra biểu diễn của một từ phụ thuộc vào ngữ cảnh của nó. Do đó, cần nhận thức rằng nghĩa của từ không cố định. Điều này khiến bài toán có phần khác với những gì ta bàn ở đầu mục, vì ta không còn một bảng tra cứu biểu diễn từ, mà là một mô hình sinh ra các biểu diễn khác nhau cho cùng một từ trong các ngữ cảnh khác nhau.

Chú giải của người dịch: Ví dụ trong tiếng Việt

Tính kết hợp: nghĩa của "mèo trắng" = nghĩa "mèo" + nghĩa "trắng". *Phi kết hợp*: thành ngữ "ăn cháo đá bát" không nói về cháo hay bát, mà chỉ người vô ơn; ghép nghĩa từng chữ sẽ hiểu sai. *Phụ thuộc ngữ cảnh*: từ "đường" trong "ăn *đường*" (chất ngọt) và "đi *đường*" (lối đi) là hai nghĩa khác hẳn; "bay" trong "chim bay" và "cái bay của thợ xây". Một vectơ cố định duy nhất cho

"đường" khó biểu diễn đúng cả hai nghĩa — đây là động lực cho các mô hình biểu diễn theo ngữ cảnh (như BERT, GPT) ở các chương sau.

Trong các mục còn lại của chương này, chúng tôi tập trung vào việc học biểu diễn vectơ của từ từ sự phân bố của chúng trong việc sử dụng ngôn ngữ. Phần thảo luận về các mô hình theo ngữ cảnh để học biểu diễn từ dày đặc được để lại cho Chương 4–6.

3.3 Các mô hình dựa trên đếm

Chúng ta đã đặt việc quy nạp nghĩa của từ thành bài toán học các vectơ từ. Trong mục này, ta đi tiếp với giả định rằng nghĩa của một từ được quyết định bởi môi trường mà từ đó được sử dụng. Điều này thường được phát biểu thành **giả thuyết phân bố** (*distributional hypothesis*): các từ có nghĩa tương tự nhau nếu chúng xuất hiện trong những ngữ cảnh tương tự nhau [Harris, 1954; Firth, 1957]. Một biểu diễn từ được học theo giả thuyết này còn được gọi là **biểu diễn từ phân bố** (*distributional word representation*) hay **biểu diễn phân bố** (*distributional representation*)[11]. Tuy nhiên, để dễ đọc, trong suốt cuốn sách này chúng tôi vẫn sẽ dùng các thuật ngữ *vectơ từ* và *biểu diễn từ*. Tiếp theo, chúng tôi giới thiệu một vài phương pháp mô hình hóa sự phân bố của từ trong văn bản, rồi đưa ra một số cải tiến.

Chú giải của người dịch: Giả thuyết phân bố nói gì?

Ý tưởng rất gần với câu tục ngữ "*Hãy cho tôi biết bạn chơi với ai, tôi sẽ nói bạn là người thế nào*". Nhà ngôn ngữ học Firth có câu nổi tiếng: "*Ta biết một từ qua những từ đi cùng nó*". Chẳng hạn, nếu bạn gặp một từ lạ "chôm chôm" trong các câu "chôm chôm chín đỏ", "ăn chôm chôm", "vườn chôm chôm",

[11] Cần lưu ý rằng *biểu diễn phân bố* (*distributional representation*) và *biểu diễn phân tán* (*distributed representation*) là hai khái niệm khác nhau. Biểu diễn phân bố là biểu diễn mô tả sự phân bố của các đơn vị ngôn ngữ trong quá trình sử dụng ngôn ngữ. Một thuật ngữ liên quan là *biểu diễn phi phân bố* (*non-distributional representation*), chỉ những biểu diễn thu được từ các cơ sở dữ liệu từ vựng, chẳng hạn lời giải nghĩa của một từ trong từ điển. Mặt khác, biểu diễn phân tán là một vectơ gồm các biến tương ứng với một số thuộc tính tiềm ẩn của một đơn vị ngôn ngữ. Trái với biểu diễn phân tán, biểu diễn one-hot chỉ mô tả ký hiệu của từ.

bạn có thể đoán đó là một loại quả, vì các từ xung quanh (*chín*, *ăn*, *vườn*) cũng hay đi cùng "xoài", "nhãn". Các phương pháp trong mục này biến ý tưởng ấy thành con số: *đếm* xem mỗi từ hay đi cùng những từ nào.

3.3.1 Ma trận đồng xuất hiện

Trong **ngữ nghĩa học phân bố** (*distributional semantics*), các từ được biểu diễn bằng những mô hình ngữ nghĩa có xét đến nhiều khía cạnh khác nhau của ngữ cảnh. Các mô hình này khác nhau ở cách mô hình hóa ngữ cảnh của một từ, chẳng hạn ngữ cảnh được xét rộng đến đâu, mỗi lần xuất hiện của một từ được đếm như thế nào, số chiều của một phân bố được định nghĩa ra sao, v.v. Trong mục này, giống như phần lớn các mô hình dùng trong NLP, ta giả định rằng biểu diễn từ được học từ một tập hợp các văn bản.

Một văn bản có thể được xem đơn giản là một tập hợp các từ không có thứ tự (tức là giả định túi từ (*bag-of-words*)). Khi đó, ta có thể coi mỗi lần xuất hiện của các từ này là một dấu hiệu ngữ cảnh độc lập. Bằng cách này, sự phân bố của một từ trong ngữ cảnh của nó có thể được mô tả bằng số lần từ đó đồng xuất hiện (*co-occur*) với các từ ngữ cảnh. Điều này được thực hiện bằng cách xây dựng một ma trận đồng xuất hiện (*co-occurrence matrix*), trong đó mỗi ô ghi lại tần suất đồng xuất hiện của một mục ở hàng và một mục ở cột. Hãy xét ví dụ các văn bản sau[12]:

Văn bản 1 *A berry is a small, pulpy, and often edible fruit.*
(Quả mọng là một loại quả nhỏ, nhiều thịt và thường ăn được.)
Văn bản 2 *In botanical terminology, a berry is a simple fruit with seeds and pulp produced from the ovary of a single flower.*
(Theo thuật ngữ thực vật học, quả mọng là một quả đơn có hạt và thịt quả, được tạo ra từ bầu nhụy của một bông hoa duy nhất.)
Văn bản 3 *The term "banana" is also used as the common name for the plants that produce the fruit.*
(Thuật ngữ "chuối" cũng được dùng làm tên gọi thông thường cho các loài cây sinh ra loại quả này.)
Văn bản 4 *Banana seeds are large and hard and spiky and liable to crack teeth.*

[12]Các đoạn văn bản được lấy từ Wikipedia.

(Hạt chuối to, cứng, có gai và dễ làm mẻ răng.)

Văn bản 5 *A banana is an elongated, edible fruit - botanically a berry - produced by several kinds of large herbaceous flowering plants in the genus Musa.*

(Chuối là một loại quả thuôn dài, ăn được – về mặt thực vật học là một quả mọng – do một số loài cây thân thảo lớn có hoa thuộc chi Musa sinh ra.)

Với mỗi cặp từ, ta thu thập tổng số lần chúng đồng xuất hiện trong các văn bản này, dẫn tới một ma trận được gọi là **ma trận đồng xuất hiện từ–từ** (*word-word co-occurrence matrix*) hay **ma trận đồng xuất hiện thuật ngữ–thuật ngữ** (*term-term co-occurrence matrix*). Dưới đây là một phần của ma trận cho các văn bản trên (các từ tiếng Anh được giữ nguyên vì ma trận được đếm trên chính các văn bản tiếng Anh này).

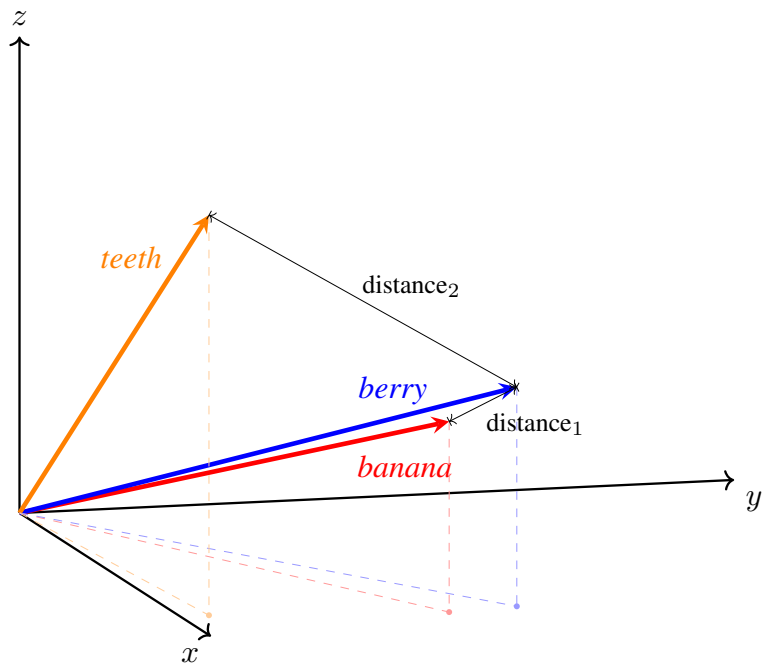
	flowering	*fruit*	*herbaceous*	…	*often*	*plants*	*seeds*
berry	1	3	1	…	1	1	1
terminology	0	1	0	…	0	0	1
common	0	1	0	…	0	1	0
teeth	0	0	0	…	0	0	1
banana	1	2	1	…	0	2	1
simple	0	1	0	…	0	0	1
and	0	2	0	…	1	0	2

Trong ma trận này, mỗi từ ở hàng gắn với một vectơ từ có $|V|$ phần tử. Các con số trong các phần tử mô tả mức độ thường xuyên mà từ ở hàng đồng xuất hiện với các từ ngữ cảnh khác nhau, tức là một từ cho trước phân bố như thế nào trong các "ngữ cảnh" khác nhau. Theo nghĩa hình học, nếu hai từ có phân bố tương tự nhau khi đồng xuất hiện với cùng một nhóm từ ngữ cảnh thì góc giữa hai vectơ từ của chúng sẽ nhỏ[13]. Ví dụ, nếu ta coi các từ này là các vectơ trong một không gian vectơ, thì *berry* (quả mọng) gần *banana* (chuối) hơn là *teeth* (răng) (xem Hình 3.4). Trực giác hình học này là nền tảng của nhiều mô hình biểu diễn. Các ví dụ khác sẽ được trình bày trong Chương 4 và Chương 5.

[13] Góc giữa hai vectơ không phụ thuộc vào độ lớn của chúng. Nếu các vectơ được chuẩn hóa theo một cách nào đó (ví dụ theo chuẩn của vectơ), thì các vectơ tương tự nhau có nghĩa là phần lớn các phần tử của hai vectơ có giá trị gần nhau.

Chú giải của người dịch: Đọc ma trận đồng xuất hiện

Hàng *berry* là $[1, 3, 1, \ldots, 1, 1, 1]$: từ *berry* đi cùng *fruit* 3 lần, cùng *flowering* 1 lần, v.v. Hàng *banana* là $[1, 2, 1, \ldots, 0, 2, 1]$ – khá giống hàng *berry*. Còn hàng *teeth* là $[0, 0, 0, \ldots, 0, 0, 1]$, hầu như không chung "bạn bè" với *berry*. Mỗi hàng chính là một vectơ nhiều chiều (mỗi cột là một tọa độ). Hai vectơ "cùng hướng" (góc nhỏ) nghĩa là hai từ hay xuất hiện với những từ giống nhau, nên có lẽ có nghĩa gần nhau. Đây chính là ý tưởng góc giữa hai vectơ mà bạn đã học ở lớp 10, chỉ là với nhiều tọa độ hơn.



Hình 3.4: Các vectơ từ trong một không gian vectơ được xây dựng từ thống kê đồng xuất hiện của từ trên dữ liệu tiếng Anh của WMT 2012. Tất cả các vectơ đều được chuẩn hóa và được biểu diễn bằng các mũi tên. Để trực quan hóa, ta chiếu các vectơ này từ không gian nhiều chiều xuống không gian 3 chiều bằng phép phân tích thành phần chính. Đúng như kỳ vọng, *berry* gần *banana* hơn là gần *teeth*. (Trên hình: *distance* = khoảng cách.)

Một vấn đề của phương pháp này là khoảng cách giữa các từ không

được tính đến, mặc dù mối tương quan không còn mạnh lắm khi từ ngữ cảnh ở xa. Một giải pháp đơn giản là giới hạn các từ ngữ cảnh trong một cửa sổ, gọi là **cửa sổ ngữ cảnh** (*context window*), hay gọi tắt là cửa sổ [Lund và Burgess, 1996]. Ví dụ, với mỗi từ trong một văn bản, ta chỉ đếm các từ ở vị trí -2 đến $+2$ xung quanh nó (tức là một cửa sổ kích thước 5).

Lưu ý rằng các vectơ từ học được bằng mô hình túi từ ở Mục 3.2 là một trường hợp đặc biệt của ma trận đồng xuất hiện. Trong ví dụ đó, toàn bộ kho ngữ liệu chỉ gồm một văn bản duy nhất, và một hàm chỉ thị đánh dấu sự có mặt thay vì tần suất. Ngoài các phép đếm đơn giản hay các chỉ thị nhị phân, các phần tử của vectơ từ có thể được điền bằng nhiều thước đo mức độ liên kết khác nhau. Trong thực tế, giá trị của một phần tử trong vectơ từ có thể được xem là mức độ tương ứng giữa các từ. Nếu hai từ tương quan với nhau trong một ngữ cảnh nào đó, một hàm đặc trưng có thể gán cho chúng một điểm số theo bất kỳ cách nào. Điểm số này không nhất thiết phải là một số đếm, mà có thể là một số thực bất kỳ. Do đó, bài toán có thể được phát biểu thành việc đo độ mạnh liên kết giữa các từ. Thông thường người ta định nghĩa thước đo như vậy dựa trên các mô hình tương quan. Trong thống kê, tương quan mô tả mức độ hai biến liên hệ với nhau, được đo bằng các **hệ số tương quan** (*correlation coefficients*). Các hệ số tương quan phổ biến gồm hệ số tương quan Pearson (r của Pearson), hệ số tương quan hạng Spearman (ρ của Spearman), v.v.[14]. Trong NLP, một thước đo được dùng rộng rãi là **thông tin tương hỗ theo điểm** (*pointwise mutual information*) (**PMI**) [Church và Hanks, 1990]. Gọi a và b là hai từ. Dạng toán học của PMI là

$$\text{PMI}(a, b) \;=\; \log \frac{\Pr(a, b)}{\Pr(a)\Pr(b)}, \tag{3.21}$$

trong đó $\Pr(a, b)$ là xác suất đồng thời để a và b cùng xuất hiện, và $\Pr(a)$ (hoặc $\Pr(b)$) là xác suất xuất hiện của a (hoặc b). Các xác suất này có thể được ước lượng đơn giản trên văn bản bằng phương pháp tần suất tương đối[15]. Cho một từ a và một bộ từ vựng gồm các từ ngữ cảnh $\{b_1, \cdots, b_{|V|}\}$,

[14] Một số hệ số tương quan giả định dữ liệu tuân theo những phân phối nhất định. Ví dụ, hệ số tương quan Pearson được tính dựa trên hai biến tuân theo phân phối chuẩn.

[15] Một vấn đề của PMI là thước đo này trở nên không ổn định khi các từ hiếm. Ví dụ, nếu một từ rất hiếm tình cờ xuất hiện trong một văn bản, thì giá trị PMI giữa từ này và bất kỳ từ nào khác trong văn bản đó sẽ lớn một cách vô lý.

vectơ từ dựa trên PMI của a được viết là

$$e(a) \;=\; \Big[\mathrm{PMI}(a, b_1) \;\; \cdots \;\; \mathrm{PMI}(a, b_{|V|})\Big]. \tag{3.22}$$

Chú giải của người dịch: Ý nghĩa của PMI qua một ví dụ số

Nếu hai từ a, b *độc lập* (không liên quan gì nhau) thì, như ở xác suất THPT, $\Pr(a, b) = \Pr(a)\Pr(b)$, nên tỉ số bằng 1 và $\mathrm{PMI} = \log 1 = 0$. PMI dương nghĩa là hai từ đi cùng nhau *thường hơn* mức ngẫu nhiên; PMI âm nghĩa là chúng "tránh" nhau. Ví dụ: giả sử trong kho ngữ liệu, $\Pr(\text{“trà”}) = 0.01$, $\Pr(\text{“đá”}) = 0.02$. Nếu độc lập, xác suất gặp cặp "trà đá" chỉ là $0.01 \times 0.02 = 0.0002$. Nhưng thực tế ta đếm được $\Pr(\text{trà, đá}) = 0.002$, gấp 10 lần, nên $\mathrm{PMI} = \log_2 10 \approx 3.3$ (nếu dùng log cơ số 2): hai từ này liên kết rất chặt. Chú thích chân trang 15 cảnh báo: với từ rất hiếm, $\Pr(a)$ rất nhỏ ở mẫu số khiến PMI bị "thổi phồng".

Các hệ số tương quan thường được dùng để kiểm định xem hai biến có liên hệ (tuyến tính) với nhau hay không. Vì vậy, một phương pháp khác là định nghĩa mỗi phần tử của vectơ từ là kết quả của một phép kiểm định. Ví dụ, phần tử (a, b) nhận giá trị 1 nếu hệ số tương quan giữa hai từ a và b lớn hơn một ngưỡng, hoặc nếu mối tương quan giữa a và b được kiểm định giả thuyết ủng hộ một cách đủ mạnh.

Tuy nhiên, việc mô hình hóa từ bằng các vectơ gồm điểm tương quan giữa các từ phần nào giới hạn phạm vi thông tin ngữ cảnh mà ta có thể sử dụng. Một cách tiếp cận khác ở mức vĩ mô là phân tích xem các từ phân bố như thế nào trên toàn bộ các văn bản, tạo thành một **ma trận đồng xuất hiện từ–văn bản** (*word-document co-occurrence matrix*) hay **ma trận đồng xuất hiện thuật ngữ–văn bản** (*term-document co-occurrence matrix*). Ví dụ, với các văn bản đã nêu ở trên, ta có thể xây dựng ma trận

	Văn bản 1	Văn bản 2	Văn bản 3	Văn bản 4	Văn bản 5
berry	1	1	0	0	1
terminology	0	1	0	0	0
common	0	0	1	0	0
teeth	0	0	0	1	0
banana	0	0	1	1	1
simple	0	1	0	0	0
and	1	1	0	2	0

Bảng 3.1: Các hàm của sơ đồ trọng số theo tần suất từ. $\text{count}(a; d)$ đếm số lần xuất hiện của từ a trong văn bản d.

Phần tử	Dạng toán học
Nhị phân	$\text{tf}(a, d) = \begin{cases} 1 & a \text{ xuất hiện trong } d \\ 0 & \text{ngược lại} \end{cases}$
Đếm	$\text{tf}(a, d) = \text{count}(a; d)$
Đếm lũy thừa	$\text{tf}(a, d) = \text{count}(a; d)^{\alpha}$
Đếm theo thang log	$\text{tf}(a, d) = \log(1 + \text{count}(a; d))$
Đếm chuẩn hóa (hay tần suất)	$\text{tf}(a, d) = \frac{\text{count}(a;d)}{\sum_{a'} \text{count}(a';d)}$

Trong ma trận này, giá trị của phần tử (a, d) được định nghĩa là số lần từ a xuất hiện trong văn bản d, thể hiện độ mạnh liên kết giữa a và d. Giá trị này thường được gọi là **tần suất từ** (*term frequency*) (**TF**) của a trong d (ký hiệu là $\text{tf}(a, d)$). Ngoài ra, ta cũng có thể dùng một hàm chỉ thị 0–1 để đánh dấu sự có mặt của từ (xem Mục 3.2). Xem Bảng 3.1 để biết một vài biến thể của hàm trọng số TF.

Cấu trúc này phục vụ hai mục đích cùng lúc: mỗi hàng cung cấp một biểu diễn vectơ của một từ cụ thể, còn mỗi cột đóng vai trò là biểu diễn vectơ của cả một văn bản. Ma trận đồng xuất hiện từ–văn bản này về mặt khái niệm giống hệt mô hình túi từ đã giới thiệu cho bài toán phân loại văn bản ở Chương 1, trong đó thứ tự từ bị bỏ đi nhưng tần suất tương đối được giữ lại. Ở đây, ta thực hiện việc vectơ hóa văn bản bằng mô hình này trên một tập hợp văn bản.

3.3.2 TF-IDF

Mô hình hóa liên kết giữa từ và văn bản là việc quan trọng đối với nhiều bài toán NLP. Một cải tiến đáng kể so với việc dùng số đếm tần suất thô để xây dựng các vectơ này là sơ đồ trọng số **tần suất từ – nghịch đảo tần suất văn bản** (*term frequency-inverse document frequency*) (**TF-IDF**). Cho một tập văn bản D, TF-IDF gán một điểm số cho mỗi cặp từ–văn bản (a, d) theo phương trình sau:

$$\text{tfidf}(a, d, D) \;=\; \text{tf}(a, d) \cdot \text{idf}(a, D), \tag{3.23}$$

trong đó

- $\text{tf}(a, d)$ là tần suất từ (xem Bảng 3.1). Khi $\text{tf}(a, d)$ lớn, từ a là một dấu hiệu tốt cho văn bản d. Ngược lại, khi $\text{tf}(a, d)$ nhỏ, liên kết giữa từ và văn bản không mạnh lắm.
- $\text{idf}(a, D)$ là **nghịch đảo tần suất văn bản** (*inverse document frequency*) (**IDF**). Nó được xây dựng dựa trên thực tế là những từ phổ biến trên nhiều văn bản thì mang ít thông tin hơn. Ví dụ, với một tập văn bản về thể thao, rất có thể ta sẽ thấy *player* (cầu thủ) và *players* (các cầu thủ) trong hầu hết các văn bản. Trong trường hợp này, các từ *player* và *players* ít có ích trong việc phân biệt các văn bản hay ngữ cảnh khác nhau. Gọi $\text{df}(a, D)$ là số văn bản trong D có chứa từ a. Một dạng phổ biến của $\text{idf}(a, D)$ là

$$\text{idf}(a, D) \;=\; \log \frac{|D|}{\text{df}(a, D)}. \tag{3.24}$$

  Phương trình này "phạt" những từ xuất hiện thường xuyên trên toàn kho ngữ liệu. Tương tự, ta có thể có một phiên bản làm trơn của $\text{idf}(a, D)$

$$\text{idf}(a, D) \;=\; \log \frac{|D|}{\text{df}(a, D) + 1} + 1. \tag{3.25}$$

Sử dụng sơ đồ trọng số TF-IDF, ta có thể xây dựng một ma trận từ–văn bản trong đó giá trị của phần tử (a, d) là $\text{tfidf}(a, d, D)$. Như đã mô tả ở trên, mỗi hàng của ma trận này đóng vai trò là biểu diễn vectơ của từ tương ứng. Theo truyền thống, TF-IDF và ma trận thuật ngữ–văn bản được dùng rộng rãi để biểu diễn văn bản. Chẳng hạn, trong truy xuất thông tin (*information retrieval*), cả truy vấn lẫn văn bản đều có thể được biểu diễn thành các vectơ cột TF-IDF, cho phép ta tính độ liên quan giữa truy vấn và văn bản thông qua độ tương tự vectơ. Mặc dù các mô hình không gian vectơ trong truy xuất thông tin nằm ngoài phạm vi của chương này, bạn đọc có thể tham khảo các giáo trình chuẩn để có cái nhìn đầy đủ [Manning và cộng sự, 2008; Buttcher và cộng sự, 2016].

Chú giải của người dịch: Tính TF-IDF bằng tay

Dùng 5 văn bản ở trên, $|D| = 5$. Từ *banana* xuất hiện trong 3 văn bản (Văn bản 3, 4, 5), nên theo Phương trình (3.24), $\text{idf} = \log(5/3) \approx 0.51$ (log tự nhiên). Từ *teeth* chỉ xuất hiện trong 1 văn bản, nên $\text{idf} = \log(5/1) \approx 1.61$. Trong Văn bản 4, cả hai từ đều xuất hiện 1 lần ($\text{tf} = 1$), nhưng $\text{tfidf}(\textit{teeth}) \approx$

1.61 lớn hơn tfidf(*banana*) ≈ 0.51: *teeth* "đặc trưng" cho Văn bản 4 hơn. Một từ có trong *mọi* văn bản (như "là", "của") sẽ có idf $= \log 1 = 0$, tức bị triệt tiêu hoàn toàn. Đó là lý do IDF được gọi là "phạt" các từ phổ biến.

3.3.3 Các mô hình ít chiều

Các ma trận đồng xuất hiện thường có số chiều rất lớn. Ví dụ, giả sử có một bộ từ vựng gồm 20,000 từ khác nhau và một tập hợp 10,000,000 văn bản. Khi đó, ma trận đồng xuất hiện từ–văn bản có $20{,}000 \times 10{,}000{,}000 = 2 \times 10^{11}$ phần tử. Với gánh nặng tính toán của một mô hình như vậy, việc biểu diễn mỗi từ bằng một vectơ 10,000,000 chiều và mỗi văn bản bằng một vectơ 20,000 chiều là không thực tế. Thay vào đó, ta mong muốn biểu diễn của một từ (hay một văn bản) chỉ cần một số lượng đặc trưng nhỏ vừa phải. Trong tiểu mục này, chúng ta bàn về các cách tiếp cận chuẩn để biến các ma trận đồng xuất hiện nhiều chiều thành các biểu diễn ít chiều hơn. Vì các kỹ thuật này đã được nghiên cứu sâu rộng và áp dụng thành công trong nhiều lĩnh vực [Barber, 2012; Wright và Ma, 2022], chúng tôi bỏ qua các chứng minh toán học bên dưới và tập trung vào việc áp dụng chúng trong thực tế để thu được các vectơ từ và vectơ văn bản.

1. Phân tích ngữ nghĩa tiềm ẩn

Trong NLP, **phân tích ngữ nghĩa tiềm ẩn** (*latent semantic analysis*) (**LSA**) là một phương pháp tìm kiếm cấu trúc ngữ nghĩa tiềm ẩn đằng sau các liên kết từ–văn bản [Deerwester và cộng sự, 1990; Landauer và cộng sự, 1998][16]. Nó giả định rằng cả từ lẫn văn bản đều có thể được biểu diễn bằng các vectơ ít chiều được chắt lọc từ ma trận đồng xuất hiện, trong khi vẫn giữ được tính chất của mô hình không gian vectơ ban đầu, chẳng hạn góc giữa các vectơ là nhỏ đối với các từ tương tự nhau.

Cụ thể hơn, LSA phân tích ma trận đồng xuất hiện thành tích của một ma trận biểu diễn từ, một ma trận biểu diễn văn bản, và một ma trận thứ ba nối hai ma trận đầu. Về mặt toán học, điều này có thể được đặt thành một quá trình **phân tích giá trị suy biến** (*singular value decomposition*) (**SVD**) [Stewart, 1993]. Gọi $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$ là một ma trận đồng xuất hiện

[16] Phân tích ngữ nghĩa tiềm ẩn còn được gọi là **lập chỉ mục ngữ nghĩa tiềm ẩn** (*latent semantic indexing*) (**LSI**). Thuật ngữ này được dùng nhiều hơn trong truy xuất thông tin và các lĩnh vực liên quan.

trên bộ từ vựng V và tập văn bản D. SVD cho ta một phân tích của $\mathbf{M}$:

$$\mathbf{M} = \mathbf{P}\mathbf{\Sigma}\mathbf{Q}^\top, \tag{3.26}$$

trong đó $\mathbf{P} \in \mathbb{R}^{|V|\times r}$, $\mathbf{\Sigma} \in \mathbb{R}^{r\times r}$ và $\mathbf{Q}^\top \in \mathbb{R}^{r\times|D|}$. Trong phân tích này, các ma trận $\mathbf{P}$ và $\mathbf{Q}^\top$ là **nửa unita** (*semi-unitary*) (hay **nửa trực giao** (*semi-orthogonal*)), nghĩa là các cột của cả $\mathbf{P}$ và $\mathbf{Q}$ đều là các **vectơ trực chuẩn** (*orthonormal vectors*) với nhau (các vectơ đơn vị đôi một trực giao). Do đó, các cột của $\mathbf{P}$ và $\mathbf{Q}$ lần lượt tạo thành các cơ sở trực chuẩn cho không gian cột và không gian hàng của $\mathbf{M}$, trong đó r là hạng của $\mathbf{M}$. Điều này bảo đảm $\mathbf{M}$ được biểu diễn bằng số chiều tối thiểu cần thiết. Ma trận $\mathbf{\Sigma}$ là ma trận đường chéo:

$$\mathbf{\Sigma} = \begin{bmatrix} \sigma_1 & 0 & \cdots & 0 \\ 0 & \sigma_2 & \cdots & 0 \\ \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & \cdots & \sigma_r \end{bmatrix}. \tag{3.27}$$

Các phần tử trên đường chéo $\{\sigma_1, \cdots, \sigma_r\}$ đều là các số thực không âm, và được gọi là các **giá trị suy biến** (*singular values*) của $\mathbf{M}$. Thông thường, $\{\sigma_1, \cdots, \sigma_r\}$ được sắp theo thứ tự giảm dần (tức là $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r$). Cách sắp xếp này giúp chuẩn hóa phép phân tích. Nếu ta viết $\mathbf{P}$ thành một dãy các vectơ cột (gọi là các **vectơ suy biến trái** (*left-singular vectors*))

$$\mathbf{P} = \Big[\mathbf{p}_1, \cdots, \mathbf{p}_r\Big], \tag{3.28}$$

và $\mathbf{Q}^\top$ thành một dãy các vectơ hàng (gọi là các **vectơ suy biến phải** (*right-singular vectors*))

$$\mathbf{Q}^\top = \begin{bmatrix} \mathbf{q}_1^\top \\ \vdots \\ \mathbf{q}_r^\top \end{bmatrix}, \tag{3.29}$$

thì ta có thể viết $\mathbf{M}$ thành

$$\mathbf{M} = \sum_{i=1}^{r} \sigma_i \mathbf{p}_i \mathbf{q}_i^\top. \tag{3.30}$$

Chú giải của người dịch: SVD bằng hình ảnh đời thường

Một ma trận lớn có thể hình dung như một bức ảnh. Phương trình (3.30) nói rằng bức ảnh đó bằng tổng của r "lớp" đơn giản $\sigma_i \mathbf{p}_i \mathbf{q}_i^\top$; mỗi lớp là tích của một cột $\mathbf{p}_i$ với một hàng $\mathbf{q}_i^\top$ (một bảng "cửu chương" kiểu $p(a) \times q(b)$), được nhân với "độ quan trọng" σ_i. Ví dụ với $\mathbf{p} = \begin{bmatrix}1\\2\end{bmatrix}$, $\mathbf{q}^\top = [3 \quad 1]$ thì $\mathbf{pq}^\top = \begin{bmatrix}3 & 1\\6 & 2\end{bmatrix}$. Vì các σ_i được xếp giảm dần, những lớp đầu chứa phần lớn "nội dung" của ma trận. *Hạng* r của ma trận có thể hiểu là số "lớp" độc lập tối thiểu cần để ghép lại đúng ma trận đó. Các vectơ *trực chuẩn* là các vectơ có độ dài 1 và đôi một vuông góc (tích vô hướng bằng 0), giống các trục Ox, Oy, Oz trong không gian.

Để thu được biểu diễn từ, ta diễn giải hàng thứ i của $\mathbf{P}$ là một vectơ đặc trưng r chiều $\mathbf{e}_i$ cho từ thứ i, ký hiệu a_i, trong bộ từ vựng. Nói cách khác, biểu diễn vectơ của a_i là

$$\mathbf{e}_i = \begin{bmatrix} p_1(i) & \cdots & p_r(i) \end{bmatrix}. \tag{3.31}$$

Tương tự, biểu diễn vectơ của một văn bản d_j có thể được viết là

$$\mathbf{h}_j = \begin{bmatrix} q_1(j) & \cdots & q_r(j) \end{bmatrix}. \tag{3.32}$$

Bằng cách này, ta có hai mô hình biểu diễn riêng biệt cho từ và văn bản: $\mathbf{P}$ đảm nhận biểu diễn từ, còn $\mathbf{Q}$ đảm nhận biểu diễn văn bản. Như vậy, ta có thể xem $\mathbf{M}$ là tích của các mô hình biểu diễn này:

$$\begin{aligned}\mathbf{M} &= \mathbf{P}\boldsymbol{\Sigma}\mathbf{Q}^\top \\ &= \underbrace{\begin{bmatrix}\mathbf{e}_1\\ \vdots \\ \mathbf{e}_{|V|}\end{bmatrix}}_{\text{các từ}} \begin{bmatrix}\sigma_1 & \cdots & 0\\ \vdots & \ddots & \vdots \\ 0 & \cdots & \sigma_r\end{bmatrix} \underbrace{\begin{bmatrix}\mathbf{h}_1^\top & \cdots & \mathbf{h}_{|D|}^\top\end{bmatrix}}_{\text{các văn bản}}.\end{aligned} \tag{3.33}$$

Trong thực tế, hạng r thường nhỏ hơn nhiều so với $|V|$ và $|D|$. Do đó, với mỗi từ (hoặc mỗi văn bản), ta có một biểu diễn mới với số chiều nhỏ hơn nhiều so với biểu diễn chứa trong ma trận đồng xuất hiện. Một cải tiến nữa là chỉ dùng r^* giá trị suy biến lớn nhất (tức là $\{\sigma_1, \cdots, \sigma_{r^*}\}$) và

bỏ đi phần còn lại. Kết quả là ta chỉ giữ r^* vectơ suy biến trái đầu tiên trong $\mathbf{P}$ và r^* vectơ suy biến phải đầu tiên trong $\mathbf{Q}$. Ở đây $r^* < r$ là một siêu tham số xác định số vectơ trong $\mathbf{P}$ và $\mathbf{Q}$, tức là số đặc trưng dùng để mô tả một từ hay một văn bản. Bằng cách này, ta có một phân tích mới của $\mathbf{M}$:

$$\mathbf{M} \approx \sum_{i=1}^{r^*} \sigma_i \mathbf{p}_i \mathbf{q}_i^\top. \tag{3.34}$$

Vế phải của Phương trình (3.34) còn được gọi là một **xấp xỉ hạng thấp** (*low-rank approximation*) của $\mathbf{M}$. Bằng cách chọn r^*, ta có thể xấp xỉ $\mathbf{M}$ bằng một ma trận có hạng tùy ý nhỏ hơn r.

Có nhiều thuật toán để cài đặt SVD [Cline và Dhillon, 2014]. Thực tế, phần lớn các cài đặt SVD hiện đại đều hiệu quả và có khả năng mở rộng. Ta có thể dùng chúng như những bộ công cụ có sẵn trong các ứng dụng NLP.

Chú giải của người dịch: Vì sao giữ lại ít chiều vẫn tốt?

Giả sử $\sigma_1 = 50$, $\sigma_2 = 20$, $\sigma_3 = 0.5$, $\sigma_4 = 0.1$. Hai lớp đầu đóng góp gần như toàn bộ ma trận, nên chọn $r^* = 2$ thì sai số rất nhỏ, nhưng mỗi từ chỉ còn được mô tả bằng 2 số thay vì hàng triệu số. Hơn nữa, những lớp nhỏ thường là "nhiễu" (do việc dùng từ ngẫu nhiên), nên bỏ chúng đi đôi khi còn làm lộ rõ hơn quan hệ ngữ nghĩa "tiềm ẩn", ví dụ *xe hơi* và *ô tô* hiếm khi cùng nằm trong một văn bản nhưng lại có vectơ ít chiều rất gần nhau.

2. Phân tích thành phần chính

Trong phân tích dữ liệu, **phân tích thành phần chính** (*principal component analysis*) (**PCA**) là một kỹ thuật giảm chiều (*dimensionality reduction*) được dùng rộng rãi. Cho một tập điểm dữ liệu, PCA tìm một dãy các hướng trực giao với nhau trong không gian tọa độ sao cho phương sai của các điểm dữ liệu theo những hướng này là lớn nhất. Các hướng này thường được biểu diễn bằng các vectơ đơn vị, gọi là **tải thành phần chính** (*principal component loadings*) hay **hệ số thành phần chính** (*principal component coefficients*). Kết quả là chúng tạo thành một không gian tọa độ mới, và ta có thể ánh xạ các điểm dữ liệu đã cho vào đó bằng một phép biến đổi tuyến tính trực giao.

Xét một ma trận đồng xuất hiện từ–văn bản $\mathbf{M} \in \mathbb{R}^{|V| \times |D|}$, trong đó

mỗi hàng là một vectơ từ (hay vectơ đặc trưng) $|D|$ chiều. PCA định nghĩa một ánh xạ tuyến tính từ $\mathbb{R}^{|D|}$ sang $\mathbb{R}^p$, nghĩa là ta biến mỗi vectơ từ $|D|$ chiều thành một vectơ từ p chiều. Ánh xạ này được cho bởi

$$\mathbf{N} \;=\; \mathbf{MC}, \tag{3.35}$$

trong đó $\mathbf{N} \in \mathbb{R}^{|V|\times p}$ biểu diễn ma trận các vectơ từ đã được ánh xạ trên bộ từ vựng V, và $\mathbf{C} \in \mathbb{R}^{|D|\times p}$ là ma trận của ánh xạ tuyến tính. Khi đó, ta có thể viết $\mathbf{C}$ thành một dãy các vectơ cột

$$\mathbf{C} \;=\; \begin{bmatrix}\mathbf{c}_1 & \cdots & \mathbf{c}_p\end{bmatrix}. \tag{3.36}$$

Mỗi vectơ cột $\mathbf{c}_i = \begin{bmatrix} c_i(1) \\ \vdots \\ c_i(|D|)\end{bmatrix}$ là một nhóm hệ số thành phần chính, biểu thị một hàm tuyến tính kết hợp các đặc trưng đầu vào thành một đặc trưng mới. Ví dụ, nếu ta xem $\mathbf{M}$ là giá trị của một loạt hàm đặc trưng (chẳng hạn các vectơ cột $\{\mathbf{m}_1, \cdots, \mathbf{m}_{|D|}\}$), ta có thể ánh xạ $\mathbf{M}$ sang một không gian đặc trưng mới theo $\mathbf{c}_i$:

$$\begin{aligned}\mathbf{Mc}_i &= \begin{bmatrix}\mathbf{m}_1 & \cdots & \mathbf{m}_{|D|}\end{bmatrix}\begin{bmatrix} c_i(1) \\ \vdots \\ c_i(|D|)\end{bmatrix} \\ &= \sum_{k=1}^{|D|} c_i(k)\mathbf{m}_k.\end{aligned} \tag{3.37}$$

Tích $\mathbf{Mc}_i$ (tạo thành cột thứ i của $\mathbf{N}$) là một vectơ cột, trong đó mỗi phần tử biểu diễn giá trị đặc trưng mới của một từ trong V. Trong PCA, các vectơ $\{\mathbf{c}_1, \cdots, \mathbf{c}_p\}$ được tạo ra lần lượt để cực đại hóa phương sai của dữ liệu sau khi chiếu. Như vậy, với mỗi $i \in [1, p]$, các hệ số thành phần chính tối ưu được định nghĩa là:

$$\begin{aligned}\hat{\mathbf{c}}_i &= \arg\max_{\mathbf{c}_i} \mathrm{Var}(\mathbf{Mc}_i) \\ &= \arg\max_{\mathbf{c}_i} \mathbf{c}_i^\top \mathbf{S}\mathbf{c}_i,\end{aligned} \tag{3.38}$$

trong đó $\mathrm{Var}(\mathbf{Mc}_i)$ là phương sai của dữ liệu đã chiếu, và $\mathbf{S}$ là ma trận hiệp phương sai (*covariance matrix*) kích thước $|D| \times |D|$ của $\mathbf{M}$. Để bảo

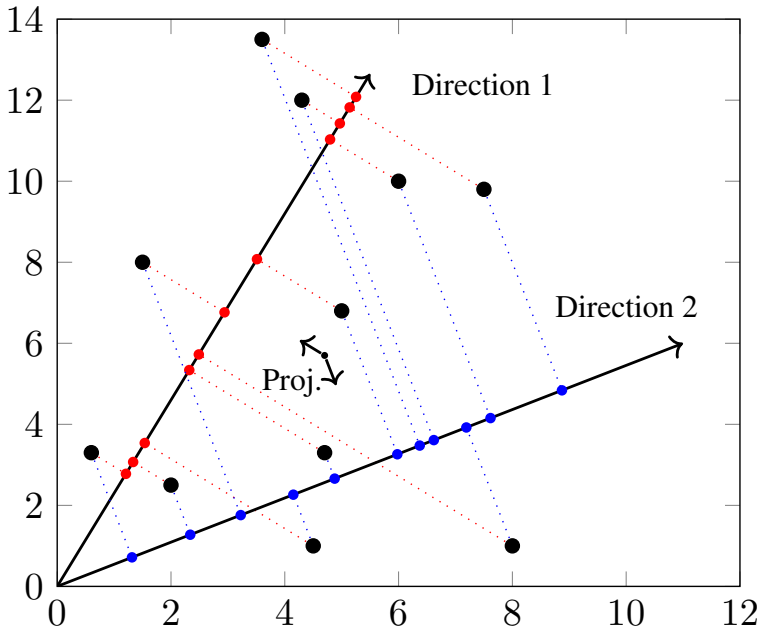
đảm bài toán có nghiệm xác định, người ta thường ràng buộc $\mathbf{c}_i$ là một vectơ đơn vị (tức là $\mathbf{c}_i^\top \mathbf{c}_i = 1$). Dùng một nhân tử Lagrange (*Lagrange multiplier*) λ_i, bài toán tối ưu có ràng buộc này có thể được phát biểu bằng cách định nghĩa hàm Lagrange:

$$\mathcal{L}(\mathbf{c}_i, \lambda_i) \;=\; \mathbf{c}_i^\top \mathbf{S}\mathbf{c}_i - \lambda_i(\mathbf{c}_i^\top \mathbf{c}_i - 1). \tag{3.39}$$

Giải tìm các điểm dừng của hàm Lagrange này cho thấy $\hat{\mathbf{c}}_i$ phải là một vectơ riêng (*eigenvector*) của $\mathbf{S}$, với λ_i là giá trị riêng (*eigenvalue*) tương ứng [Jolliffe, 2002]. Vì $\mathbf{S}$ là một ma trận đối xứng kích thước $|D| \times |D|$, nó có đúng $|D|$ giá trị riêng cùng các vectơ riêng tương ứng đôi một trực giao. Bằng cách sắp xếp các vectơ riêng này theo giá trị riêng tương ứng giảm dần, ta chọn ra p vectơ riêng đầu tiên để tạo thành $\{\hat{\mathbf{c}}_1, \cdots, \hat{\mathbf{c}}_p\}$. Nói cách khác, $\hat{\mathbf{c}}_1$ là vectơ riêng ứng với giá trị riêng lớn nhất, $\hat{\mathbf{c}}_2$ ứng với giá trị riêng lớn thứ hai, và cứ thế tiếp tục. Phép chiếu $\mathbf{M}\hat{\mathbf{c}}_i$ được gọi là **thành phần chính** (*principal component*) thứ i của $\mathbf{M}$.

Một cách hình dung PCA về mặt hình học là xem nó như một phép đổi hệ tọa độ trong không gian Euclid. Với tập dữ liệu $\mathbf{M}$, mỗi hàng biểu diễn tọa độ của một điểm dữ liệu trong một không gian $|D|$ chiều A. PCA tìm cách biểu diễn các điểm dữ liệu này trong một hệ tọa độ mới p chiều B. Chiều thứ i của hệ mới này được xác định bởi hướng của vectơ đơn vị $\mathbf{c}_i$. Để tìm tọa độ thứ i của một điểm dữ liệu trong B, ta chiếu tọa độ ban đầu của nó lên đường thẳng sinh bởi $\mathbf{c}_i$. Hướng tối ưu $\mathbf{c}_i$ được chọn sao cho độ trải rộng (phương sai) của các điểm đã chiếu là lớn nhất. Về bản chất, ta đi tìm trục mà dọc theo nó các điểm dữ liệu phân tán nhiều nhất. Qua quá trình này, ta tạo ra một dãy các hệ số thành phần chính bằng cách lần lượt giải Phương trình (3.38). Hình 3.5 minh họa ý tưởng này bằng cách chiếu dữ liệu 2 chiều xuống một không gian con 1 chiều.

Trong các ứng dụng thực tế, p thường được chọn nhỏ hơn nhiều so với $|D|$, cho phép PCA giảm đáng kể số chiều của biểu diễn từ. Lưu ý rằng PCA là một phương pháp rất tổng quát và được thấy là hữu ích trong nhiều lĩnh vực. Rộng hơn, nếu $\mathbf{M}$ biểu diễn các quan sát trên một tập biến, PCA có thể cô đọng các quan sát này thành một tập nhỏ hơn các biến không tương quan với nhau (các thành phần chính).



Hình 3.5: Minh họa việc biến dữ liệu 2 chiều thành dữ liệu 1 chiều bằng PCA. Xét một tập điểm dữ liệu (biểu diễn bằng các chấm tròn đen) trên một mặt phẳng Euclid. Dùng PCA, ta tìm một hướng (biểu diễn bằng mũi tên) sao cho phương sai của dữ liệu đã chiếu (biểu diễn bằng các chấm tròn màu) theo hướng này là lớn nhất. Một hướng như vậy có thể được biểu diễn bằng một vectơ đơn vị gồm các hệ số thành phần chính. Trong ví dụ này, các hệ số thành phần chính mô tả một không gian tọa độ 1 chiều. Ta có thể ánh xạ dữ liệu từ không gian tọa độ 2 chiều sang không gian tọa độ 1 chiều bằng phép biến đổi tuyến tính. Dữ liệu sau ánh xạ được gọi là thành phần chính của các điểm dữ liệu ban đầu. (Trên hình: *Direction* = Hướng; *Proj.* = phép chiếu.)

Chú giải của người dịch: PCA bằng trực giác

Hãy nhìn Hình 3.5: các chấm đen nằm rải rác theo một dải nghiêng. Nếu buộc phải mô tả mỗi điểm chỉ bằng *một* con số, cách tốt nhất là chọn một trục (như "Direction 1") chạy dọc theo dải đó, rồi lấy hình chiếu vuông góc của mỗi điểm lên trục (giống hình chiếu vectơ ở lớp 10). Theo hướng đó, các hình chiếu trải ra rộng nhất nên giữ lại được nhiều thông tin phân biệt các điểm nhất; còn chiếu lên một hướng mà các hình chiếu dồn cục lại thì mất nhiều thông tin. *Phương sai* chính là thước đo "độ trải rộng" trong thống kê mô tả THPT. Về các ký hiệu: phương pháp *nhân tử Lagrange* là kỹ thuật tìm cực trị khi có ràng buộc (ở đây ràng buộc là $\mathbf{c}_i$ có độ dài 1, nếu không thì cứ kéo dài $\mathbf{c}_i$ là phương sai tăng vô hạn); còn *vectơ riêng* của ma trận $\mathbf{S}$ là vectơ $\mathbf{c}$ mà khi nhân với $\mathbf{S}$ chỉ bị co giãn chứ không đổi hướng: $\mathbf{Sc} = \lambda\mathbf{c}$, với hệ số co giãn λ là *giá trị riêng*.

3. Các phương pháp khác

Giảm chiều là một bài toán nền tảng trong học máy và đã được tổng quát hóa theo nhiều hướng. Chẳng hạn, các mô hình nhúng từ dùng mạng nơ-ron được mô tả trong Mục 3.4 và Mục 3.5 vốn đã học các vectơ từ ít chiều, giá trị thực từ văn bản. Dưới đây chúng tôi trình bày một số phương pháp giảm chiều mà bạn có thể gặp trong các tài liệu về NLP và học máy.

- **Mô hình chủ đề** (*topic models*). Về mặt kỹ thuật, mô hình chủ đề không phải là cách giảm chiều, mà là công cụ mô tả cách các văn bản và các từ được sinh ra dựa trên các phân phối theo chủ đề [Blei, 2012]. Ví dụ, **phân bổ Dirichlet tiềm ẩn** (*latent Dirichlet allocation*) (**LDA**) mô hình hóa quá trình sinh một văn bản bằng cách dùng các phân phối văn bản–chủ đề và chủ đề–từ [Blei và cộng sự, 2003]. Như một sản phẩm phụ, ta thu được một phân phối trên các từ cho mỗi chủ đề, cho biết khả năng một từ xuất hiện khi biết một chủ đề cụ thể. Nếu ta viết tất cả các phân phối chủ đề–từ này thành một ma trận $|V| \times K$, trong đó $|V|$ là số từ và K là số chủ đề, ta sẽ thu được các biểu diễn từ tương tự về mặt khái niệm với những biểu diễn đã mô tả ở các mục trước. Thông thường, K là một số tương đối nhỏ (ví dụ 200). Khi đó, ta có một mô hình ít chiều để biểu diễn từ. Mặc dù LDA không phổ biến lắm trong việc học biểu diễn từ cho các ứng dụng NLP, nó cung cấp một cách biểu

diễn từ dưới dạng các phân phối trên những cấu trúc chủ đề tiềm ẩn.

- **Bộ tự mã hóa** (*auto-encoders*). Bộ tự mã hóa thiếu đủ (*undercomplete auto-encoder*) là một loại mô hình nơ-ron mã hóa các đặc trưng thành những mã ít chiều sao cho có thể khôi phục lại các đặc trưng đầu vào từ các mã đó. Một ưu điểm của bộ tự mã hóa là chúng không đặt giả định nào về cấu trúc ẩn của các đặc trưng. Vì vậy, bộ tự mã hóa học cách biến bất kỳ loại dữ liệu nào thành các biểu diễn ít chiều. Ví dụ, trong Chương 7 ta sẽ thấy các ví dụ áp dụng bộ tự mã hóa để học biểu diễn câu. Để biết thêm chi tiết về bộ tự mã hóa, bạn đọc có thể tham khảo Chương 2.

- **Giảm chiều có giám sát** (*supervised dimensionality reduction*). Các phương pháp giảm chiều truyền thống (như PCA) hoạt động theo cách không giám sát. Tuy nhiên, khi có sẵn nhãn của bài toán đích, việc tận dụng tín hiệu giám sát này sẽ có lợi. Một ví dụ kinh điển là giảm chiều có giám sát cho bài toán phân loại. Trong các phương pháp như **phân biệt tuyến tính Fisher** (*Fisher's linear discriminant*) và **phân tích phân biệt tuyến tính** (*linear discriminant analysis*), dữ liệu nhiều chiều được ánh xạ vào một không gian ít chiều hơn sao cho các lớp đã định trước được tách biệt nhau nhiều nhất. Ý tưởng này được tổng quát hóa cho các không gian đích nhiều chiều thông qua phân tích biến chính tắc (*Canonical Variates analysis*) [Barber, 2012].

- **Lựa chọn đặc trưng** (*feature selection*). Lựa chọn đặc trưng là quá trình xác định một tập con gồm những đặc trưng liên quan nhất để biểu diễn một đối tượng, qua đó giảm số chiều của biểu diễn một cách tường minh. Đây là một chủ đề rộng trong học máy với nhiều phương pháp [Guyon và Elisseeff, 2003; Liu và Motoda, 2012]. Cách đơn giản nhất là xem nó như một bài toán tìm kiếm: ta tìm kiếm trong không gian các tập con đặc trưng sao cho các đặc trưng được chọn làm cực đại (hoặc cực tiểu) một mục tiêu nào đó. Nói chung, cách thiết kế mục tiêu phụ thuộc vào bài toán mà ta áp dụng các đặc trưng. Sự gắn kết chặt chẽ này khiến việc lựa chọn đặc trưng trở nên khó khăn, vì nó đòi hỏi cân bằng nhiều yếu tố, chẳng hạn thước đo hiệu năng của bài toán đích, hiệu quả tìm kiếm, và cách

biểu diễn mỗi tập con đặc trưng. Lưu ý rằng lựa chọn đặc trưng thường được bàn đến trong học có giám sát, vốn cần dữ liệu có nhãn để tính hàm mất mát phục vụ tối ưu hóa. Bạn đọc có thể tham khảo bài tổng quan của Solorio-Fernández và cộng sự [2020] về các phương pháp lựa chọn đặc trưng không giám sát.

Trong thống kê, nhiều phương pháp có thể được xếp vào khuôn khổ giảm chiều và có liên hệ với những gì ta đã bàn trong mục này. Ví dụ, **phân tích nhân tố** (*factor analysis*) là một phương pháp tương tự PCA vì cả hai đều tìm một ánh xạ tuyến tính từ các biến đầu vào sang một số ít biến mới hơn. Điểm khác nhau là phân tích nhân tố tập trung vào mô hình hóa phần phương sai chung của các biến, còn PCA tập trung vào cực đại hóa phương sai của dữ liệu đã chiếu. Một ví dụ khác là **phân tích thành phần độc lập** (*independent component analysis*) (**ICA**). Khác với PCA, mục tiêu của ICA là tìm các thành phần độc lập có thể tách rời nhau theo phép cộng. Có thể tìm thêm các ví dụ khác trong các giáo trình học máy và thống kê [McClave và Sincich, 2006; Freedman và cộng sự, 2007; Barber, 2012].

3.4 Học nhúng từ từ các mô hình ngôn ngữ nơ-ron

Đếm số lần đồng xuất hiện từ–từ hay từ–văn bản là một phương pháp đơn giản, trực tiếp để biểu diễn từ dựa trên tính chất phân bố của chúng trong một kho ngữ liệu. Mặc dù phương pháp này hiệu quả trong nhiều ứng dụng, nó áp đặt một ràng buộc chặt lên biểu diễn từ: các phần tử của một vectơ từ phải diễn giải được như bằng chứng tường minh về việc từ đó phân bố như thế nào trong các ngữ cảnh khác nhau. Lý tưởng nhất, ta muốn biểu diễn từ ở dạng tổng quát hơn, cụ thể là các vectơ dày đặc, giá trị thực (thường được gọi là **vectơ nhúng từ** (*word embeddings*)), mà không áp đặt ràng buộc hay giả định định sẵn nào về ý nghĩa cụ thể của từng phần tử trong vectơ.

Học vectơ từ mà không có ràng buộc thì phải trả giá. Khác với các phương pháp dựa trên đếm trình bày ở Mục 3.3, ta không dựa vào các quy tắc kinh nghiệm hay tri thức thống kê có trước để ước lượng giá trị của vectơ. Thay vào đó, ta muốn quy nạp các biểu diễn có ý nghĩa trực tiếp

từ dữ liệu. Một thách thức chính ở đây là không có "đáp án chuẩn" (*gold standard*), vì việc gán nhãn thủ công các biểu diễn liên tục, nhiều chiều cho cả một bộ từ vựng là không khả thi. Vì vậy, ta thường đặt việc quy nạp vectơ từ thành một mục tiêu phụ bên trong một bài toán được định nghĩa rõ ràng (thường gọi là **bài toán nền** (*background task*)). Các vectơ từ học được xuất hiện như một sản phẩm phụ giá trị của việc huấn luyện trên bài toán nền này.

Một ví dụ phổ biến là quy nạp vectơ từ từ các mô hình ngôn ngữ nơ-ron (*neural language models*) (NLM). Hãy nhớ lại NLM đã mô tả ở Chương 2. Mục tiêu của nó là dự đoán xác suất của một từ khi biết các từ đứng trước nó [Bengio và cộng sự, 2003a]. Một cách hình thức hơn, gọi w_i là từ ta muốn dự đoán, và $\{w_{i-n+1}, \cdots, w_{i-1}\}$ là các từ ngữ cảnh ta đã thấy. Đầu tiên, các từ ngữ cảnh $\{w_{i-n+1}, \cdots, w_{i-1}\}$ được chiếu thành các vectơ dày đặc d_e chiều $\{\mathbf{e}_{i-n+1}, \cdots, \mathbf{e}_{i-1}\}$ thông qua một tầng nhúng (*embedding layer*). Giả sử $\mathbf{1}_{w_j}$ là biểu diễn one-hot của từ j (một vectơ hàng kích thước $|V|$), vectơ từ dày đặc $\mathbf{e}_j$ được cho bởi

$$\mathbf{e}_j \;=\; \mathbf{1}_{w_j}\mathbf{C}, \tag{3.40}$$

trong đó $\mathbf{C} \in \mathbb{R}^{|V|\times d_e}$ là tham số của tầng nhúng. $\mathbf{C}$ thường được gọi là bảng nhúng từ (*word embedding table*), trong đó hàng thứ k là biểu diễn của từ thứ k trong V.

Chú giải của người dịch: Nhân vectơ one-hot với bảng nhúng = tra bảng

Giả sử bộ từ vựng có 3 từ {*mèo*, *chó*, *cá*} và $d_e = 2$, với $\mathbf{C} = \begin{bmatrix} 0.1 & 0.9 \\ 0.2 & 0.8 \\ 0.7 & -0.3 \end{bmatrix}$. Từ *chó* có one-hot $\mathbf{1}_{\text{chó}} = [0 \;\; 1 \;\; 0]$. Khi đó $\mathbf{1}_{\text{chó}}\mathbf{C} = 0 \cdot [0.1 \;\; 0.9] + 1 \cdot [0.2 \;\; 0.8] + 0 \cdot [0.7 \;\; -0.3] = [0.2 \;\; 0.8]$, đúng bằng hàng thứ 2 của $\mathbf{C}$. Vậy Phương trình (3.40) thực chất chỉ là "lấy ra hàng ứng với từ đó"; các con số trong $\mathbf{C}$ ban đầu được khởi tạo ngẫu nhiên rồi được điều chỉnh dần trong quá trình huấn luyện.

Sau đó, ta dùng một mạng nơ-ron truyền thẳng để tính phân phối xác suất của từ tại vị trí i. Phân phối này được cho bởi

$$\Pr(\cdot|w_{i-n+1}, \cdots, w_{i-1}) \;=\; F_\theta(\mathbf{e}_{i-n+1}, \cdots, \mathbf{e}_{i-1}), \tag{3.41}$$

trong đó $F_\theta(\cdot)$ là một mạng nơ-ron truyền thẳng có tham số θ. Thông thường, tầng nhúng có thể được xem là một thành phần của NLM. Ở đây, ta dùng ký hiệu hơi khác một chút để nhấn mạnh rằng NLM là một hàm của cả θ và $\mathbf{C}$

$$\Pr_{\theta,\mathbf{C}}(\cdot|w_{i-n+1},\cdots,w_{i-1}) \;=\; F_{\theta,\mathbf{C}}(w_{i-n+1},\cdots,w_{i-1}). \qquad (3.42)$$

Khi huấn luyện, ta tối ưu cả θ lẫn $\mathbf{C}$ để cực tiểu hóa một hàm mất mát. Một phương pháp phổ biến là huấn luyện theo hợp lý cực đại, tức là cực đại hóa tổng log-hợp lý trên tất cả các n-gram trong dữ liệu. Cho một chuỗi từ $w_1 \cdots w_m$, mục tiêu huấn luyện được định nghĩa là:[17]

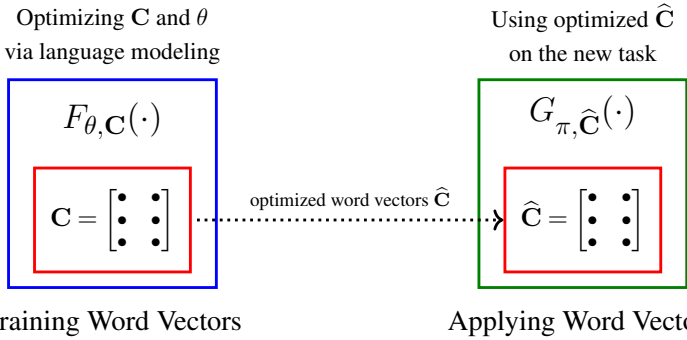
$$(\hat{\theta}, \widehat{\mathbf{C}}) \;=\; \arg\max_{\theta,\mathbf{C}} \sum_{i=n}^{m} \log \Pr_{\theta,\mathbf{C}}(w_i|w_{i-n+1},\cdots,w_{i-1}). \qquad (3.43)$$

Sau khi có các tham số tối ưu $\hat{\theta}$ và $\widehat{\mathbf{C}}$, ta có thể áp dụng $F_{\hat{\theta},\widehat{\mathbf{C}}}(\cdot)$ cho các n-gram chưa từng thấy. Quan trọng hơn, ta có được một số vectơ từ đã được huấn luyện tốt (tức là $\widehat{\mathbf{C}}$) có thể dùng trong các hệ thống khác ngoài NLM. Đây còn được gọi là tiền huấn luyện (*pre-training*) vectơ từ. Trong tiền huấn luyện, ta có thể định nghĩa $F_{\theta,\mathbf{C}}(\cdot)$ là bất kỳ hệ thống nào sử dụng các vectơ từ $\mathbf{C}$. Như vậy, bài toán học $\mathbf{C}$ được chuyển thành bài toán tối ưu $F_{\theta,\mathbf{C}}(\cdot)$ trên bài toán nền (xem minh họa ở Hình 3.6). Ưu điểm chính của phương pháp này là ta có thể tận dụng các bài toán NLP sẵn có để huấn luyện vectơ từ. Một rủi ro ở đây là các vectơ từ "tốt nhất" tìm được khi huấn luyện $F_{\theta,\mathbf{C}}(\cdot)$ có thể không thực sự phù hợp với hệ thống mà các vectơ từ được dùng trong thực tế. Thú vị là, trong nhiều trường hợp, các vectơ từ được tiền huấn luyện bằng NLM có chất lượng tốt cho các bài toán hạ nguồn (*downstream tasks*), hoặc ít nhất cũng là một điểm khởi đầu tốt để tiếp tục tinh chỉnh các vectơ từ này trong hệ thống đích.

Chú giải của người dịch: Ý tưởng "học ké"

Không ai cho máy biết vectơ "đúng" của từ *mèo* là gì. Nhưng ta có một bài toán có đáp án rõ ràng và dữ liệu gần như vô tận: đoán từ tiếp theo trong câu (mọi văn bản trên mạng đều là dữ liệu huấn luyện). Để đoán giỏi, mô hình buộc phải xếp các từ dùng giống nhau (*mèo*, *chó*) vào những vectơ gần nhau. Sau khi huấn luyện xong, ta "tháo" bảng $\widehat{\mathbf{C}}$ ra dùng cho bài toán khác, như

[17] Điều này có thể được tổng quát hóa cho một tập dữ liệu gồm nhiều chuỗi.



Hình 3.6: Minh họa việc tiền huấn luyện vectơ từ trong một NLM. NLM có thể được ký hiệu là một hàm $F_{\theta,\mathbf{C}}(\cdot)$ của bảng nhúng từ (tức là $\mathbf{C}$) và các tham số khác của NLM (tức là θ). Việc tiền huấn luyện $\mathbf{C}$ về bản chất là quá trình huấn luyện $F_{\theta,\mathbf{C}}(\cdot)$ trên một bài toán nền. Kết quả là các vectơ từ tối ưu $\widehat{\mathbf{C}}$, sau đó được áp dụng vào một hệ thống mới $G_{\pi,\widehat{\mathbf{C}}}(\cdot)$ có thể khác với NLM. Trong hệ thống mới, $\widehat{\mathbf{C}}$ là bảng nhúng từ học được từ NLM và π biểu diễn các tham số dành riêng cho $G(\cdot)$. (Các nhãn trên hình: *Training Word Vectors* = Huấn luyện vectơ từ, tối ưu $\mathbf{C}$ và θ bằng mô hình hóa ngôn ngữ; *Applying Word Vectors* = Áp dụng vectơ từ, dùng $\widehat{\mathbf{C}}$ đã tối ưu cho bài toán mới; *optimized word vectors* = các vectơ từ đã tối ưu.)

học sinh mang vốn từ vựng từ việc đọc truyện sang làm bài văn. Đây là mầm mống của ý tưởng tiền huấn luyện, nền tảng của các mô hình ngôn ngữ lớn ở các chương sau.

3.5 Các mô hình nhúng từ

Về nguyên tắc, vectơ từ có thể được học theo bất kỳ cách nào. Coi vectơ từ là thành phần của các hệ thống NLP sẵn có là một lựa chọn, nhưng cách tiếp cận này thường thiếu những cân nhắc dành riêng cho bài toán. Một lựa chọn khác là phát triển các phương pháp được thiết kế riêng cho bài toán học biểu diễn. Vì việc huấn luyện các hệ thống như vậy không phải thỏa mãn những ràng buộc chặt chẽ của các bài toán NLP hạ nguồn tiêu chuẩn, việc quy nạp các vectơ từ chất lượng cao trở nên dễ xử lý hơn đáng kể.

3.5.1 Word2Vec

Word2Vec là tên gọi của một bộ các mô hình được đề xuất trong [Mikolov và cộng sự, 2013a;c]. Giống như các mô hình ngôn ngữ nơ-ron, các mô hình Word2Vec được xây dựng dựa trên mạng nơ-ron. Tuy nhiên, thay vì dùng cách mô hình hóa sinh tự hồi quy nặng nề trên các n-gram, Word2Vec đặt việc học vectơ từ theo dạng log-tuyến tính (*log-linear*) hiệu quả về mặt tính toán. Do đó, kiến trúc của các mô hình này khác biệt rõ rệt so với kiến trúc dùng trong mô hình hóa ngôn ngữ truyền thống. Họ Word2Vec có hai kiến trúc chính:

- **Mô hình túi từ liên tục** (*continuous bag-of-words model*) (hay **mô hình CBOW**). Mô hình CBOW là một mô hình dự đoán từ. Nó được dùng để dự đoán khả năng một từ xuất hiện tại vị trí i khi biết các cửa sổ từ $-n$ và $+n$ xung quanh nó. Cấu trúc của mô hình CBOW tương tự mô hình ngôn ngữ nơ-ron đã giới thiệu ở Chương 2 (xem Hình 3.7(a)). Đầu tiên, ta dùng một tầng nhúng để biến các từ ngữ cảnh $w_{i-n} \cdots w_{i-1}$ và $w_{i+1} \cdots w_{i+n}$ thành các vectơ từ tương ứng. Việc này được thực hiện bằng cách nhân biểu diễn one-hot của mỗi từ đầu vào w_j với bảng nhúng $\mathbf{C} \in \mathbb{R}^{|V| \times d_e}$, như trong Phương trình (3.40). Các vectơ từ này sau đó được lấy trung bình để tạo ra

một biểu diễn duy nhất cho các từ đầu vào, cho ta

$$\mathbf{h} \;=\; \frac{1}{2n}\left(\sum_{j=i-n}^{i-1}\mathbf{1}_{w_j}\mathbf{C} + \sum_{j=i+1}^{i+n}\mathbf{1}_{w_j}\mathbf{C}\right). \tag{3.44}$$

Lưu ý rằng công thức trên định nghĩa một mô hình hoàn toàn bỏ qua thứ tự của các từ đầu vào do dùng phép tổng. Điều này giải thích vì sao mô hình CBOW được gọi là *túi từ*. Tầng đầu ra của mô hình CBOW là một tầng Softmax tiêu chuẩn chiếu **h** thành một phân phối xác suất trên bộ từ vựng

$$\mathbf{y} \;=\; \text{Softmax}(\mathbf{hU} + \mathbf{b}), \tag{3.45}$$

trong đó $\mathbf{U} \in \mathbb{R}^{d_e \times |V|}$ là ma trận tham số của ánh xạ tuyến tính và $\mathbf{b} \in \mathbb{R}^{|V|}$ là số hạng độ lệch. **y** là một phân phối trên bộ từ vựng, và $\Pr(w_i|w_{i-n}, \cdots, w_{i-1}, w_{i+1}, \cdots, w_{i+n}) \;=\; y(w_i)$. Phương trình (3.44–3.45) mô tả một mạng nơ-ron rất đơn giản. Một ưu điểm là mô hình thu được nhỏ gọn và hiệu quả so với các NLM. Việc huấn luyện mô hình CBOW rất đơn giản. Ta có thể đặt nó thành bài toán tìm ước lượng hợp lý cực đại cho các tham số của mô hình. Để đơn giản, gọi θ là các tham số khác ngoài **C** (tức là $\theta = \{\mathbf{U}, \mathbf{b}\}$). Ta có

$$(\hat{\theta}, \widehat{\mathbf{C}}) \;=\; \arg\max_{\theta,\mathbf{C}} \sum_{i=n+1}^{m-n} \log \Pr_{\theta,\mathbf{C}}(w_i|w_{i-n}, \cdots, w_{i-1}, w_{i+1}, \cdots, w_{i+n}), \tag{3.46}$$

trong đó m là độ dài của chuỗi từ. Sau khi huấn luyện, ta chỉ cần bỏ $\hat{\theta}$ đi và dùng $\widehat{\mathbf{C}}$ làm bảng tra cứu vectơ từ.

- **Mô hình skip-gram liên tục** (*continuous skip-gram model*) (hay **mô hình skip-gram**). Mô hình skip-gram là một mô hình dự đoán từ khác. Nó mô hình hóa bài toán ngược với bài toán mô tả trong Phương trình (3.44–3.45). Chính xác hơn, mục tiêu của ta là dự đoán từng từ trong $\pm n$ từ ngữ cảnh khi biết w_i. Điều này thường được đặt thành bài toán ước lượng xác suất w_j xuất hiện khi biết w_i ($i - n \leq j \leq i - 1$ hoặc $i + 1 \leq j \leq i + n$). Hình 3.7(b) cho thấy cấu trúc của mô hình skip-gram. Tầng nhúng xử lý w_i như thường lệ. Biểu diễn của w_i được cho bởi

$$\mathbf{h} \;=\; \mathbf{1}_{w_i}\mathbf{C}. \tag{3.47}$$

Sau đó nó được đưa vào một tầng Softmax để dự đoán xác suất của mỗi từ ngữ cảnh w_j (giả sử $j = i + k$)[18]

$$\mathbf{y}_k \;=\; \text{Softmax}(\mathbf{h}\mathbf{V}_k + \mathbf{b}_k), \tag{3.48}$$

trong đó $\mathbf{V}_k$ và $\mathbf{b}_k$ là các tham số của mô hình ($-n \le k \le -1$ và $1 \le k \le n$). Ta có

$$\begin{aligned}\Pr(w_j|w_i) &= \Pr(w_{i+k}|w_i) \\ &= y_k(w_{i+k}).\end{aligned} \tag{3.49}$$

Gọi θ là cách viết gọn của $\{\mathbf{V}_k\}$ và $\{\mathbf{b}_k\}$. Bài toán huấn luyện có thể được định nghĩa là:

$$(\hat{\theta}, \widehat{\mathbf{C}}) \;=\; \arg\max_{\theta,\mathbf{C}} \sum_{i=n+1}^{m-n} \sum_{\substack{-n\le k\le -1,\\ 1\le k\le n}} \log \Pr_{\theta,\mathbf{C}}(w_{i+k}|w_i). \tag{3.50}$$

Chú giải của người dịch: CBOW và skip-gram: hai trò chơi điền từ

Xét câu "*con mèo đang ngủ trên ghế*" với $n = 2$ và từ trung tâm $w_i = $ *ngủ*. **CBOW** chơi trò "điền vào chỗ trống": nhìn *mèo, đang, trên, ghế* rồi đoán từ ở giữa là *ngủ*; vectơ ngữ cảnh $\mathbf{h}$ là *trung bình* của 4 vectơ từ (cộng lại rồi chia cho $2n = 4$). **Skip-gram** chơi trò ngược lại: biết *ngủ*, đoán từng từ xung quanh *mèo*, *đang*, *trên*, *ghế*, nên mỗi vị trí $k \in \{-2, -1, 1, 2\}$ có một bộ tham số $\mathbf{V}_k, \mathbf{b}_k$ riêng. Hàm Softmax (xem Chương 2) biến một dãy điểm số bất kỳ thành các xác suất dương có tổng bằng 1. Tổng $\sum \log \Pr$ trong các Phương trình (3.46) và (3.50) là log của tích các xác suất, nên cực đại hóa nó nghĩa là làm cho mô hình "đoán trúng" dữ liệu thật nhiều nhất có thể.

Cả hai mô hình trên đều tương tự như bài kiểm tra điền khuyết (*cloze test*), khi chỉ xét quan hệ phụ thuộc theo từng cặp giữa các từ. Vì chúng bỏ qua thứ tự từ và các quan hệ cú pháp phức tạp, các phân phối dự đoán của chúng kém chính xác hơn so với các mô hình ngôn ngữ. Tuy nhiên, cần lưu ý rằng mục tiêu của các mô hình này không phải là dự đoán chính xác các từ bị thiếu khi biết ngữ cảnh, mà là học biểu diễn từ từ một bài

[18] Khi $k > 0$, w_j là một từ trong cửa sổ ngữ cảnh bên phải của w_i; khi $k < 0$, w_j là một từ trong cửa sổ ngữ cảnh bên trái.

toán nào đó nắm bắt được quan hệ giữa các từ. Vì vậy, hiệu năng dự đoán từ của mô hình học được không quá quan trọng.

Một ưu điểm khác của các mô hình này là chúng có kiến trúc rất đơn giản, dễ huấn luyện. Ví dụ, trong cả hai mô hình đều không có tầng ẩn, và tầng nhúng được nối trực tiếp với tầng đầu ra. Các cấu trúc mô hình này có thể được xem là những trường hợp của mô hình hóa log-tuyến tính trong học máy: các biến đầu vào được biến đổi tuyến tính thành một vectơ đặc trưng dày đặc (ví dụ Phương trình (3.44)), theo sau ngay là một hàm phân loại log-tuyến tính (ví dụ Phương trình (3.45)).

3.5.2 GloVe

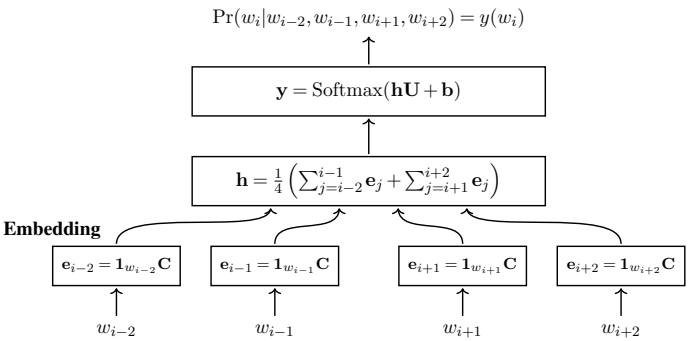
Vectơ toàn cục (*Global vectors*), thường được gọi là **GloVe**, là các vectơ nhúng từ được học bằng cách kết hợp ưu điểm của phương pháp phân tích ma trận toàn cục và phương pháp cửa sổ ngữ cảnh cục bộ [Pennington và cộng sự, 2014]. Phương pháp GloVe bắt đầu bằng việc xây dựng một ma trận đồng xuất hiện từ–từ toàn cục (như đã bàn ở Mục 3.3), rồi sau đó xây dựng một mô hình log-song tuyến tính dựa trên một tập giả định lý thuyết cụ thể về nghĩa của từ.

Cho một ma trận đồng xuất hiện từ–từ $\mathbf{M}$, trong đó mỗi ô $M(a, b) = \text{count}(a, b)$ biểu diễn số lần từ $a \in V$ đồng xuất hiện với từ $b \in V$ trong một cửa sổ ngữ cảnh xác định, ta có thể tính xác suất có điều kiện $\Pr(b|a)$ như sau:

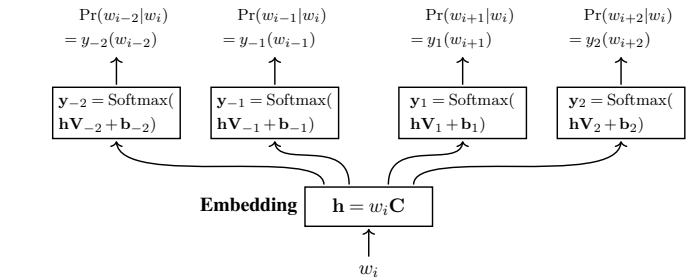
$$\begin{aligned}\Pr(b|a) &= \frac{\text{count}(a, b)}{\sum_{b'} \text{count}(a, b')} \\ &= \frac{\text{count}(a, b)}{\text{count}(a)},\end{aligned} \tag{3.51}$$

trong đó $\text{count}(a)$ biểu diễn tổng số lần từ a xuất hiện trong kho ngữ liệu.

Để hiểu động cơ đằng sau GloVe, hãy xét một ví dụ trong đó ta muốn phân biệt hai khái niệm nhiệt động học *air* (không khí) và *water* (nước). Dùng Phương trình (3.51), ta có thể tính xác suất các từ đích này xuất hiện khi biết các từ ngữ cảnh "thăm dò" khác nhau. Bảng sau hiển thị một phần nhỏ của ma trận $\Pr(b|a)$ được tính từ một kho ngữ liệu 3.8 tỷ từ trong WMT14 (*fly* = bay, *drink* = uống, *breath* = hơi thở, *live* = sống, *flow* = chảy):



(a) CBOW



(b) Skip-gram

Hình 3.7: Kiến trúc CBOW và skip-gram. Mô hình CBOW tính xác suất $\Pr(w_i|w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2})$, trong đó w_i là một từ trong chuỗi và $\{w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}\}$ là các từ trong các cửa sổ ngữ cảnh ± 2. Biểu diễn ngữ cảnh **h** là trung bình của các vectơ từ được tạo ra qua một tầng nhúng. Sau đó **h** được đưa vào một tầng Softmax để cho ra một phân phối trên bộ từ vựng (tức là **y**). Xác suất dự đoán của w_i là $\Pr(w_i|w_{i-2}, w_{i-1}, w_{i+1}, w_{i+2}) = y(w_i)$. Mô hình skip-gram cũng dựa trên cấu trúc nhúng + softmax. Nó mô hình hóa xác suất của mỗi từ ngữ cảnh w_j khi biết từ w_i. Điều này đạt được bằng cách đơn giản là tính đầu ra của một tầng softmax tiêu chuẩn nhận biểu diễn vectơ của w_i làm đầu vào. Cả hai mô hình CBOW và skip-gram đều được huấn luyện theo cách hợp lý cực đại. Bảng tra cứu thu được của tầng nhúng chính là các vectơ từ (hay vectơ nhúng) của các từ trong bộ từ vựng. (Trên hình: *Embedding* = tầng nhúng.)

Phần tử	$w = \textit{fly}$	$w = \textit{drink}$	$w = \textit{breath}$	$w = \textit{live}$	$w = \textit{flow}$
$\Pr(\textit{air}\|w)$	1.5×10^{-4}	6.2×10^{-5}	2.2×10^{-4}	1.6×10^{-4}	3.6×10^{-4}
$\Pr(\textit{water}\|w)$	1.3×10^{-5}	4.1×10^{-4}	1.8×10^{-5}	1.4×10^{-4}	3.0×10^{-4}
$\Pr(\textit{air}\|w)/\Pr(\textit{water}\|w)$	11.54	0.15	12.2	1.14	1.2

Trong bảng này, $\Pr(\textit{air}|w)$ và $\Pr(\textit{water}|w)$ cho biết *air* và *water* tương quan mạnh đến mức nào với các từ thăm dò w khác nhau. Nhận xét then chốt xuất hiện khi ta xem xét tỉ số xác suất $\Pr(\textit{air}|w)/\Pr(\textit{water}|w)$ ở hàng cuối. Tỉ số này tách riêng một cách hiệu quả sự khác biệt ngữ nghĩa giữa hai từ đích. Khi w rất liên quan đến *air* nhưng không liên quan đến *water* (ví dụ $w = \textit{fly}$ hay $w = \textit{breath}$), tỉ số này lớn. Ngược lại, khi w liên quan đến *water* nhưng không liên quan đến *air* (ví dụ $w = \textit{drink}$), tỉ số này nhỏ. Khi w liên quan đến cả hai (ví dụ $w = \textit{live}$ hay $w = \textit{flow}$), hoặc không liên quan đến cả hai, tỉ số gần bằng 1.

Tiền đề nền tảng của GloVe là các vectơ từ nên mã hóa một cách tường minh tỉ số xác suất đồng xuất hiện này. Một điểm xuất phát tự nhiên là giả định có một hàm F nhận các vectơ từ làm đầu vào và xấp xỉ tỉ số này:

$$F(\mathbf{e}_a, \mathbf{e}_b, \tilde{\mathbf{e}}_w) \;=\; \frac{\Pr(a|w)}{\Pr(b|w)}, \tag{3.52}$$

trong đó $\mathbf{e}_a, \mathbf{e}_b \in \mathbb{R}^{d_e}$ là các biểu diễn vectơ đích của các từ a và b, còn $\tilde{\mathbf{e}}_w \in \mathbb{R}^{d_e}$ là biểu diễn vectơ ngữ cảnh của từ thăm dò w. Hãy chú ý sự phân biệt giữa $\mathbf{e}$ và $\tilde{\mathbf{e}}$: cái trước được lấy từ một bảng nhúng đích $\mathbf{C}$, còn cái sau được lấy từ một bảng nhúng ngữ cảnh riêng $\widetilde{\mathbf{C}}$. Việc dùng hai bảng nhúng khác nhau giúp giảm quá khớp trong quá trình huấn luyện. Cuối cùng, biểu diễn từ sau cùng thường được lấy là tổng (hoặc trung bình) của hai vectơ: $\mathbf{e}_{a,\text{final}} = \mathbf{1}_a(\mathbf{C} + \widetilde{\mathbf{C}})$.

Vì các không gian vectơ vốn có tính tuyến tính, cách tự nhiên nhất để mã hóa quan hệ giữa hai từ đích a và b là dùng phép trừ vectơ: $\mathbf{e}_a - \mathbf{e}_b$. Hơn nữa, để định lượng quan hệ giữa hiệu này và từ ngữ cảnh w, ta có thể dùng tích vô hướng của vectơ. Điều này giới hạn hàm F chỉ nhận một đầu vào vô hướng duy nhất:

$$F\Big((\mathbf{e}_a - \mathbf{e}_b)\tilde{\mathbf{e}}_w^\top\Big) \;=\; \frac{\Pr(a|w)}{\Pr(b|w)}. \tag{3.53}$$

Để áp đặt một cấu trúc đại số chặt chẽ, ta yêu cầu F là một đồng cấu (*homomorphism*) giữa nhóm cộng trong không gian vectơ và nhóm nhân

trong không gian xác suất. Điều này bảo đảm phép trừ vectơ được ánh xạ trực tiếp thành phép chia xác suất:

$$F\Big((\mathbf{e}_a - \mathbf{e}_b)\tilde{\mathbf{e}}_w^\top\Big) \;=\; \frac{F(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top)}{F(\mathbf{e}_b\tilde{\mathbf{e}}_w^\top)}. \tag{3.54}$$

Nghiệm liên tục duy nhất của Phương trình (3.54) là hàm mũ, $F(x) = \exp(x)$. Do đó, ta có:

$$\begin{aligned} F(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) &= \exp(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) \\ &= \Pr(a|w) \\ &= \frac{\text{count}(a,w)}{\text{count}(a)}. \end{aligned} \tag{3.55}$$

Lấy logarit hai vế, ta được:

$$\mathbf{e}_a\tilde{\mathbf{e}}_w^\top \;=\; \log\text{count}(a,w) - \log\text{count}(a). \tag{3.56}$$

Chú giải của người dịch: Vì sao lại là hàm mũ?

Phương trình (3.54) đòi hỏi $F(x-y) = F(x)/F(y)$: hiệu ở đầu vào phải thành thương ở đầu ra. Ở THPT bạn đã biết hàm mũ có đúng tính chất này: $e^{x-y} = e^x/e^y$. Ví dụ $e^{5-2} = e^3$ và $e^5/e^2 = e^3$. Lấy logarit là phép ngược của hàm mũ, nên từ $\exp(\mathbf{e}_a\tilde{\mathbf{e}}_w^\top) = \frac{\text{count}(a,w)}{\text{count}(a)}$ ta suy ra Phương trình (3.56) nhờ quy tắc $\log\frac{x}{y} = \log x - \log y$. Nói nôm na: GloVe muốn *tích vô hướng* của hai vectơ từ bằng *log của số lần chúng đi cùng nhau* (sau khi trừ đi các phần "độ phổ biến" riêng của từng từ).

Một vấn đề của Phương trình (3.56) là số hạng $\log\text{count}(a)$ phá vỡ tính đối xứng; việc hoán đổi từ đích a và từ ngữ cảnh w (tương ứng với việc chuyển vị ma trận đồng xuất hiện) lẽ ra không được làm thay đổi cấu trúc hình học bên dưới. Để khôi phục tính đối xứng này, số hạng $\log\text{count}(a)$ được hấp thụ vào hai số hạng độ lệch được học, β_a và $\tilde{\beta}_w$:

$$\mathbf{e}_a\tilde{\mathbf{e}}_w^\top + \beta_a + \tilde{\beta}_w \;=\; \log\text{count}(a,w). \tag{3.57}$$

Phương trình (3.57) thiết lập quan hệ lý tưởng mà ta muốn các vectơ nhúng học được thỏa mãn. Để áp đặt điều này, ta xây dựng một bài toán hồi quy, nhằm cực tiểu hóa bình phương hiệu giữa log đồng xuất hiện dự

đoán và log đồng xuất hiện thực tế. Hàm mất mát bình phương cho một cặp từ (a, w) được định nghĩa là:

$$\mathcal{L}_{a,w} = \left(\mathbf{e}_a \tilde{\mathbf{e}}_w^\top + \beta_a + \tilde{\beta}_w - \log \text{count}(a, w)\right)^2. \tag{3.58}$$

Hàm chi phí tổng trên toàn bộ bộ từ vựng là tổng có trọng số của các hàm mất mát riêng lẻ này:

$$L_{\text{GloVe}} = \sum_{a,w \in V} f(\text{count}(a, w)) \cdot L_{a,w}, \tag{3.59}$$

trong đó $f(\text{count}(a, w))$ là một hàm trọng số được thiết kế để ngăn các cặp từ hiếm chi phối hàm mất mát, đồng thời giới hạn ảnh hưởng của các cặp từ quá phổ biến. Pennington và cộng sự [2014] đề xuất hàm trọng số sau:

$$f(x) = \begin{cases} \left(\dfrac{x}{x_{\max}}\right)^\alpha & \text{nếu } x < x_{\max} \\ 1 & \text{ngược lại} \end{cases}, \tag{3.60}$$

trong đó $x_{\max}$ và α là các siêu tham số (thường $x_{\max} = 100$ và $\alpha = 3/4$). Như vậy, $f(x)$ "phạt" các cặp từ hiếm bằng cách gán cho chúng trọng số nhỏ hơn, đồng thời bảo đảm trọng số của các cặp phổ biến không tăng vô hạn.

Chú giải của người dịch: Tính thử hàm trọng số

Với $x_{\max} = 100$, $\alpha = 3/4$: một cặp từ chỉ đi cùng nhau $x = 1$ lần có trọng số $f(1) = (1/100)^{0.75} \approx 0.03$; cặp xuất hiện $x = 16$ lần có $f(16) = (0.16)^{0.75} \approx 0.25$; cặp xuất hiện từ 100 lần trở lên có trọng số 1, dù là 100 hay 1 triệu lần. Nhờ vậy, các cặp ngẫu nhiên hiếm hoi (dễ là nhiễu) ít ảnh hưởng, còn các cặp cực phổ biến (như "của" đi với "là") cũng không lấn át tất cả. Lưu ý: Phương trình (3.59) chỉ tính trên các cặp có $\text{count}(a, w) > 0$ trong thực tế, vì $f(0) = 0$ và $\log 0$ không xác định.

Các Phương trình (3.58–3.59) mô tả một khuôn khổ rất hiệu quả để học vectơ từ bằng các phép toán mạng nơ-ron tiêu chuẩn (ví dụ tích vô hướng và phép cộng vô hướng). Một đặc điểm quyết định của GloVe là nó dựa vào một hàm mục tiêu log-song tuyến tính đơn giản. Vì việc huấn luyện tránh được hàm mất mát entropy chéo và phép chuẩn hóa softmax tốn kém về tính toán mà các NLM đầy đủ và Word2Vec đòi hỏi, GloVe

huấn luyện nhanh đáng kể. Trực giác cốt lõi là các quan hệ ngữ nghĩa có thể được mô hình hóa bằng phép hồi quy đơn giản trên các thống kê toàn cục của kho ngữ liệu, thay vì chỉ dựa vào độ phân kỳ dựa trên xác suất trên các cửa sổ ngữ cảnh cục bộ.

Một lưu ý nữa về việc sử dụng dữ liệu toàn cục đáng được nhắc lại. Ma trận đồng xuất hiện mã hóa thông tin được tổng hợp trên toàn bộ kho ngữ liệu. Bằng cách phân tích trực tiếp ma trận này, GloVe đặt việc học thành một bài toán tối ưu toàn cục một cách tường minh. Mặc dù Word2Vec cũng tìm một tập tham số tối ưu toàn cục (ví dụ cực đại hóa hợp lý trên toàn bộ không gian đầu vào), hàm mục tiêu của nó phức tạp và thường được tối ưu bằng học trực tuyến (*online learning*) (hạ gradient ngẫu nhiên trên các lô cục bộ). Còn GloVe, nhờ tổng hợp trước dữ liệu thành một ma trận đồng xuất hiện, đã chắt lọc quá trình học thành một mô hình toàn cục trực tiếp hơn và hiệu quả hơn về tính toán.

3.5.3 Nhận xét

Ở các mục trước, ta đã thấy vectơ từ có thể được quy nạp bằng nhiều phương pháp khác nhau. Giờ đây ta bàn về một số cân nhắc quan trọng khi huấn luyện hay áp dụng các vectơ nhúng từ này trong thực tế.

1. Dựa trên đếm và dựa trên mạng nơ-ron

Sự đơn giản và tính diễn giải được của các phương pháp dựa trên đếm từ lâu đã được cộng đồng NLP đánh giá cao. Cả các mô hình dựa trên đếm lẫn các mô hình dựa trên mạng nơ-ron về cơ bản đều dựa trên giả thuyết phân bố, tức ý tưởng rằng các từ xuất hiện trong ngữ cảnh tương tự nhau thì có nghĩa tương tự nhau. Tuy nhiên, các phương pháp dựa trên đếm xây dựng một cách tường minh những ma trận nhiều chiều và thưa dựa trên giả định này, điều thường dẫn đến vấn đề dữ liệu thưa do lời nguyền số chiều (*curse of dimensionality*).

Ở đầu kia là các mô hình dựa trên mạng nơ-ron. Thay vì dựa vào việc đếm đồng xuất hiện tường minh và các sơ đồ trọng số thủ công, các phương pháp này xem vectơ từ là những trạng thái trung gian ít chiều, dày đặc bên trong một mạng nơ-ron được tối ưu cho một bài toán nền cụ thể. Bằng cách loại bỏ các ràng buộc ngôn ngữ học tường minh về ý nghĩa cụ thể của từng chiều trong vectơ, các mô hình dựa trên mạng nơ-ron học hiệu

quả những đặc trưng ngữ nghĩa tiềm ẩn vốn khó thiết kế thủ công.

Sự đối lập giữa hai cách tiếp cận này giống với sự so sánh rộng hơn giữa hai mô thức học máy nổi tiếng: kỹ nghệ đặc trưng (*feature engineering*) và học biểu diễn đầu-cuối (*end-to-end representation learning*). Tuy nhiên, ở đây chúng tôi sẽ không đi sâu vào cuộc tranh luận mang tính triết lý này. Chỉ cần lưu ý rằng không có dạng nào tuyệt đối vượt trội so với dạng kia. Nói chung, việc chọn loại vectơ từ hoàn toàn phụ thuộc vào ứng dụng hạ nguồn đích và mức độ diễn giải được mong muốn. Ví dụ, nếu mục tiêu là thu được một dạng tính toán diễn giải được một cách tường minh, minh bạch về toán học trực tiếp từ một kho ngữ liệu, thì phân tích ma trận đồng xuất hiện vẫn là một lựa chọn vững chắc. Ngược lại, nếu mục tiêu là trích xuất các vectơ từ liên tục, dày đặc để tích hợp liền mạch vào một kiến trúc nơ-ron sâu, thì học biểu diễn bằng mạng nơ-ron thường được ưu tiên.

Cần lưu ý rằng việc học các vectơ nhúng từ liên tục, dày đặc đã trở thành mô thức chủ đạo trong những năm gần đây, cho thấy những tiến bộ vượt bậc của các mô hình nơ-ron cho NLP. Đồng thời, giới học thuật cũng rất quan tâm đến việc so sánh các phương pháp dựa trên đếm và dựa trên mạng nơ-ron, và làm sáng tỏ những quan hệ lý thuyết sâu sắc giữa chúng [Levy và Goldberg, 2014b; Baroni và cộng sự, 2014; Levy và Goldberg, 2014c; Schnabel và cộng sự, 2015a; Levy và cộng sự, 2015; Gladkova và cộng sự, 2016].

Chú giải của người dịch: "Thưa" và "dày đặc"

Một vectơ *thưa* là vectơ mà hầu hết phần tử bằng 0, như các hàng của ma trận đồng xuất hiện: với bộ từ vựng 20 nghìn từ, một từ thường chỉ đi cùng vài trăm từ khác, còn lại toàn số 0. Một vectơ *dày đặc* (như $[0.21 \quad -1.3 \quad 0.07 \quad \ldots]$ với vài trăm chiều) thì hầu hết phần tử khác 0 và mỗi chiều không có ý nghĩa riêng dễ gọi tên. "Lời nguyền số chiều" ám chỉ rằng khi số chiều quá lớn, dữ liệu trở nên quá thưa thớt so với không gian, khiến việc ước lượng và so sánh kém tin cậy.

2. Mô hình nông và mô hình sâu

Mặc dù mạng nơ-ron đã trở thành cách tiếp cận tiêu chuẩn để quy nạp vectơ từ, các cấu trúc mô hình giới thiệu từ đầu chương đến giờ đều đơn giản. Về mặt kỹ thuật, chúng chỉ gồm một hoặc hai tầng nơ-ron, và do đó được xếp vào loại mô hình nông (*shallow models*). Một ví dụ đáng chú ý là mô hình nhúng từ **vLBL** [Mnih và Kavukcuoglu, 2013], mô hình hóa tương tác giữa các từ bằng một mạng nơ-ron chỉ có đúng hai tầng. Kiến trúc này, thậm chí bỏ cả hàm chuẩn hóa softmax, theo hiểu biết của chúng tôi là một trong những mô hình nhúng từ đơn giản nhất. Dù đơn giản như vậy, các mô hình này vẫn rất hiệu quả trong nhiều tình huống. Lợi ích chính của mô hình nông là hiệu quả tính toán và khả năng mở rộng tới những tập dữ liệu khổng lồ. Sự hiệu quả này cho phép chúng được áp dụng cho các bài toán NLP phức tạp hơn. Một ví dụ nổi bật là hệ thống **fastText** cho phân loại văn bản [Joulin và cộng sự, 2017]. Nó có kiến trúc tương tự mô hình CBOW (xem Mục 3.5.1). Trong fastText, văn bản đầu vào được biểu diễn như một túi các vectơ từ, được lấy trung bình để tạo thành một biểu diễn ẩn toàn cục của văn bản. Theo sau ngay là một tầng đầu ra ánh xạ biểu diễn ẩn này thành một phân phối xác suất trên các lớp định trước. Bằng cơ chế này, mục tiêu phân loại và các vectơ từ bên dưới được huấn luyện đồng thời.

Mặc dù mô hình nông rất hiệu quả trong việc học các biểu diễn từ tĩnh, mạng nơ-ron sâu mang lại năng lực biểu diễn lớn hơn đáng kể. Như với hầu hết các kiến trúc nhiều tầng, học biểu diễn bằng mạng nơ-ron sâu có một số ưu điểm rõ rệt [Telgarsky, 2016]. Thứ nhất, mô hình sâu cho phép ta khám phá một không gian giả thuyết lớn hơn nhiều, có khả năng tìm ra những ánh xạ hàm tốt hơn. Thứ hai, việc đưa vào nhiều phép biến đổi phi tuyến làm tăng mạnh khả năng nắm bắt những khuôn mẫu rất phức tạp trong dữ liệu của mô hình. Có nhiều chiến lược tận dụng độ sâu trong học biểu diễn. Cách trực tiếp nhất là xếp chồng dần các tầng truyền thẳng, hồi quy hoặc tích chập lên trên một tầng nhúng cơ sở. Với việc các mô hình tiền huấn luyện quy mô lớn được dùng rộng rãi trong NLP, mô thức đã chuyển sang việc trích xuất các biểu diễn theo ngữ cảnh (*contextualized representations*) trực tiếp từ các kiến trúc rất sâu, đạt hiệu năng tiên tiến nhất trên rất nhiều bộ đánh giá chuẩn NLP [Radford và cộng sự, 2018; Devlin và cộng sự, 2019]. Tuy nhiên, năng lực mô hình hóa tăng lên này kéo theo những nút thắt tính toán nghiêm trọng và sự bất ổn định khi huấn

luyện. Do đó, một khối lượng lớn nghiên cứu tập trung vào việc giảm nhẹ những thách thức tối ưu hóa này [Pascanu và cộng sự, 2013; Bapna và cộng sự, 2018; Wang và cộng sự, 2019a; Zhang và cộng sự, 2019a; Pham và cộng sự, 2019; Li và cộng sự, 2020b]. Trong Chương 4–6, ta sẽ tìm hiểu một số mô hình NLP thành công được xây dựng trên các kiến trúc nơ-ron sâu này.

3. Mục tiêu huấn luyện

Ý tưởng coi vectơ từ là tham số mô hình rất phù hợp với mô hình hóa biến tiềm ẩn (*latent-variable modeling*): một mô hình được tham số hóa bằng các vectơ từ có thể học, và giá trị tối ưu của chúng được suy ra bằng cách tối ưu một hàm mục tiêu toàn cục. Mặc dù mô thức học này là tiêu chuẩn, mục tiêu huấn luyện cụ thể có thể khác nhau rất nhiều. Một khó khăn chính là không có một mục tiêu tường minh chỉ dành riêng cho việc giám sát biểu diễn vectơ từ. Một giải pháp trực tiếp là tận dụng các bài toán NLP được định nghĩa rõ ràng. Chẳng hạn, ta có thể dùng vectơ từ làm biểu diễn đầu vào cho các hệ thống mô hình hóa ngôn ngữ hay phân loại văn bản. Khi đó, các vectơ từ đóng vai trò như những tham số mô hình thông thường và được tối ưu theo kiểu đầu-cuối.

Một cách tiếp cận khác là thiết kế những mục tiêu huấn luyện mới. Như thường thấy trong học máy, không gian thiết kế các mục tiêu như vậy là rất rộng. Vì vậy, một chiến lược phổ biến là điều chỉnh các bài toán sẵn có. Ví dụ, mục tiêu huấn luyện của mô hình CBOW về bản chất là một bài toán dự đoán từ tổng quát hóa và có dạng toán học tương tự mô hình hóa ngôn ngữ truyền thống.

Ta sẽ tìm hiểu một số mục tiêu mới bắt nguồn từ mô hình hóa ngôn ngữ trong Chương 7. Nói chặt chẽ, các bài toán huấn luyện này vốn không nhằm học vectơ từ. Đúng hơn, chúng cung cấp một mục tiêu thay thế được định nghĩa rõ ràng cho việc học biểu diễn. Hơn nữa, ta thường không thể giả định một kịch bản học có giám sát thuần túy: các vectơ nhúng học được cuối cùng sẽ được triển khai trong nhiều hệ thống hạ nguồn khác nhau mà ta chưa biết trong giai đoạn huấn luyện ban đầu. Điều này khiến quá trình quy nạp trở thành một dạng học tự giám sát, thiếu sự giám sát trực tiếp từ ứng dụng đích cuối cùng. Tuy nhiên, khi tiếp cận được ứng dụng đích và có sẵn dữ liệu có nhãn của bài toán, ta có thể tinh chỉnh thêm các vectơ nhúng từ đã tiền huấn luyện này để nắm bắt ngữ nghĩa đặc

thù của lĩnh vực.

3.6 Đánh giá vectơ nhúng từ

Sau khi có được biểu diễn vectơ của các từ, ta cần đánh giá chất lượng của chúng. Lý tưởng nhất, ta sẽ đánh giá các vectơ từ này so với một đáp án chuẩn. Đáng tiếc là đáp án chuẩn như vậy hiếm khi tồn tại, vì người gán nhãn không thể trực tiếp gán các giá trị số liên tục, nhiều chiều để nắm bắt ngữ nghĩa của một từ. Một giải pháp thực tế là đánh giá các vectơ dựa trên hiệu năng của một hệ thống hạ nguồn đang hoạt động có sử dụng chúng. Nhìn chung, các cách đánh giá được chia thành hai loại [Schnabel và cộng sự, 2015b]:

- **Đánh giá ngoại tại** (*extrinsic evaluation*) (hay kiểm thử đầu-cuối). Ta đưa các vectơ từ vào một hệ thống NLP hạ nguồn và đo tác động của chúng lên hiệu năng tổng thể của hệ thống.
- **Đánh giá nội tại** (*intrinsic evaluation*). Ta kiểm tra khả năng của các vectơ từ trong việc nắm bắt những tính chất hình thái, cú pháp và ngữ nghĩa cụ thể của ngôn ngữ.

Dưới đây, chúng tôi mô tả ngắn gọn cách áp dụng các cách tiếp cận này vào việc đánh giá vectơ từ.

3.6.1 Đánh giá ngoại tại

Cách tiếp cận này được dùng rộng rãi trong thực tế, vì nó cho phép các nhà nghiên cứu và kỹ sư nhanh chóng kiểm tra một hệ thống thực tế hoạt động ra sao khi các biểu diễn bên dưới của nó bị thay đổi. Vì nhiều hệ thống NLP nhận đầu vào là các từ, việc thay biểu diễn từ dạng ký hiệu bằng các vectơ từ dày đặc là cách làm tiêu chuẩn. Ta đã thấy các hệ thống loại này, vốn thường dùng một tầng nhúng để biến biểu diễn one-hot của mỗi từ đầu vào thành một vectơ giá trị thực (ví dụ mô hình ngôn ngữ nơ-ron ở Chương 2).

Cho một hệ thống như vậy và một tập vectơ từ đã tiền huấn luyện, ta có thể dùng hiệu năng hạ nguồn của hệ thống làm thước đo gián tiếp cho chất lượng của các vectơ nhúng. Tùy theo cách sử dụng các vectơ từ, có

hai mô thức huấn luyện chính:

- **Vectơ từ là tham số cố định**. Ta "đóng băng" các vectơ từ và chỉ huấn luyện các tham số còn lại của hệ thống như thường lệ.
- **Vectơ từ là tham số khởi tạo** (tinh chỉnh). Ta khởi tạo tầng nhúng bằng các vectơ từ đã tiền huấn luyện và cập nhật chúng cùng với các tham số còn lại của mô hình trong quá trình huấn luyện.

Cả hai phương pháp đều tận dụng khái niệm tiền huấn luyện, và có thể được mở rộng cho nhiều tình huống mà một phần của mô hình được huấn luyện kỹ trước khi áp dụng vào một bài toán hạ nguồn. Đóng băng vectơ từ giúp đơn giản hóa quá trình huấn luyện và tách riêng việc đánh giá bản thân các biểu diễn. Ngược lại, coi vectơ từ là tham số có thể học làm tăng độ phức tạp của việc huấn luyện nhưng cho phép mô hình tinh chỉnh các biểu diễn, điều chỉnh chúng riêng cho hệ thống đích.

Lưu ý rằng mặc dù đánh giá ngoại tại rất thực tế, kết quả của nó phụ thuộc nhiều vào kiến trúc và bài toán hạ nguồn cụ thể. Vì việc phát triển các hệ thống NLP tiên tiến nhất bao gồm các kiến trúc phức tạp và việc tinh chỉnh siêu tham số rất công phu, sự thay đổi hiệu năng không phải lúc nào cũng quy được cho riêng chất lượng của vectơ nhúng từ. Sự phụ thuộc vào bài toán này là phổ biến trong NLP. Chẳng hạn, một phương pháp tách token rất tốt trong hệ thống dịch máy có thể hoạt động kém trong một hệ thống truy xuất thông tin. Vì vậy, để kiểm tra chặt chẽ khả năng tổng quát hóa của một tập vectơ từ, cách làm tiêu chuẩn là đánh giá chúng trên một loạt các bài toán NLP hạ nguồn đa dạng.

3.6.2 Đánh giá nội tại

Mặc dù phần lớn nghiên cứu về biểu diễn từ liên quan tới kiểm thử đầu-cuối trong các ứng dụng NLP, nó cũng bao gồm việc kiểm tra khả năng của biểu diễn trong việc giải quyết một số vấn đề nhất định, chẳng hạn diễn giải quan hệ giữa hai từ. Có nhiều cách thiết kế đánh giá nội tại, mỗi cách giải quyết một vấn đề cụ thể. Dưới đây chúng tôi mô tả một số phương pháp trong đó. Để có cái nhìn tổng quan đầy đủ hơn về các phương pháp đánh giá nội tại, mời bạn đọc tham khảo các tài liệu phong phú về chủ đề này [Baroni và cộng sự, 2014; Bakarov, 2018; Rogers và cộng sự, 2018].

1. Độ liên quan ngữ nghĩa

Mô hình hóa độ liên quan giữa các từ có lẽ là phương pháp phổ biến nhất để đánh giá chất lượng vectơ từ trong NLP [Reisinger và Mooney, 2010; Huang và cộng sự, 2012; Baroni và cộng sự, 2014]. Về cơ bản, việc này gồm tính một khoảng cách không gian giữa các từ (gọi là **khoảng cách ngữ nghĩa của từ** (*word semantic distance*) hay đơn giản là **khoảng cách từ** (*word distance*)). Động cơ ở đây là khoảng cách hình học trong không gian vectơ liên tục nên tương quan với phán đoán nhận thức của con người về độ liên quan ngữ nghĩa [Rubenstein và Goodenough, 1965]. Ví dụ, ta mong đợi vectơ của *dog* (chó) nằm gần *wolf* (sói) về mặt hình học, còn *peach* (quả đào) thì nên ở xa *television* (ti vi). Về mặt toán học, có nhiều độ đo có thể dùng để tính khoảng cách hay góc giữa hai vectơ. Một độ đo đơn giản và thường dùng là khoảng cách Euclid. Ngoài ra, tính độ tương tự cosin (*cosine similarity*) giữa hai vectơ sẽ cho một điểm số trong khoảng $[-1, 1]$[19].

Chú giải của người dịch: Độ tương tự cosin

Với hai vectơ $\mathbf{u}$, $\mathbf{v}$, độ tương tự cosin là $\cos(\theta) = \dfrac{\mathbf{u} \cdot \mathbf{v}}{|\mathbf{u}|\,|\mathbf{v}|}$, đúng công thức tính góc giữa hai vectơ ở lớp 10, chỉ khác là có nhiều tọa độ hơn. Ví dụ $\mathbf{u} = (1; 2)$, $\mathbf{v} = (2; 4)$ thì $\cos\theta = \frac{10}{\sqrt{5}\cdot\sqrt{20}} = 1$ (cùng hướng, rất giống nhau); $\mathbf{u} = (1; 0)$, $\mathbf{v} = (0; 1)$ thì $\cos\theta = 0$ (vuông góc, không liên quan). Điểm cosin không phụ thuộc độ dài vectơ, nên một từ phổ biến (vectơ đếm lớn) và một từ hiếm vẫn có thể được đánh giá là "giống nhau" nếu chúng cùng hướng.

Trong thiết lập đánh giá này, ta được cung cấp một tập dữ liệu gồm các cặp từ, mỗi cặp được người gán nhãn cho một điểm số độ liên quan kỳ vọng. Với một cặp từ, ta tính tương quan giữa điểm kỳ vọng của con người và khoảng cách hình học đo được trong không gian vectơ từ. Chất lượng của các vectơ từ được phản ánh qua độ mạnh của mối tương quan này. Tuy nhiên, một thách thức chính là không có một thước đo chuẩn tuyệt đối cho khoảng cách ngữ nghĩa. Ngay cả người gán nhãn cũng khó định lượng mức độ gần gũi ngữ nghĩa với độ chính xác số tuyệt đối. Một giải pháp thực tế

[19] Trong thực tế, người ta thường chuyển nó thành *khoảng cách cosin* (*cosine distance*), định nghĩa là $1 - \cos(\theta)$, trong đó khoảng cách bằng 0 nghĩa là hai vectơ chỉ đúng cùng một hướng, còn bằng 1 nghĩa là hai vectơ vuông góc.

là ánh xạ khoảng cách vào các hạng mục rời rạc hoặc các thang đánh giá kiểu Likert, chẳng hạn các số nguyên trong $[1, 5]$ [Reisinger và Mooney, 2010]. Điều này giảm đáng kể gánh nặng nhận thức trong quá trình gán nhãn dữ liệu. Một biến thể đánh giá khác yêu cầu mô hình xác định từ giống nhất với một từ đích cho trước trong một tập nhỏ các ứng viên (ví dụ bài kiểm tra từ đồng nghĩa). Cách này tránh việc phải dự đoán giá trị tuyệt đối. Thay vào đó, ta chỉ cần một cơ chế xếp hạng các độ tương tự tương đối [Baroni và cộng sự, 2014].

Tuy nhiên, việc đánh giá quan hệ giữa các từ có thể mơ hồ do bản chất linh hoạt của ngôn ngữ. Nói chung, nhiều yếu tố ngôn ngữ học ảnh hưởng tới phán đoán của con người về độ liên quan [Faruqui và cộng sự, 2016]. Ví dụ, *corn* (ngô) và *cornea* (giác mạc) giống nhau về hình thái do trùng một phần từ con, nhưng về ngữ nghĩa lại hoàn toàn không liên quan. Sự mơ hồ cũng bắt nguồn từ chính định nghĩa của các bài toán. **Độ liên quan ngữ nghĩa** (*semantic relatedness*) và **độ tương tự ngữ nghĩa** (*semantic similarity*) thường được dùng thay thế cho nhau, nhưng chúng biểu thị những khái niệm ngôn ngữ học khác nhau. Chẳng hạn, *car* (xe hơi) liên quan về chủ đề với *road* (con đường), nhưng theo nghĩa phân loại chặt chẽ thì *car* tương tự với *van* (xe tải nhỏ).

Hơn nữa, nghĩa của một từ nói chung phụ thuộc vào ngữ cảnh, và do đó gây nhiễu cho việc đánh giá những từ có nhiều nghĩa khác nhau (**đa nghĩa** (*polysemy*)). Nói rộng ra, đây là một hạn chế cố hữu của các mô hình vectơ từ tĩnh, trong đó mỗi loại từ được ánh xạ một cách tất định tới một vectơ cố định duy nhất. Để giải quyết vấn đề này, các chương sau sẽ giới thiệu các vectơ nhúng từ theo ngữ cảnh (*contextualized word embeddings*), vốn sinh ra một cách động một dạng tính toán riêng cho một từ dựa hoàn toàn vào văn bản xung quanh nó.

Chú giải của người dịch: Ví dụ tiếng Việt về đa nghĩa

Từ "*đường*" trong "*bỏ thêm đường vào cà phê*" và trong "*con đường làng*" có hai nghĩa hoàn toàn khác. Một mô hình vectơ từ tĩnh (như Word2Vec, GloVe) chỉ có *một* vectơ cho "đường", nên vectơ này buộc phải là một kiểu "trộn lẫn" của cả hai nghĩa, vừa hơi gần "muối", vừa hơi gần "phố". Vectơ nhúng theo ngữ cảnh (sẽ học ở các chương sau) tạo ra vectơ khác nhau cho "đường" tùy câu chứa nó.

2. Loại suy từ

Bài toán loại suy từ (*word analogy*) liên quan đến việc mô hình hóa quan hệ tương tự giữa các cặp từ. Bài toán này xuất phát từ tiền đề rằng các quan hệ ngữ nghĩa giữa các từ có thể được nắm bắt bằng những phép toán đại số đơn giản trên các biểu diễn liên tục tương ứng. Một ví dụ kinh điển do Mikolov và cộng sự [2013d] đưa ra cho thấy ngữ nghĩa quan hệ có thể được mô tả bằng phép trừ vectơ. Cụ thể, lấy vectơ của *king* (vua) trừ đi vectơ của *man* (đàn ông) rồi cộng vectơ của *woman* (phụ nữ) sẽ cho một vectơ rất gần với vectơ của *queen* (nữ hoàng):

$$\mathbf{e}_{\text{king}} - \mathbf{e}_{\text{man}} + \mathbf{e}_{\text{woman}} \approx \mathbf{e}_{\text{queen}}. \tag{3.61}$$

Một cách hình thức, loại suy từ là bài toán so sánh hai cặp từ (a, a^*) và (b, b^*). Một phép loại suy đúng nếu quan hệ ngữ nghĩa giữa a và a^* tương ứng với quan hệ giữa b và b^*. Điều này phản ánh những **quy luật ngôn ngữ** (*linguistic regularities*) tiềm ẩn trong không gian vectơ, có thể được phát biểu bằng đại số là:

$$\mathbf{e}_{a^*} - \mathbf{e}_a \approx \mathbf{e}_{b^*} - \mathbf{e}_b. \tag{3.62}$$

Như vậy, loại suy từ có thể được đặt thành một bài toán suy luận tương tự: mục tiêu là dự đoán $\mathbf{e}_{b^*}$ khi biết $\mathbf{e}_a$, $\mathbf{e}_{a^*}$ và $\mathbf{e}_b$. Cụ thể hơn, ta kỳ vọng vectơ $\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$ xấp xỉ sát $\mathbf{e}_{b^*}$ nếu (a, a^*) và (b, b^*) có quan hệ ngôn ngữ tương tự nhau. Có nhiều cách cải tiến cách phát biểu này. Chẳng hạn, thay vì chỉ tối ưu khoảng cách không gian để dự đoán $\mathbf{e}_{b^*}$, ta có thể đánh giá cosin của góc giữa các vectơ hiệu $\mathbf{e}_{a^*} - \mathbf{e}_a$ và $\mathbf{e}_{b^*} - \mathbf{e}_b$ để nắm bắt độ tương tự quan hệ một cách vững chắc hơn [Levy và Goldberg, 2014b].

Chú giải của người dịch: Phép loại suy như hình bình hành

Phương trình (3.62) nói rằng "mũi tên" đi từ a đến a^* gần bằng "mũi tên" đi từ b đến b^*; bốn điểm tạo thành gần như một hình bình hành, đúng như quy tắc hình bình hành của vectơ ở lớp 10. Ví dụ với $a = \textit{man}$, $a^* = \textit{woman}$, $b = \textit{king}$: vectơ $\mathbf{e}_{\text{woman}} - \mathbf{e}_{\text{man}}$ như mã hóa hướng "nam $\rightarrow$ nữ"; cộng hướng đó vào $\mathbf{e}_{\text{king}}$ ta đến gần $\mathbf{e}_{\text{queen}}$. Tương tự có thể thử "Hà Nội – Việt Nam + Nhật Bản $\approx$ Tokyo" (quan hệ thủ đô). Trong thực tế, người ta tìm từ có vectơ gần nhất (theo cosin) với $\mathbf{e}_{a^*} - \mathbf{e}_a + \mathbf{e}_b$, loại trừ chính các từ a, a^*, b.

Bài toán loại suy từ cung cấp một cơ chế đơn giản để kiểm tra các tính chất đại số tuyến tính của một mô hình vectơ từ. Những đặc điểm như vậy

hiếm khi quan sát được ở các phương pháp dựa trên đếm truyền thống. Đáng chú ý là các mô hình nhúng từ hiện đại thể hiện hành vi tuyến tính rõ rệt dù các ràng buộc hình học như vậy không được áp đặt tường minh trong quá trình huấn luyện. Tính chất tự phát này cho các nhà nghiên cứu cái nhìn sâu sắc về những cấu trúc tiềm ẩn mà các mô hình này học được và các ứng dụng hạ nguồn tiềm năng của chúng [Levy và Goldberg, 2014b; Linzen, 2016; Allen và Hospedales, 2019]. Tuy nhiên, độ chính xác loại suy từ không phải là một thước đo có tính chỉ báo phổ quát. Trong nhiều tình huống, nó không tương quan mạnh với hiệu năng của các hệ thống NLP hạ nguồn. Vì vậy, nó thường được dùng như một công cụ phân tích để thăm dò các tính chất cấu trúc và ngữ nghĩa của biểu diễn từ.

3. Phân loại từ theo nhóm (hay phân cụm)

Một phương pháp khác để đánh giá xem vectơ từ có khớp với hiểu biết ngữ nghĩa của con người hay không là kiểm tra khả năng chúng được nhóm vào các hạng mục có ý nghĩa. Việc này thường được thực hiện bằng cách áp dụng các thuật toán phân cụm (*clustering*) tiêu chuẩn lên các vectơ từ. Lý tưởng nhất, các từ tương tự nhau về ngữ nghĩa nên được xếp vào cùng một cụm, còn các từ khác nhau được tách vào các cụm riêng. Ví dụ, *apple* (táo), *grape* (nho), *peach* (đào) và *orange* (cam) nên tự nhiên được gom vào cùng một cụm thuộc hạng mục ngữ nghĩa rộng hơn là trái cây. Một ưu điểm của cách đánh giá này là có thể áp dụng ngay nhiều thuật toán phân cụm và các bộ đánh giá chuẩn về phân loại từ đã có sẵn. Tuy nhiên, như với hầu hết các bài toán phân cụm không giám sát, cần giải quyết một số thách thức thực tế, chẳng hạn xác định trước số cụm tối ưu.

Vì phần lớn các phương pháp phân cụm trong học máy dựa trên việc tính khoảng cách không gian giữa các điểm dữ liệu, phân cụm từ bắt nguồn từ khái niệm độ liên quan ngữ nghĩa. Tuy nhiên, khác với các bài toán đã bàn ở Mục 3.6.2, phân cụm đánh giá cấu trúc toàn cục thay vì phán xét tường minh từng điểm số khoảng cách theo cặp. Điều này cho thấy mối liên hệ nội tại giữa các quy trình đánh giá khác nhau, nhưng cũng có nghĩa là phân cụm từ mang những hạn chế cố hữu của các phương pháp liên quan đó. Đáng chú ý nhất, rất khó thiết lập một tiêu chí chuẩn tuyệt đối cho chất lượng phân cụm, vì các hệ phân loại ngữ nghĩa mang tính chủ quan, và các từ có thể được phân loại theo nhiều cách tùy vào góc nhìn ngữ nghĩa được chọn.

4. Đánh giá tiềm thức

Mục tiêu của đánh giá tiềm thức (*subconscious evaluation*) (hay đánh giá nhận thức) là xem xét mối tương quan giữa việc sử dụng vectơ từ và các hành vi tiềm thức hay hoạt động não bộ của con người trong khi đọc. Rất nhiều hiện tượng ngôn ngữ học tâm lý có thể dùng làm đại lượng đánh giá gián tiếp [Mitchell và Lapata, 2010]. Một mô thức nổi tiếng là **mồi** (*priming*), nghiên cứu cách một người phản ứng với các kích thích nối tiếp nhau [Schacter và Buckner, 1998; Tulving và Schacter, 1990; Wiggs và Martin, 1998]. Ví dụ, có thể thiết kế một thí nghiệm đo tốc độ một người tham gia đọc một từ cho trước (gọi là **từ đích** (*target word*)) ngay sau một từ khác (**từ mồi** (*prime word*)) [Meyer và Schvaneveldt, 1971; Lund, 1995; McNamara, 2005]. Nếu từ đích t được xử lý nhanh hơn khi đứng sau từ a so với khi đứng sau từ b, điều đó cho thấy liên kết nhận thức giữa t và a mạnh hơn. Độ trễ tâm lý này khi đó có thể dùng làm đại lượng gián tiếp để đánh giá độ tương tự ngữ nghĩa giữa các vectơ từ tương ứng. Để đo độ trễ khi đọc, một phương pháp phổ biến là bài toán **đọc tự điều chỉnh nhịp** (*self-paced reading*)[20]. Một cách khác là dùng thiết bị theo dõi mắt để tự động ghi lại thông tin chính xác về chuyển động và điểm dừng của mắt. Nhờ các kỹ thuật này, nhiều tập dữ liệu và phương pháp đã được phát triển để nghiên cứu nhiều vấn đề ngôn ngữ học tâm lý khác nhau [Mitchell và Lapata, 2010; Hutchison và cộng sự, 2013; Lapesa và Evert, 2014; Klerke và cộng sự, 2015; Søgaard, 2016; Auguste và cộng sự, 2017].

Ngoài việc theo dõi phản ứng hành vi trong khi đọc, ta có thể trực tiếp theo dõi hoạt động não bằng các kỹ thuật chụp ảnh thần kinh, chẳng hạn chụp cộng hưởng từ chức năng (fMRI) và điện não đồ (EEG) [Devereux và cộng sự, 2010; Søgaard, 2016; Bhattasali và cộng sự, 2020]. Ví dụ, việc hiểu ngôn ngữ gây ra những mẫu kích hoạt thần kinh cụ thể trong não. Do đó, ta có thể thử tìm tương quan trực tiếp giữa các mẫu kích hoạt sinh lý này với dạng tính toán của nghĩa từ do biểu diễn vectơ cung cấp. Tuy nhiên, một hạn chế là hiểu biết của chúng ta về các cơ chế nhận thức nằm bên dưới những quá trình này vẫn chưa đầy đủ, khiến việc liên hệ thỏa đáng kết quả của các nghiên cứu ngôn ngữ học tâm lý như vậy với hiệu năng của các hệ thống NLP thực tế trở nên khó khăn [Baroni và cộng

[20]Trong đọc tự điều chỉnh nhịp, văn bản được chia thành các từ hoặc cụm từ, và người tham gia nhấn một nút để hiển thị lần lượt từng đoạn.

sự, 2014; Bakarov, 2018].

Chú giải của người dịch: Hiệu ứng mồi trong đời thường

Nếu bạn vừa đọc chữ "*bác sĩ*", bạn sẽ nhận ra chữ "*y tá*" nhanh hơn một chút (vài chục mili giây) so với khi vừa đọc chữ "*bàn ghế*". Não đã được "mồi" sẵn cho các khái niệm liên quan. Ý tưởng đánh giá: nếu vectơ của "bác sĩ" gần "y tá" hơn "bàn ghế", thì vectơ từ phản ánh đúng cách con người liên kết các khái niệm.

5. Đánh giá dựa trên ngôn ngữ học

Đánh giá dựa trên ngôn ngữ học xuất phát từ giả định rằng các vectơ từ được học từ dữ liệu nên phản ánh tri thức cấu trúc và ngữ nghĩa được mã hóa trong các tài nguyên ngôn ngữ học. Một cách thú vị để thực hiện việc đánh giá này là căn chỉnh các vectơ từ với biểu diễn của các mục từ trong từ điển [Tsvetkov và cộng sự, 2015; Acs và Kornai, 2016]. Chất lượng của vectơ nhúng từ khi đó được định lượng bằng mức tương quan giữa các vectơ liên tục này với các biểu diễn ngôn ngữ học rời rạc[21]. Ngoài các từ điển tiêu chuẩn, vectơ từ có thể được đánh giá theo cấu trúc tô-pô trong các mạng ngữ nghĩa, chẳng hạn WordNet. Bằng cách này, các thước đo đánh giá có thể được làm phong phú hơn nhờ dùng các thuật toán trên đồ thị để đo mức độ khớp về cấu trúc giữa không gian vectơ và các tài nguyên ngữ nghĩa này [Agirre và cộng sự, 2009].

3.6.3 Trực quan hóa

Bằng cách coi các vectơ từ là những điểm dữ liệu nhiều chiều, ta có thể dùng các kỹ thuật giảm chiều để chiếu chúng xuống không gian hai hoặc ba chiều nhằm trực quan hóa. Các phép chiếu như vậy cho phép ta phân tích trực quan những khuôn mẫu ngữ nghĩa được mã hóa trong các vectơ này và khám phá các quan hệ tiềm ẩn giữa các từ. Một phương pháp là PCA, vốn tìm một ánh xạ tuyến tính từ không gian nhiều chiều xuống không gian ít chiều (xem Mục 3.3.3). Một phương pháp nổi tiếng khác là **nhúng láng giềng ngẫu nhiên phân phối t** (*t-distributed stochastic neighbor embedding*) (**t-SNE**) [Hinton và Roweis, 2002; Van der Maaten

[21] Trong ngữ cảnh này, biểu diễn ngôn ngữ học thường là một vectơ đặc trưng được xây dựng thủ công từ một tài nguyên ngôn ngữ học chính quy, chẳng hạn một từ điển từ vựng.

và Hinton, 2008]. Đây là một phương pháp giảm chiều phi tuyến, và đã được dùng rộng rãi trong trực quan hóa dữ liệu nhiều chiều. Ngoài ra, ta có thể xem xét các phương pháp trình bày trong Mục 3.3.3 cũng như các phương pháp được thiết kế riêng cho việc trực quan hóa vectơ từ [Zhang và cộng sự, 2019b; Liu và cộng sự, 2017].

3.7 Tóm tắt

Trong chương này, chúng ta đã bàn về hai vấn đề thú vị trong NLP: tách token và biểu diễn từ (hay biểu diễn token). Thứ nhất, chúng tôi đã giới thiệu các mô hình chia một câu thành những đơn vị có nghĩa và/hoặc phù hợp với các bài toán hạ nguồn. Thứ hai, chúng tôi đã giới thiệu ý tưởng về các mô hình vectơ từ, đặc biệt chú trọng vào việc học cả các mô hình nhiều chiều dựa trên đếm lẫn các mô hình ít chiều giá trị thực. Mặc dù phần lớn các mô hình này đơn giản, chúng thường được dùng trong các hệ thống NLP phức tạp và là nền tảng của nhiều mô hình tiên tiến, như sẽ thấy ở các chương sau.

Tách token (hay phân đoạn) là một quá trình quan trọng trong NLP, thường là bước tiền xử lý cho nhiều ứng dụng [Webster và Kit, 1992]. Tuy nhiên, việc dùng thuật ngữ *tách token* (*tokenization*) hơi dễ gây hiểu lầm vì ban đầu nó chỉ quá trình cơ bản trong khoa học máy tính là chia một xâu thành các xâu con. Trong NLP hiện đại, tách token dựa trên các khái niệm từ nhiều lĩnh vực con. Về mặt ngôn ngữ học, nó gắn chặt với hai câu hỏi cốt lõi: các từ được cấu tạo về hình thái như thế nào và chúng kết hợp về cú pháp thành câu ra sao. Vì vậy, việc dùng các lý thuyết hình thái học và cú pháp học để định nghĩa các đơn vị cơ bản của một ngôn ngữ là điều tự nhiên, dẫn tới nhiều hệ thống tách token dựa trên quy tắc cho nhiều ngôn ngữ khác nhau. Từ góc nhìn học máy, tách token từ lâu đã được xem là bài toán học ranh giới token từ dữ liệu theo cách có giám sát hoặc không giám sát [Mielke và cộng sự, 2021]. Một cách tiếp cận phổ biến là trước hết gán nhãn một ít văn bản đã tách token bằng tri thức của con người về việc các đơn vị ngôn ngữ cơ bản nên là gì, rồi học cách tách token trên dữ liệu đã gán nhãn này (xem Mục 3.1.3). Gần đây hơn, việc học các bộ tách token không có ràng buộc ngôn ngữ học được thấy là đầy hứa hẹn (xem Mục 3.1.4). Vì bản thân ngôn ngữ tự nhiên là tập các chuỗi ký tự hay chuỗi byte, ta cũng có thể chia một câu thành các ký tự hoặc byte [Ling và

cộng sự, 2015; Lee và cộng sự, 2017]. Các phương pháp không cần tách token nói chung hữu ích khi ta muốn một bộ tách token độc lập với ngôn ngữ và một quy trình xử lý văn bản đơn giản hơn.

Từ góc độ toán học, tách token có thể được xem là một ánh xạ từ dữ liệu đầu vào sang một chuỗi biến. Do đó, khái niệm tách token có thể được tổng quát hóa bằng cách nới lỏng giả định rằng cả biến đầu vào lẫn đầu ra đều bị giới hạn ở các giá trị rời rạc. Chẳng hạn, các hệ thống xử lý ảnh và tiếng nói gần đây biến dữ liệu cảm quan liên tục (như điểm ảnh và tín hiệu âm thanh) thành các chuỗi "token" dạng vectơ [Schneider và cộng sự, 2019; Dosovitskiy và cộng sự, 2021]. Một số mở rộng thú vị của những ý tưởng này thậm chí còn biến dữ liệu ảnh và tiếng nói thành một chuỗi các chỉ số rời rạc, qua đó đưa thị giác máy tính và xử lý tiếng nói đến gần hơn với các mô thức NLP truyền thống [Oord và cộng sự, 2017; Baevski và cộng sự, 2020; Hsu và cộng sự, 2021].

Sau khi chia văn bản đầu vào thành các đoạn nhỏ hơn, bước tự nhiên tiếp theo là biểu diễn những mảnh này theo cách nào đó nắm bắt được các đặc trưng bên dưới của chúng. Mặc dù biểu diễn các đơn vị ngôn ngữ bằng các vectơ số dày đặc đã trở thành tiêu chuẩn trên thực tế trong NLP hiện đại, các nghiên cứu về biểu diễn vectơ đã có từ những ngày đầu của ngôn ngữ học tính toán. Theo nhiều giáo trình và bài báo phổ biến [Manning và Schütze, 1999; Jurafsky và Martin, 2008], ý tưởng dùng một phân bố thống kê để biểu diễn nghĩa của từ, được gọi là **ngữ nghĩa học phân bố**, xuất hiện vào những năm 1950 cùng với sự trỗi dậy của chủ nghĩa kinh nghiệm. Trong thời kỳ đó, phần lớn các nghiên cứu chịu ảnh hưởng mạnh mẽ của thuyết phân bố của Harris [Harris, 1954] và các khuôn khổ triết học liên quan [Firth, 1957; Wittgenstein, 1953]. Cùng thời gian đó, Osgood [1952] đề xuất định nghĩa nghĩa của một khái niệm theo góc độ tâm lý học, như một điểm trong một không gian nhiều chiều. Những ý tưởng này đã định hình đáng kể cách các nhà ngôn ngữ học và các nhà nghiên cứu NLP quan niệm về nghĩa của từ trong những thập kỷ sau đó.

Các cách tiếp cận hiện đại cho ngữ nghĩa học phân bố xuất hiện vào những năm 1990, chủ yếu là kết quả của sự hồi sinh chủ nghĩa kinh nghiệm trong trí tuệ nhân tạo [Church, 2011]. Phần lớn chúng được dẫn dắt bởi giả thuyết phân bố: các từ có nghĩa tương tự nhau thì có nhiều khả năng xuất hiện trong những ngữ cảnh tương tự nhau. Theo đó, nhiều phương pháp đã được phát triển, khác nhau ở cách mô hình hóa ngữ cảnh. Ví dụ,

ngữ cảnh có thể là các từ trong một cửa sổ ngữ cảnh, hoặc các từ có quan hệ với từ đã cho trong một cây cú pháp. Ngoài những phương pháp đã nêu ở Mục 3.3, các phương pháp không được đề cập trong chương này gồm có mô hình tương tự siêu không gian của ngôn ngữ (*hyperspace analogue of language*) (HAL) [Lund và Burgess, 1996], bộ nhớ phân bố (*distributional memory*) [Baroni và Lenci, 2010], các mô hình không gian ngữ nghĩa dựa trên quan hệ phụ thuộc [Padó và Lapata, 2007], v.v. Để có cái nhìn tổng quan đầy đủ về ngữ nghĩa học phân bố, bạn đọc có thể tham khảo các tài liệu chuyên sâu [Lenci, 2018; Mitchell và Lapata, 2010]. Lưu ý rằng phần lớn các công trình nêu trên có thể được xem là những trường hợp của mô hình không gian vectơ (*vector space model*), vốn có thể giải quyết các vấn đề vượt ra ngoài ngữ nghĩa từ vựng. Ví dụ, trong ngữ nghĩa học phân bố kết hợp (*compositional distributional semantics*), nghĩa của một cụm từ hay một câu được suy ra bằng cách thực hiện các phép toán đại số có hệ thống trên các vectơ từ thành phần của nó [Clark và cộng sự, 2008; Mitchell và Lapata, 2010; Blacoe và Lapata, 2012].

Mặc dù các mô hình phân bố đã thu hút sự chú ý của cộng đồng NLP trong nhiều năm, các mô hình nhúng từ học các vectơ từ ít chiều, giá trị thực trực tiếp từ văn bản gần đây đã trở thành cách tiếp cận chủ đạo. Như đã mô tả trong Mục 3.4–3.5, thay vì xây dựng tường minh các ma trận đồng xuất hiện thưa, các mô hình này tận dụng ngầm giả thuyết phân bố bằng cách tối ưu một vectơ các thuộc tính tiềm ẩn để dự đoán các ngữ cảnh xung quanh. Mô hình thu được là một mở rộng của mô hình ngữ nghĩa dựa trên đặc trưng [Markman, 2013]. Một khác biệt được thừa nhận so với các phương pháp dựa trên đặc trưng truyền thống là ta không cần định nghĩa thủ công các đặc trưng. Thay vào đó, ta coi các đặc trưng này là tham số của mô hình và huấn luyện chúng theo kiểu đầu-cuối.

Việc phát biểu biểu diễn từ thành một bài toán học đầu-cuối mang lại một số lợi ích. Ưu điểm chính là có thể tìm ra những đặc trưng mới, vì không có ràng buộc nào đặt lên cách học và diễn giải các đặc trưng này. Hơn nữa, như đã trình bày ở Mục 3.6.2, các vectơ từ thu được theo cách này thực sự thể hiện một số tính chất ngôn ngữ học, dù các mô hình nhúng từ không được huấn luyện để đạt điều đó. Một lợi ích khác là các mô hình nhúng từ cũng thuộc mô thức rộng hơn của mô hình không gian vectơ, giúp dễ dàng sử dụng vectơ từ trong nhiều ứng dụng. Cũng có nhiều ví dụ về các phương pháp cố gắng cải tiến các hệ thống nhúng từ tiêu chuẩn.

Các nhà nghiên cứu đã thành công trong việc bổ sung tri thức ngôn ngữ học bên ngoài vào các vectơ nhúng đã tiền huấn luyện [Levy và Goldberg, 2014a; Cotterell và Schütze, 2015; Tissier và cộng sự, 2017] và căn chỉnh các không gian vectơ để thu được những biểu diễn phổ quát cho nhiều ngôn ngữ [Klementiev và cộng sự, 2012; Mikolov và cộng sự, 2013b; Ammar và cộng sự, 2020; Smith và cộng sự, 2017; Artetxe và cộng sự, 2017].

Gắn liền với các mô hình nơ-ron trong NLP, ý tưởng biểu diễn phân tán (*distributed representation*) đã được tổng quát hóa thành công cho những đơn vị cấu trúc vượt xa các từ riêng lẻ. Điều này bao gồm việc học biểu diễn dày đặc cho câu [Le và Mikolov, 2014; Kalchbrenner và cộng sự, 2014; Kiros và cộng sự, 2015; Hill và cộng sự, 2016; Arora và cộng sự, 2017; Lin và cộng sự, 2017; Conneau và cộng sự, 2017b] cũng như cho các cấu trúc cây và đồ thị phức tạp [Socher và cộng sự, 2011; Perozzi và cộng sự, 2014; Tai và cộng sự, 2015; Grover và Leskovec, 2016]. Đặc biệt, các biểu diễn theo ngữ cảnh của từ, dù không được bàn trong chương này, nhìn chung được đánh giá cao trong việc mô hình hóa dữ liệu dạng chuỗi [McCann và cộng sự, 2017; Peters và cộng sự, 2018; Devlin và cộng sự, 2019].

Bản dịch tiếng Việt có chú giải (không chính thức, phi thương mại) của *Natural Language Processing: Neural Networks and Large Language Models*, © 2021–2026 Tong Xiao và Jingbo Zhu, `https://github.com/NiuTrans/NLPBook`. Nguyên tác và bản dịch theo giấy phép CC BY-NC 4.0.